



C++函数使用手册

Version 1.0.1

遨博（北京）智能科技有限公司

使用手册会定期进行检查和修正，更新后的内容将出现在新版本中。本手册中的内容或信息如有变更，恕不另行通知。

对本手册中可能出现的任何错误或遗漏，或因使用本手册及其中所述产品而引起的意外或间接伤害，遨博（北京）智能科技有限公司概不负责。

安装、使用产品前，请阅读本手册。

请保管好本手册，以便可以随时阅读和参考。

本手册为遨博（北京）智能科技有限公司专有财产，非经遨博（北京）智能科技有限公司书面许可，不得复印、全部或部分复制或转变为任何其他形式使用。

Copyright © 2015-2022 AUBO 保留所有权利。

目录

目录	3
简介	9
1 数据类型.....	10
1.1 机械臂运动相关的类型.....	10
1.2 机械臂关节状态.....	14
1.3 事件类型.....	14
1.4 诊断信息.....	22
1.5 回调函数类型.....	23
1.6 工具参数类型.....	24
1.7 工具标定.....	25
1.8 坐标系标定.....	26
1.9 IO 相关数据类型.....	28
2 接口函数.....	32
2.1 机械臂系统接口	32
2.1.1 登录.....	32
2.1.2 退出登录.....	32
2.1.3 启动机械臂.....	33
2.1.4 关机.....	33
2.2 状态推送.....	34
2.2.1 设置是否允许实时关节状态推送.....	34
2.2.2 注册用于获取关节状态的回调函数.....	34
2.2.3 设置是否允许实时路点信息推送.....	34
2.2.4 注册用于获取实时路点的回调函数.....	34
2.2.5 设置是否允许实时末端速度推送.....	35
2.2.6 注册用于获取实时末端速度的回调函数.....	35
2.2.7 注册用于获取机械臂事件信息的回调函数.....	35
2.2.8 注册 movep 进度通知的回调函数.....	36
2.2.9 注册日志输出回调函数.....	36
2.3 机械臂运动相关的接口	37
2.3.1 初始化全局的运动属性.....	37
2.3.2 设置关节型运动的最大加速度.....	38
2.3.3 设置关节型运动的最大速度.....	38
2.3.4 获取关节型运动的最大加速度.....	38
2.3.5 获取关节型运动的最大速度.....	39
2.3.6 设置末端型运动的最大线加速度.....	39
2.3.7 设置末端型运动的最大线速度.....	39
2.3.8 获取末端型运动的最大线加速度.....	39
2.3.9 获取末端型运动的最大线速度.....	40
2.3.10 设置末端型运动的最大角加速度.....	40
2.3.11 设置末端型运动的最大角速度.....	40
2.3.12 获取末端型运动的最大角加速度.....	40

2.3.13	获取末端型运动的最大角速度.....	40
2.3.14	设置加加速度.....	41
2.3.15	获取加加速度.....	41
2.3.16	清除路点容器.....	41
2.3.17	添加路点.....	41
2.3.18	获取路点容器.....	42
2.3.19	设置交融半径.....	42
2.3.20	获取交融半径.....	42
2.3.21	获取圆轨迹时圆的圈数.....	42
2.3.22	设置圆轨迹时圆的圈数.....	42
2.3.23	设置运动属性中的偏移属性.....	43
2.3.24	设置无提前到位.....	43
2.3.25	设置提前到位距离模式.....	43
2.3.26	设置提前到位时间模式.....	44
2.3.27	设置提前到位交融半径模式.....	44
2.3.28	关节运动.....	44
2.3.29	保持当前位姿通过关节运动的方式运动到目标位置, 其中目标位置是 通过相对当前位置的偏移给出的.....	45
2.3.30	保持当前位姿通过关节运动的方式运动到目标位置.....	45
2.3.31	基于跟随模式的轴动.....	46
2.3.32	直线运动.....	46
2.3.33	保持当前位姿通过直线运动的方式运动到目标位置, 其中目标位置是 通过相对当前位置的偏移给出.....	47
2.3.34	保持当前位姿通过直线运动的方式运动到目标位置.....	48
2.3.35	保持当前位置变换姿态做旋转运动.....	48
2.3.36	根据当前位姿及基坐标系下表示的旋转轴、旋转角, 获取目标位姿 49	
2.3.37	将用户坐标系描述的旋转轴变换到基坐标系下描述.....	49
2.3.38	旋转运动到目标路点.....	49
2.3.39	轨迹运动.....	50
2.3.40	开始示教.....	50
2.3.41	设置示教运动的坐标系.....	51
2.3.42	结束示教.....	51
2.3.43	清除离线轨迹路点.....	51
2.3.44	添加离线轨迹路点.....	51
2.3.45	开始离线轨迹运动.....	52
2.3.46	结束离线轨迹运动.....	52
2.3.47	进入 TCP 转 CAN 透传模式.....	52
2.3.48	发送坐标数据到关节 CAN 总线.....	52
2.3.49	退出 TCP 转 CAN 透传模式.....	53
2.4	工具接口.....	53
2.4.1	正解.....	53
2.4.2	逆解.....	53
2.4.3	基坐标系转用户坐标系.....	54

2.4.4	基坐标系转基坐标得到工具末端点的位置和姿态	55
2.4.5	用户坐标系位置和姿态信息转基坐标系下的位置和姿态	55
2.4.6	将空间内一个基于用户坐标系的位置信息 (x, y, z) 转换成基于基坐标系下的位置信息 (x, y, z)	56
2.4.7	法兰盘姿态转成工具姿态	57
2.4.8	工具姿态转成法兰盘姿态	57
2.4.9	根据位置获取目标路点信息	57
2.4.10	四元数转欧拉角	58
2.4.11	欧拉角转四元数	58
2.4.12	根据错误号获取错误信息	58
2.5	机械臂控制接口	59
2.5.1	机械臂运动控制：停止、暂停、继续	59
2.5.2	机械臂控制	59
2.5.3	机械臂快速停止	59
2.5.4	机械臂运动停止	59
2.5.5	设置机械臂的电源状态	60
2.5.6	松刹车	60
2.6	末端工具接口	60
2.6.1	设置无工具的动力学参数	60
2.6.2	设置工具的动力学参数	60
2.6.3	获取工具的动力学参数	61
2.6.4	设置无工具的运动学参数	61
2.6.5	设置工具的运动学参数	61
2.6.6	获取工具的运动学参数	61
2.7	设置和获取机械臂相关参数接口	62
2.7.1	获取当前的连接状态	62
2.7.2	设置当前机械臂模式：仿真或真实	62
2.7.3	获取当前机械臂模式	62
2.7.4	获取重力分量	62
2.7.5	获取当前碰撞等级	63
2.7.6	设置碰撞等级	63
2.7.7	获取设备信息	63
2.7.8	获取是否存在真实机械臂	63
2.7.9	获取 6 关节旋转 360 使能标志	64
2.7.10	获取机械臂关节状态	64
2.7.11	获取机械臂诊断信息	64
2.7.12	获取机械臂当前关节角信息	64
2.7.13	获取机械臂当前路点信息	65
2.7.14	当前机械臂是否运行在联机模式	65
2.7.15	当前机械臂是否运行在联机主模式	65
2.7.16	获取机械臂当前运行状态	65
2.7.17	获取 MAC 通信状态	65
2.7.18	获取机械臂安全配置	66
2.8	接口板 IO 相关的接口	66

2.8.1	获取接口板指定 IO 集合的配置信息	66
2.8.2	获取接口板指定 IO 集合的状态信息	66
2.8.3	设置接口板 IO 状态	67
2.9	工具 IO 相关的接口	68
2.9.1	设置工具端电源电压类型	68
2.9.2	获取工具端电源电压类型	68
2.9.3	获取工具端的电源电压	68
2.9.4	设置工具端电源电压类型和所有数字量 IO 的类型	68
2.9.5	设置工具端数字量 IO 的类型：输入或者输出	69
2.9.6	获取工具端所有数字量 IO 的状态	69
2.9.7	根据地址设置工具端数字量 IO 的状态	69
2.9.8	根据名称设置工具端数字量 IO 的状态	69
2.9.9	根据名称获取工具端 IO 的状态	70
2.9.10	获取工具端所有 AI 的状态	70
3	错误码	71
3.1	接口函数错误码定义	71
3.2	由于控制器异常事件导致的错误码	72
3.3	由于硬件层异常事件导致的错误码	73
4	使用案例	77
4.1	使用 SDK 构建一个最简单的机械臂的控制工程	77
4.2	用回调函数的方式来获取实时路点、末端速度、机械臂事件、关节状态	79
4.3	正逆解	83
4.4	坐标系转换	87
4.4.1	基坐标系转用户坐标系 baseToUserCoordinate()	87
	baseToUserCoordinate 函数示例 1	87
	baseToUserCoordinate 函数示例 2	90
	baseToUserCoordinate 函数示例 3	94
4.4.2	基坐标系转基坐标得到工具末端点的位置姿态 baseToBaseAdditionalTool()	96
	baseToBaseAdditionalTool 函数示例	96
4.4.3	用户坐标系转基坐标系 userToBaseCoordinate()	99
	userToBaseCoordinate 函数示例 1	99
	userToBaseCoordinate 函数示例 2	102
	userToBaseCoordinate 函数示例 3	106
4.4.4	用户坐标系下位置参数转基坐标系 userCoordPointToBasePoint()	108
	userCoordPointToBasePoint 函数示例	108
4.4.5	法兰盘姿态转工具姿态 endOrientation2ToolOrientation()	111
	endOrientation2ToolOrientation 函数示例	111
4.4.6	工具姿态转法兰盘姿态 toolOrientation2EndOrientation()	113
	toolOrientation2EndOrientation 函数示例	113
4.5	设置和获取机械臂相关参数	115
4.6	IO	122
4.6.1	工具 IO	122
4.6.2	用户 IO	127

4.6.3	安全 IO.....	131
4.7	TCP 转 CAN 透传模式.....	134
4.8	关节运动.....	136
4.8.1	robotServiceJointMove 函数.....	136
4.8.2	robotMoveJointToTargetPositionByRelative 函数.....	138
	示例 1: 法兰盘中心在基坐标系下偏移.....	138
	示例 2: 法兰盘中心在用户坐标系下偏移.....	141
	示例 3: 工具末端在工具坐标系下偏移.....	144
	示例 4: 工具末端在基坐标系下偏移.....	146
	示例 5: 工具末端在用户坐标系下偏移.....	149
4.8.3	robotMoveJointToTargetPosition 函数.....	152
	示例 1: 法兰盘中心在基坐标系下的位置.....	152
	示例 2: 法兰盘中心在用户坐标系下的位置.....	155
	示例 3: 工具末端在基坐标系下的位置.....	158
	示例 4: 工具末端在用户坐标系下的位置.....	161
4.9	跟随模式.....	164
4.9.1	跟随模式的轴动 robotServiceFollowModeJointMove.....	164
4.9.2	跟随模式之提前到位.....	166
4.10	直线运动.....	170
4.10.1	robotServiceLineMove 函数.....	170
4.10.2	robotMoveLineToTargetPositionByRelative 函数.....	172
	示例 1: 法兰盘中心在基坐标系下偏移.....	172
	示例 2: 法兰盘中心在用户坐标系下偏移.....	175
	示例 3: 工具末端在工具坐标系下偏移.....	178
	示例 4: 工具末端在基坐标系下偏移.....	180
	示例 5: 工具末端在用户坐标系下偏移.....	183
4.10.3	robotMoveLineToTargetPosition 函数.....	186
	示例 1: 法兰盘中心在基坐标系下的位置.....	186
	示例 2: 法兰盘中心在用户坐标系下的位置.....	189
	示例 3: 工具末端在基坐标系下的位置.....	192
	示例 4: 工具末端在用户坐标系下的位置.....	195
4.11	偏移运动.....	198
4.11.1	robotServiceSetMoveRelativeParam 函数.....	198
	示例 1: 法兰盘中心在基坐标系下.....	198
	示例 2: 法兰盘中心在用户坐标系下.....	201
	示例 3: 工具末端在工具坐标系下.....	204
	示例 4: 工具末端在基坐标系下.....	206
	示例 5: 工具末端在用户坐标系下.....	209
4.12	旋转运动.....	212
4.12.1	robotServiceRotateMove 函数.....	212
	示例 1: 法兰盘中心在基坐标系下.....	212
	示例 2: 法兰盘中心在用户坐标系下.....	214
	示例 3: 工具末端在工具坐标系下.....	217
	示例 4: 工具末端在基坐标系下.....	220

示例 5：工具末端在用户坐标系下	222
4.13 轨迹运动	226
4.13.1 robotServiceTrackMove 函数	226
示例 1：圆运动	226
示例 2：圆弧运动	228
示例 3：MOVEP	231
4.14 示教运动	233
4.15 打印路点信息、关节状态信息、事件信息、诊断信息	236
5 环境配置说明	242
5.1 新建自己的程序	242
5.2 打开项目	256

简介

AUBO API 是基于网络实现的，提供了大量用于操作机械臂的接口，包括登录与退出、关节运动、直线运动、旋转运动等等。本文件定义了使用机械臂的接口类，是基于 C++ 开发的，其中类中的方法是操作机械臂的接口。

这些接口可被应用来实现 2D 智能相机的使用、3D 视觉集成、码垛示例的实现、软件控制力控、工具端控制力控等这些功能。

本手册包含了五个部分。第 1 章介绍数据类型，第 2 章介绍了接口函数的定义，第 3 章介绍了错误码，第 4 章介绍了接口函数的使用案例，第 5 章介绍了配置环境。

注意：接口中关于长度的单位都为米，关于角度的单位都为弧度。

1 数据类型

1.1 机械臂运动相关的类型

```
/**
 * @brief 机械臂关节数
 */
enum {
    ARM_DOF = 6,
};
```

```
/**
 * @brief DH 参数
 */
typedef struct
{
    double A3;
    double A4;
    double D1;
    double D2;
    double D5;
    double D6;

    // general dh model
    double alpha[ARM_DOF];
    double a[ARM_DOF];
    double d[ARM_DOF];
    double theta[ARM_DOF];
}RobotDhPara;
```

```
/**
 * @brief 位置信息
 */
struct Pos
{
    double x;
    double y;
    double z;
};
```

```
/**
 * @brief 位置信息的共用体描述
```

```
    **/  
union cartesianPos_U  
{  
    Pos    position;  
    double positionVector[3];  
};
```

```
/**  
 * @brief 姿态的四元素表示方法  
 */  
struct Ori  
{  
    double w;  
    double x;  
    double y;  
    double z;  
};
```

```
/**  
 * @brief 姿态的欧拉角表示方法  
 */  
struct Rpy  
{  
    double rx;  
    double ry;  
    double rz;  
};
```

```
typedef struct  
{  
    double jointPos[ARM_DOF];  
}JointParam;
```

```
/**  
 * @brief 描述关节的速度和加速度  
 */  
typedef struct  
{  
    double jointPara[ARM_DOF];  
}JointVelcAccParam;
```

```
/**  
 * @brirf 关节碰撞补偿（范围 0.00~0.51 度）
```

```

/**/
typedef struct
{
    double jointOffset[ARM_DOF];
}RobotJointOffset;

```

```

/**
 * @brief 机械臂的路点信息
 */
typedef struct
{
    cartesianPos_U    cartPos;           // 机械臂的位置信息(x,y,z)
    Ori               orientation;        // 机械臂姿态信息,四元素(w,x,y,z)
    double            jointpos[ARM_DOF]; // 机械臂关节角信息
}wayPoint_S;

```

```

/**
 * @brief 力传感器数据
 */
typedef struct
{
    double data[6];
}ForceSensorData;

```

```

/**
 * @brief 机械臂的位置和姿态
 */
typedef struct
{
    Pos position;
    Ori quaternion;
}PositionAndQuaternion;

```

```

/**
 * @brief 描述运动属性中的偏移属性
 */
typedef struct
{
    bool ena;           // 是否使能偏移
    float relativePosition[3]; // 偏移量 x,y,z
    Ori relativeOri;    // 姿态偏移
}MoveRelative;

```

```
/**
 * @brief 示教模式枚举
 **/
enum teach_mode
{
    NO_TEACH = 0,
    JOINT1,
    JOINT2,
    JOINT3,
    JOINT4,
    JOINT5,
    JOINT6,
    MOV_X,
    MOV_Y,
    MOV_Z,
    ROT_X,
    ROT_Y,
    ROT_Z
};
```

```
/**
 * @brief 运动轨迹枚举
 **/
enum move_track
{
    NO_TRACK = 0,

    //for moveJ and moveL
    TRACKING,

    //cartesian motion for moveP
    ARC_CIR,
    CARTESIAN_MOVEP,
    CARTESIAN_CUBICSPLINE,
    CARTESIAN_UBSPLINEINTP,
    CARTESIAN_GNUBSPLINEINTP,
    CARTESIAN_LOOKAHEAD,

    //joint motion for moveP
    JIONT_CUBICSPLINE,
    JOINT_UBSPLINEINTP,
    JOINT_GNUBSPLINEINTP,

    ARC,
```

```

    CIRCLE,
    ARC_ORI_ROTATED,
    CIRCLE_ORI_ROTATED,

    ORI_POSITION_ROTATE_CIRCUMFERENCE=101,
};

```

```

typedef struct
{
    double jointMaxAcc[ARM_DOF];    //关节型运动的最大加速度
    double jointMaxVelc[ARM_DOF];   //关节型运动的最大速度
    double endMaxLineAcc;           //末端型运动的最大加速度
    double endMaxLineVelc;          //末端型运动的最大速度
    MoveRelative relative;           //偏移参数
    CoordCalibrateByJointAngleAndTool relativeOnCoord; //偏移量基于那个坐标系
    double blendRadius;             //交融半径
    ToolInEndDesc toolInEndDesc;    //工具属性
}MoveProfile_t;

```

1.2 机械臂关节状态

```

/**
 * 描述机械臂的关节状态
 */
typedef struct PACKED
{
    int    jointCurrentI;           // 关节电流 Current of driver
    int    jointSpeedMoto;          // 关节速度 Speed of driver
    float  jointPosJ;              // 关节角 Current position in radian
    float  jointCurVol;           // 关节电压 Rated voltage of motor. Unit: mV
    float  jointCurTemp;          // 当前温度 Current temprature of joint
    int    jointTagCurrentI;        // 电机目标电流 Target current of motor
    float  jointTagSpeedMoto;       // 电机目标速度 Target speed of motor
    float  jointTagPosJ;           // 目标关节角 Target position of joint in radian
    uint16 jointErrorNum;          // 关节错误码 Joint error of joint num
}JointStatus;

```

1.3 事件类型

```

/**
 * 描述机械臂事件类型
 *
 * 机械臂的很多信息（比如故障，通知）是通过事件通知到客户的，所以在使

```

```

* 用 SDK 时，务必注册接收事件的回调函数。
*/
typedef enum{
    RobotEvent_armCanbusError, //机械臂 CAN 总线错误已过时，不建议使用
    RobotEvent_remoteHalt,     //远程关机
    RobotEvent_remoteEmergencyStop, //机械臂远程急停
    RobotEvent_jointError,      //关节错误，PS:已过时，不建议使用
    RobotEvent_forceControl,    //力控制
    RobotEvent_exitForceControl, //退出力控制
    RobotEvent_softEmergency,   //软急停
    RobotEvent_exitSoftEmergency, //退出软急停
    RobotEvent_collision,       //碰撞，PS：已过时，不建议使用
                                //已用 RobotEventJointCollision(2123) 替代
    RobotEvent_collisionStatusChanged, //碰撞状态改变，已过时，不建议使用
                                //已用 RobotEventJointCollision(2123) 替代
    RobotEvent_tcpParametersSucc, //工具动力学参数设置成功
                                //系统事件，用户可以忽略
    RobotEvent_powerChanged,      //机械臂电源开关状态改变
    RobotEvent_ArmPowerOff,       //机械臂电源关闭，不建议使用
                                //已用 RobotEventArmPowerOff(2600) 替代
    RobotEvent_mountingPoseChanged, //安装位置发生改变
    RobotEvent_encoderError,       //编码器错误，不建议使用
    RobotEvent_encoderLinesError,  //编码器线数不一致，不建议使用
                                //已用 RobotEventEncoderLineError（2203）替代
    RobotEvent_singularityOverspeed, //奇异点超速
    RobotEvent_currentAlarm,        //机械臂电流异常
    RobotEvent_toolioError,         //机械臂工具端错误
    RobotEvent_robotStartupPhase,   //机械臂启动阶段
                                //系统事件，用户可以忽略
    RobotEvent_robotStartupDoneResult, //机械臂启动完成结果
                                //系统事件，用户可以忽略
    RobotEvent_robotShutdownDone,   //机械臂关机结果
                                //系统事件，用户可以忽略
    RobotEvent_atTrackTargetPos,    //机械臂轨迹运动到位信号通知
                                //系统事件，用户可以忽略
    RobotSetPowerOnDone,            //设置电源状态完成
    RobotReleaseBrakeDone,          //机械臂刹车释放完成
                                //系统事件，用户可以忽略
    RobotEvent_robotControllerStateChaned, //机械臂控制状态改变
                                //系统事件，用户可以忽略
    RobotEvent_robotControllerError, //机械臂控制错误，
                                //一般是算法规划出现问题时返回
    RobotEvent_socketDisconnected,  //socket 断开连接
    RobotEvent_robotControlException,

```

```

RobotEvent_trackPlayInterrupte,
RobotEvent_staticCollisionStatusChanged, //不建议使用, 已过时
RobotEvent_MountingPoseWarning,
RobotEvent_MacDataInterruptWarning,
RobotEvent_ToolIoError,
RobotEvent_InterfacBoardSafeIoEvent, //安全 IO 通知型事件
RobotEvent_RobotHandShakeSucc, //系统事件, 用户可以忽略
RobotEvent_RobotHandShakeFailed, //系统事件, 用户可以忽略
RobotEvent_RobotErrorInfoNotify, //不建议使用, 已过时
RobotEvent_InterfacBoardDIChanged, //通知型事件 DI 状态改变
RobotEvent_InterfacBoardDOChanged, //通知型事件 DO 状态改变
RobotEvent_InterfacBoardAIChanged, //通知型事件 AI 状态改变
RobotEvent_InterfacBoardAOChanged, //通知型事件 AO 状态改变
RobotEvent_UpdateJoint6Rot360Flag, //系统事件, 用户可以忽略
RobotEvent_RobotMoveControlDone, //系统事件, 用户可以忽略
RobotEvent_RobotMoveControlStopDone, //系统事件, 用户可以忽略
RobotEvent_RobotMoveControlPauseDone, //系统事件, 用户可以忽略
RobotEvent_RobotMoveControlContinueDone, //系统事件, 用户可以忽略

//主从模式切换
RobotEvent_RobotSwitchToOnlineMaster, //通知型事件, 进入联动主模式
RobotEvent_RobotSwitchToOnlineSlave, //通知型事件, 进入联动从模式

RobotEvent_ConveyorTrackRobotStartup, //系统事件, 用户可以忽略
RobotEvent_ConveyorTrackRobotCatchup, //系统事件, 用户可以忽略

RobotEvent_exceptEvent = 100,
RobotEventInvalid = 1000, // 无效的事件

/**
 * RobotControllerErrorEvent 控制器异常事件 1001~1499
 *
 * 事件处理建议
 * 建议采取措施:停止当前运动
 *
 * PS: 这些事件会引起机械臂运动的错误返回
 * 使用时尽量用枚举变量,枚举变量值只是为了查看日志方便
 *
 */
RobotEventMoveJConfigError = 1001,
// moveJ configuration error 关节运动属性配置错误
RobotEventMoveLConfigError = 1002,
// moveL configuration error 直线运动属性配置错误
RobotEventMovePConfigError = 1003,

```



```

// moveP configuration error 轨迹运动属性配置错误
RobotEventInvailConfigError = 1004,
// invail configuration 无效的运动属性配置
RobotEventWaitRobotStopped = 1005,
// please wait robot stopped 等待机器人停止
RobotEventJointOutOfRange = 1006,
// joint out of range 超出关节运动范围
RobotEventFirstWaypointSetError = 1007,
// please set first waypoint correctly in modep
// 请正确设置 MOVEP 第一个路点
RobotEventConveyorTrackConfigError = 1008,
// configuration error for conveyor tracking 传送带跟踪配置错误
RobotEventConveyorTrackTrajectoryTypeError = 1009,
// unsupported conveyor tracking trajectory type 传送带轨迹类型错误
RobotEventRelativeTransformIKFailed = 1010,
// inverse kinematics failure due to invalid relative transform
// 相对坐标变换逆解失败
RobotEventTeachModeCollision = 1011,
// collision in teach-mode 示教模式发生碰撞
RobotEventExternalToolConfigError = 1012,
// configuration error for external tool and hand workobject
// 运动属性配置错误,外部工具或手持工件配置错误
RobotEventTrajectoryAbnormal = 1101,
// Trajectory is abnormal 轨迹异常
RobotEventOnlineTrajectoryPlanError = 1102,
// Trajectory is abnormal,online planning failed 轨迹规划错误
RobotEventOnlineTrajectoryTypeIIError = 1103,
// Trajectory is abnormal,type II online planning failed
// 二型在线轨迹规划失败
RobotEventIKFailed = 1104,
// Trajectory is abnormal,inverse kinematics failed 逆解失败
RobotEventAbnormalLimitProtect = 1105,
// Trajectory is abnormal,abnormal limit protection 动力学限制保护
RobotEventConveyorTrackingFailed = 1106,
// Trajectory is abnormal,conveyor tracking failed 传送带跟踪失败
RobotEventConveyorOutWorkingRange = 1107,
// Trajectory is abnormal,exceeding the conveyor working range
// 超出传送带工作范围
RobotEventTrajectoryJointOutOfRange = 1108,
// Trajectory is abnormal,joint out of range 关节超出范围
RobotEventTrajectoryJointOverspeed = 1109,
// Trajectory is abnormal,joint overspeed 关节超速
RobotEventOfflineTrajectoryPlanFailed = 1110,
// Trajectory is abnormal,Offline track planning failed 离线轨迹规划失败

```

```

RobotEventTrajectoryJointAccOutOfRange      = 1111,
// Trajectory is abnormal,joint acc out of range 轨迹异常,关节加速度超限

RobotEventForceModeException                = 1120, // 力控模式异常
RobotEventForceModeIKFailed                 = 1121,
// Trajectory is abnormal,force control mode ik failed
// 轨迹异常, 力控模式下失败
RobotEventForceModeTrackJointverspeed      = 1122,
// Trajectory is abnormal,joint overspeed 关节超速
RobotEventControllerIKFailed               = 1200,
// The controller has an exception and the inverse kinematics failed
// 控制器异常, 逆解失败
RobotEventControllerStatusException        = 1201,
// The controller has an exception and the status is abnormal
// 控制器异常, 状态异常
RobotEventControllerTrackingLost           = 1202,
// Exception that joint tracking is lost, 关节跟踪误差过大.
RobotEventMonitorErrTrackingLost           = 1203,
// Exception that joint tracking is lost, 关节跟踪误差过大.
RobotEventMonitorErrNoArrivalInTime        = 1204, // not used 预留
RobotEventMonitorErrCurrentOverload        = 1205, // not used 预留
RobotEventMonitorErrJointOutOfRange        = 1206,
// Exception that joint out of range 机械臂关节超出限制范围
RobotEventMonitorErrFifoDataTimeNotRead    = 1207,
// controller fifo data timeout was not read 队列中数据超时未被读取
RobotEventMoveEnterStopState               = 1300,
// Movement enters the stop state 运动进入到 stop 阶段

/**
 * RobotHardwareErrorEvent 来自硬件反馈的异常事件 2001~2999
 *
 * 事件处理建议
 * RobotEventJointEncoderPollustion      建议采取措施:警告性通知
 * RobotEventDriveVersionError           建议采取措施:警告性通知
 * RobotEventJointCollision               建议采取措施: 如需回复当前运动, 调
 * 用暂停函数, 恢复的时候先调用碰撞回复函数, 在调用 continue 函数; 如不需
 * 恢复当前运动, 调用停止函数, 恢复的时候调用碰撞回复函数可以
 * 其余的事件建议采取措施:停止当前运动
 */

RobotEventHardwareErrorNotify             = 2001,
// Robot hardware error 机械臂硬件错误

```

RobotEventJointError	= 2101,
// Robot joint error 机械臂关节错误	
RobotEventJointOverCurrent	= 2102,
// Robot joint over current. 机械臂关节过流	
RobotEventJointOverVoltage	= 2103,
// Robot joint over voltage. 机械臂关节过压	
RobotEventJointLowVoltage	= 2104,
// Robot joint low voltage. 机械臂关节欠压	
RobotEventJointOverTemperature	= 2105,
// Robot joint over temperature. 机械臂关节过温	
RobotEventJointHallError	= 2106,
// Robot joint hall error. 机械臂关节霍尔错误	
RobotEventJointEncoderError	= 2107,
// Robot joint encoder error. 机械臂关节编码器错误	
RobotEventJointAbsoluteEncoderError	= 2108,
// Robot joint absolute encoder error. 机械臂关节绝对编码器错误	
RobotEventJointCurrentDetectError	= 2109,
// Robot joint current position error. 机械臂关节当前位置错误	
RobotEventJointEncoderPollution	= 2110,
// Robot joint encoder pollution.	
// 机械臂关节编码器污染 建议采取措施:警告性通知	
RobotEventJointEncoderZSignalError	= 2111,
// Robot joint encoder Z signal error. 机械臂关节编码器 Z 信号错误	
RobotEventJointEncoderCalibrateInvalid	= 2112,
// Robot joint encoder calibrate invalid. 机械臂关节编码器校准失效	
RobotEventJoint_IMU_SensorInvalid	= 2113,
// Robot joint IMU sensor invalid. 机械臂关节 IMU 传感器失效	
RobotEventJointTemperatureSensorError	= 2114,
// Robot joint temperature sensor error. 机械臂关节温度传感器出错	
RobotEventJointCanBusError	= 2115,
// Robot joint CAN BUS error. 机械臂关节 CAN 总线出错	
RobotEventJointCurrentError	= 2116,
// Robot joint current error. 机械臂关节当前电流错误	
RobotEventJointCurrentPositionError	= 2117,
// Robot joint current position error. 机械臂关节当前位置错误	
RobotEventJointOverSpeed	= 2118,
// Robot joint over speed. 机械臂关节超速	
RobotEventJointOverAccelerate	= 2119,
// Robot joint over accelerate. 机械臂关节加速度过大错误	
RobotEventJointTraceAccuracy	= 2120,
// Robot joint trace accuracy. 机械臂关节跟踪精度错误	
RobotEventJointTargetPositionOutOfRange	= 2121,
// Robot joint target position out of range. 机械臂关节目标位置超范围	
RobotEventJointTargetSpeedOutOfRange	= 2122,

```

// Robot joint target speed out of range. 机械臂关节目标速度超范围
RobotEventJointCollision                                = 2123,
// Robot joint collision. 机械臂碰撞      建议采取措施:暂停当前运动

RobotEventDataAbnormal                                = 2200,
// Robot data abnormal 机械臂信息异常
RobotEventRobotTypeError                                = 2201,
// Robot type error 机械臂类型错误
RobotEventAccelerationSensorError                      = 2202,
// Robot acceleration sensor error 机械臂加速度计芯片错误
RobotEventEncoderLineError                             = 2203,
// Robot encoder line error 机械臂编码器线数错误
RobotEventEnterDragAndTeachModeError                   = 2204,
// Robot enter drag and teach mode error 机械臂进入拖动示教模式错误
RobotEventExitDragAndTeachModeError                    = 2205,
// Robot exit drag and teach mode error 机械臂退出拖动示教模式错误
RobotEventMACDataInterruptionError                    = 2206,
// Robot MAC data interruption error 机械臂 MAC 数据中断错误
RobotEventDriveVersionError                            = 2207,
// Drive version error 驱动器版本错误(关节固件版本不一致)

RobotEventInitAbnormal                                = 2300,
// Robot init abnormal 机械臂初始化异常
RobotEventDriverEnableFailed                           = 2301,
// Robot driver enable failed 机械臂驱动器使能失败
RobotEventDriverEnableAutoBackFailed                   = 2302,
// Robot driver enable auto back failed 机械臂驱动器使能自动回应失败
RobotEventDriverEnableCurrentLoopFailed                = 2303,
// Robot driver enable current loop failed 机械臂驱动器使能电流环失败
RobotEventDriverSetTargetCurrentFailed                 = 2304,
// Robot driver set target current failed 机械臂驱动器设置目标电流失败
RobotEventDriverReleaseBrakeFailed                     = 2305,
// Robot driver release brake failed 机械臂释放刹车失败
RobotEventDriverEnablePostionLoopFailed                = 2306,
// Robot driver enable postion loop failed 机械臂使能位置环失败
RobotEventSetMaxAccelerateFailed                       = 2307,
// Robot set max accelerate failed 设置最大加速度失败

RobotEventSafetyError                                  = 2400,
// Robot Safety error 机械臂安全出错
RobotEventExternEmergencyStop                          = 2401,
// Robot extern emergency stop 机械臂外部紧急停止
RobotEventSystemEmergencyStop                         = 2402,
// Robot system emergency stop 机械臂系统紧急停止

```

```

RobotEventTeachpendantEmergencyStop          = 2403,
// Robot teachpendant emergency stop 机械臂示教器紧急停止
RobotEventControlCabinetEmergencyStop        = 2404,
// Robot control cabinet emergency stop 机械臂控制柜紧急停止
RobotEventProtectionStopTimeout              = 2405,
// Robot protection stop timeout 机械臂保护停止超时
RobotEventEeducedModeTimeout                  = 2406,
// Robot reduced mode timeout 机械臂缩减模式超时

RobotEventSystemAbnormal                      = 2500,
// Robot system abnormal 机械臂系统异常
RobotEvent_MCU_CommunicationAbnormal          = 2501,
// Robot mcu communication error 机械臂 mcu 通信异常
RobotEvent485CommunicationAbnormal            = 2502,
// Robot RS485 communication error 机械臂 485 通信异常

// #if 0 // 非实时版本
RobotEventCurrentJointOutOfRange              = 2550,
// Joint out of Range
// #else
RobotEventSoftEmergency                       = 2550, // 软急停
RobotEventSoftEmergencyExit                   = 2551, // 软急停退出
// #endif

RobotEventArmPowerOff                         = 2600,
// Disconnecting the contactor causes the arm 48V power off
// 控制柜接触器断开导致机械臂 48V 断电

RobotEventHardwareErrorNotifyMaximumIndex    = 2999, // 索引

RobotEventNotifyEvent                         = 3000,
// Robot notification event 机械臂通知性事件
RobotEventNotifyCollisionLevelChange          = 3001,
// Robot Collision level change 机械臂事件通知-碰撞等级被改变
RobotEventNotifyEnterFlexibleControlMode      = 3010,
// 进入柔性控制模式通知
RobotEventNotifyExitFlexibleControlMode       = 3011,
// 退出柔性控制模式通知
RobotEventNotifyEnterSpeedReducedMode        = 3015,
// 进入速度缩减模式通知
RobotEventNotifyExitSpeedReducedMode         = 3016,
// 退出速度缩减模式通知

RobotEventNotifyStopCurrentMove               = 3100,

```

```

// 停止掉当前运动

//unknown event
robot_event_unknown                = 10000,

//user event
RobotEvent_User                    = 9000,
// first user event id
RobotEvent_MaxUser                 = 9999
// last user event id

}RobotEventType;

```

```

/** 事件类型 */
typedef struct{
    RobotEventType    eventType;           //事件类型号
    int               eventCode;
    std::string       eventContent;        //事件内容
}RobotEventInfo;

```

1.4 诊断信息

```

/**
 * 机械臂诊断信息
 */
typedef struct PACKED
{
    uint8    armCanbusStatus; // CAN 通信状态:0x01~0x80: 关节 CAN 通信错误
                                   // （每个关节占用 1bit） 0x00: 无错误

    float    armPowerCurrent;      // 机械臂 48V 电源当前电流
    float    armPowerVoltage;      // 机械臂 48V 电源当前电压
    bool     armPowerStatus;       // 机械臂 48V 电源状态（开、关）
    char     contorllerTemp;       // 控制箱温度
    uint8    contorllerHumidity;   // 控制箱湿度
    bool     remoteHalt;           // 远程关机信号
    bool     softEmergency;        // 机械臂软急停
    bool     remoteEmergency;      // 远程急停信号
    bool     robotCollision;       // 碰撞检测位
    bool     forceControlMode;     // 机械臂进入力控模式标志位
    bool     brakeStuats;          // 刹车状态
    float    robotEndSpeed;        // 末端速度
    int      robotMaxAcc;          // 最大加速度
    bool     orpeStatus;           // 上位机软件状态位
}

```

```

bool    enableReadPose;           // 位姿读取使能位
bool    robotMountingPoseChanged; // 安装位置状态
bool    encoderErrorStatus;       // 磁编码器错误状态
bool    staticCollisionDetect;     // 静止碰撞检测开关
uint8   jointCollisionDetect;      // 关节碰撞检测 每个关节占用 1bit
                                     // 0-无碰撞 1-存在碰撞
bool    encoderLinesError; // 光电编码器不一致错误：0-无错误，1-有错误
bool    jointErrorStatus;         // joint error status
bool    singularityOverSpeedAlarm; // 机械臂奇异点过速警告
bool    robotCurrentAlarm;        // 机械臂电流错误警告
uint8   toolIoError;              // tool error
bool    robotMountingPoseWarning; // 机械臂安装位置错位
                                     // （只在力控模式下起作用）

uint16   macTargetPosBufferSize;  // mac 缓冲器长度，预留
uint16   macTargetPosDataSize;    // mac 缓冲器有效数据长度，预留
uint8    macDataInterruptWarning; // mac 数据中断，预留
uint8    controlBoardAbnormalStateFlag; // 主控板(接口板)异常状态标志
}RobotDiagnosis;

```

1.5 回调函数类型

```

/**
 * @brief          获取实时关节状态回调函数类型
 * @param jointStatus 当前的关节状态;
 * @param size      上一个参数（jointStatus）的长度;
 * @param arg       使用者在注册回调函数中传递的第二个参数;
 */
typedef void (*RealTimeJointStatusCallback) (const aubo_robot_namespace::JointStatus *jointStatus, int size, void *arg);

```

```

/**
 * @brief          获取实时路点信息的回调函数类型
 * @param Waypoint 当前的路点信息;
 * @param arg       使用者在注册回调函数中传递的第二个参数;
 */
typedef void (*RealTimeRoadPointCallback) (const aubo_robot_namespace::Waypoint_S *Waypoint, void *arg);

```

```

/**
 * @brief          获取实时末端速度的回调函数类型
 * @param speed     当前的末端速度;
 * @param arg       使用者在注册回调函数中传递的第二个参数;
 */

```

```
typedef void (*RealTimeEndSpeedCallback) (double speed, void *arg);
```

```
/**
 * @brief          获取实时 Movep 执行进度的回调函数类型
 * @param num      当前的 Movep 执行进度;
 * @param arg      使用者在注册回调函数中传递的第二个参数;
 */
typedef void (*RealTimeMovepStepNumNotifyCallback) (int num, void *arg);
```

```
/**
 * @brief          获取机械臂事件信息的回调函数类型
 * @param arg      使用者在注册回调函数中传递的第二个参数;
 */
typedef void (*RobotEventCallback) (const aubo_robot_namespace::RobotEventInfo
 *eventInfo, void *arg);
```

```
/**
 * @brief          日志输出对应的回调函数类型
 * @param logLevel  日志级别;
 * @param str       日志信息;
 */
typedef void (*RobotLogPrintCallback) (aubo_robot_namespace::LOG_LEVEL logL
evel, const char *str, void *arg);
```

1.6 工具参数类型

```
/** 工具描述, 工具的运动学参数
 *
 * 该工具用于描述一个工具或者工具的运动学参数
 */
typedef struct
{
    Pos      toolInEndPosition;    // 工具相对法兰盘的位置
    Ori      toolInEndOrientation; // 工具相对法兰盘的姿态
} ToolInEndDesc;
```

```
typedef ToolInEndDesc ToolKinematicsParam; //运动学参数
```

```
/**
 * 工具动力学参数描述
 *
 * 注意:
 *      机械臂上电之前, 安装在机器人末端的工具发生改变时都需要重新
```



```

* 设置工具的动力学参数。
*      一般情况下，工具的动力学参数和运动学参数是需要一起设置的；
* 切记：
*      该参数如果不能正确设置会影响机械臂的安全等级和运动轨迹。
**/
typedef struct
{
    double    positionX;    // 工具重心的 X 坐标
    double    positionY;    // 工具重心的 Y 坐标
    double    positionZ;    // 工具重心的 Z 坐标
    double    payload;      // 工具重量
    ToolInertia toolInertia; // 工具惯量，预留，使用是全部设置为 0
}ToolDynamicsParam;

```

```

/**
* 该结构体描述工具惯量
*
* 注：该结构体属于冗余数据类型，使用时把所有参数都设置为{0,0,0,0,0,0}。
**/
typedef struct
{
    double xx;
    double xy;
    double xz;
    double yy;
    double yz;
    double zz;
}ToolInertia;

```

1.7 工具标定

```

/**
* @brief 工具姿态标定的方法枚举
*
*/
enum ToolKinematicsOriCalibrateMethod
{
    ToolKinematicsOriCalibrateMethod_Invalid = -1,

    ToolKinematicsOriCalibrateMethod_xOxy,
    // 原点、x 轴正半轴、x、y 轴平面的第一象限上任意一点
    ToolKinematicsOriCalibrateMethod_yOyz,
    // 原点、y 轴正半轴、y、z 轴平面的第一象限上任意一点
    ToolKinematicsOriCalibrateMethod_zOzx,

```

```

// 原点、z 轴正半轴、z、x 轴平面的第一象限上任意一点
ToolKinematicsOriCalibrateMethod_TxRBz_TxyPBzAndTyABnz,
// 工具 x 轴平行反向于基坐标系 z 轴;
// 工具 xOy 平面平行于基坐标系 z 轴、工具 y 轴与基坐标系负 z 轴夹角为锐角
ToolKinematicsOriCalibrateMethod_TyRBz_TyzPBzAndTzABnz,
// 工具 y 轴平行反向于基坐标系 z 轴;
// 具 yOz 平面平行于基坐标系 z 轴、工具 z 轴与基坐标系负 z 轴夹角为锐角
ToolKinematicsOriCalibrateMethod_TzRBz_TzxPBzAndTxABnz,
// 工具 z 轴平行反向于基坐标系 z 轴;
// 工具 zOx 平面平行于基坐标系 z 轴、工具 x 轴与基坐标系负 z 轴夹角为锐角

ToolKinematicsOriCalibrateMethodCount
};

```

```

typedef struct
{
    int posCalibrateNum ;           //用于位置标定点的数量
    wayPoint_S posCalibrateWaypoint[4];    //位置标定定点
    int oriCalibrateNum;    //用于姿态标定点的数量
    wayPoint_S oriCalibrateWaypoint[3];    //姿态标定定点
    ToolKinematicsOriCalibrateMethod CalibrateMethod;    //姿态标定方法
}ToolCalibrate;

```

1.8 坐标系标定

```

/**
 * 坐标系描述
 *
 * 该结构体描述一个坐标系。系统通过该结构体描述一个坐标系(基座坐标系,
 * 用户坐标系, 末端坐标系或工具坐标系)。
 *
 * 坐标系分 3 种类型:    基座坐标系(BaseCoordinate);
 *                      用户坐标系(WorldCoordinate);
 *                      末端坐标系或工具坐标系(EndCoordinate);
 *
 * 定义:
 *    基座坐标系是根据机械臂基座建立的坐标系;
 *    用户坐标系是用户坐标系定义在工件上, 在机器人动作允许范围内的任
 *    意位置, 设定任意角度的 X、Y、Z 轴, 原点位于机器人抓取的工件上, 坐标
 *    系的方向根据客户要求任意定义。
 *    末端坐标系是安装在机器人末端的工具坐标系, 原点及方向都是随着末
 *    置与角度不断变化的, 该坐标系实际是将基础坐标系通过旋转及位移变化而

```

```

* 来的；法兰盘是一个特殊的末端坐标系。
*
* 结构体参数描述：
*     coordType: 坐标系类型,描述坐标系属于那种类型
*     methods: 用户坐标系的标定方法，仅在 coordType 为用户坐标系(WorldC
* oordinate)时有效；
*     wayPointArray[3]: 标定用户坐标系的 3 个路点信息，仅在 coordType 为
* 用户坐标系(WorldCoordinate)时有效；
*     toolDesc: 末端工具描述，当 coordType=WorldCoordinate，表示标定用
* 户坐标系时，安装在机器人末端的工具;当 coordType=EndCoordinate，描述是
* 哪个工具的坐标系
*
* 使用说明：
*     基座坐标系: coordType=BaseCoordinate，其他参数默认
*     用户坐标系: coordType=WorldCoordinate
*             methods: 为标定方法
*             wayPointArray[3]: 为标定坐标系的 3 个路点
*             toolDesc: 标定用户坐标系时，安装在机器人末端的工具
*     末端坐标系或工具坐标系: coordType=EndCoordinate
*             methods: 缺省，不需要设置
*             wayPointArray[3]: 缺省，不需要设置
*             toolDesc: 机器人末端的工具
*
* 备注：
*     法兰盘为特殊的工具,工具描述中的位置设置为(0,0,0)，
*     姿态信息设置为(1,0,0,0)
*     其结构体定义为
*     {
*         pos{0,0,0},
*         Ori{1,0,0,0}
*     }
*
*     该结构同时用于用户坐标系的标定。一般通过示教 3 个示教点实现，第一
*     个示教点是用户坐标系的原点；第二个和第三个示教点的选择根据标定方法
*     来确定,遵循右手手法。
*/
typedef struct
{
    coordinate_refer    coordType;        // 坐标系类型
    CoordCalibrateMethod methods;         // 用户坐标系的标定方法
    JointParam          wayPointArray[3]; // 用于标定用户坐标系的 3 个点
                                     // (关节角)
    ToolInEndDesc       toolDesc;         // 工具描述
}CoordCalibrateByJointAngleAndTool;

```

```

/**
 * 坐标系类型枚举
 **/
enum coordinate_refer
{
    BaseCoordinate = 0, // 基座坐标系
    EndCoordinate,      // 末端坐标系或工具坐标系
    WorldCoordinate,    // 用户坐标系
};

```

```

/**
 * 用户坐标系标定方法枚举
 *
 * 描述:3 点标定坐标系    3 个示教点的含义.
 **/
enum CoordCalibrateMethod
{
    Origin_AnyPointOnPositiveXAxis_AnyPointOnPositiveYAxis,
    // 原点、x 轴正半轴、y 轴正半轴
    Origin_AnyPointOnPositiveYAxis_AnyPointOnPositiveZAxis,
    // 原点、y 轴正半轴、z 轴正半轴
    Origin_AnyPointOnPositiveZAxis_AnyPointOnPositiveXAxis,
    // 原点、z 轴正半轴、x 轴正半轴
    Origin_AnyPointOnPositiveXAxis_AnyPointOnFirstQuadrantOfXOYPlane,
    // 原点、x 轴正半轴、x、y 轴平面的第一象限上任意一点
    Origin_AnyPointOnPositiveXAxis_AnyPointOnFirstQuadrantOfXOZPlane,
    // 原点、x 轴正半轴、x、z 轴平面的第一象限上任意一点
    Origin_AnyPointOnPositiveYAxis_AnyPointOnFirstQuadrantOfYOZPlane,
    // 原点、y 轴正半轴、y、z 轴平面的第一象限上任意一点
    Origin_AnyPointOnPositiveYAxis_AnyPointOnFirstQuadrantOfYOXPlane,
    // 原点、y 轴正半轴、y、x 轴平面的第一象限上任意一点
    Origin_AnyPointOnPositiveZAxis_AnyPointOnFirstQuadrantOfZOXPlane,
    // 原点、z 轴正半轴、z、x 轴平面的第一象限上任意一点
    Origin_AnyPointOnPositiveZAxis_AnyPointOnFirstQuadrantOfZOYPlane,
    // 原点、z 轴正半轴、z、y 轴平面的第一象限上任意一点

    CoordTypeCount,
};

```

1.9 IO 相关数据类型

```

/**
 * @brief IO 的类型枚举

```

```

*
**/
typedef enum
{
    RobotBoardControllerDI,    // 接口板控制器 DI(数字量输入),
                                // 只读(一般系统内部使用)
    RobotBoardControllerDO,    // 接口板控制器 DO(数字量输出),
                                // 只读(一般系统内部使用)
    RobotBoardControllerAI,    // 接口板控制器 AI(模拟量输入),
                                // 只读(一般系统内部使用)
    RobotBoardControllerAO,    // 接口板控制器 AO(模拟量输出),
                                // 只读(一般系统内部使用)

    RobotBoardUserDI,          // 接口板用户 DI(数字量输入), 可读可写
    RobotBoardUserDO,          // 接口板用户 DO(数字量输出), 可读可写
    RobotBoardUserAI,          // 接口板用户 AI(模拟量输入), 可读可写
    RobotBoardUserAO,          // 接口板用户 AO(模拟量输出), 可读可写

    RobotToolDI,               // 工具端 DI
    RobotToolDO,               // 工具端 DO
    RobotToolAI,               // 工具端 AI
    RobotToolAO,               // 工具端 AO
}RobotIoType;

```

```

/**
* IO 类型
**/
typedef enum
{
    IO_IN = 0,                //输入
    IO_OUT                    //输出
}ToolIoType;

```

```

/**
* 工具的电源类型
**/
typedef enum
{
    OUT_0V = 0,
    OUT_12V = 1,
    OUT_24V = 2
}ToolPowerType;

```

```

typedef enum

```

```
{
    RobotToolIoTypeDI=RobotToolDI,           //工具端 DI
    RobotToolIoTypeDO=RobotToolDO           //工具端 DO
}RobotToolIoType;
```

```
typedef enum           // I O 状态
{
    IO_STATUS_INVALID = 0,    //有效
    IO_STATUS_VALID        //无效
}IO_STATUS;
```

```
typedef enum
{
    TOOL_DIGITAL_IO_0 = 0,
    TOOL_DIGITAL_IO_1 = 1,
    TOOL_DIGITAL_IO_2 = 2,
    TOOL_DIGITAL_IO_3 = 3
}ToolDigitalIOAddr;
```

```
/**
 * 综合描述一个 IO
 **/
typedef struct PACKED
{
    char        ioId[32];           //IO-ID 目前未使用
    RobotIoType ioType;            //IO 类型
    char        ioName[32];        //IO 名称
    int         ioAddr;            //IO 地址
    double      ioValue;           //IO 状态
}RobotIoDesc;
```

```
//接口板数字量数据
typedef struct
{
    uint8 addr ;
    uint8 value;
    uint8 type;
}RobotDiagnosisIODesc;
```

```
//接口板模拟量数据
typedef struct
{
    uint8 addr ;
```

```
float    value;  
uint8    type;  
}RobotAnalogIODesc;
```

```
typedef struct PACKED  
{  
    ToolIOType ioType;  
    uint8      ioData;  
}ToolDigitalStatus;
```

2 接口函数

2.1 机械臂系统接口

2.1.1 登录

<pre>int robotServiceLogin(const char *host, int port, const char *userName, const char *password);</pre>	
功能描述:	登录，与机械臂服务器建立网络连接。该接口的成功是调用其他接口的前提，只有在该接口正确返回的情况下，才能使用其他接口。
参数说明:	1. host: 机械臂服务器的 IP 地址，即控制器的 IP。 2. port: 机械臂服务器的端口号，默认为 8899。 3. userName: 用户名，默认为 Aubo。 4. password: 密码，默认为 123456。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

<pre>int robotServiceLogin(const char* host, int port, const char *userName, const char* password, aubo_robot_namespace::RobotType &robotType, aubo_robot_namespace::RobotDhPara &robotDhPara);</pre>	
功能描述:	登录，与机械臂服务器建立网络连接。该接口的成功是调用其他接口的前提，只有在该接口正确返回的情况下，才能使用其他接口。
参数说明:	1. host: 机械臂服务器的 IP 地址，即控制器的 IP。 2. port: 机械臂服务器的端口号，默认为 8899。 3. userName: 用户名，默认为 Aubo。 4. password: 密码，默认为 123456。 5. robotType: 机械臂类型。 6. robotDhPara: DH 参数。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.1.2 退出登录

<pre>int robotServiceLogout();</pre>	
功能描述:	退出登录，断开与机械臂服务器的连接。
参数说明:	无。

返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.1.3 启动机械臂

<pre>int rootServiceRobotStartup(const aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam, uint8 collisionClass, bool readPose, bool staticCollisionDetect, int boardBaxAcc, aubo_robot_namespace::ROBOT_SERVICE_STATE &result, bool IsBolck = true);</pre>	
功能描述:	启动机械臂，即初始化-----该操作会完成机械臂的上电，松刹车，设置碰撞等级，设置动力学参数等功能。 该函数完成需要的时间比较长，因此可以通过设置是否阻塞来调整函数返回的时间，当设置为非阻塞时，函数调用后立即返回，调用结果通过事件来通知。当设置为阻塞时，参数返回值代表接口是否被调用成功，result 表示机械臂启动结果。
参数说明:	1. toolDynamicsParam: 动力学参数，如果末端夹持工具，此参数应该根据具体的来设定；如果末端没有夹持工具，将此参数的各项设置为 0。 2. collisionClass: 碰撞等级。 3. readPose: 是否允许读取位置，默认是 true。 4. staticCollisionDetect: 默认为 true。 5. boardBaxAcc: 默认为 1000。 6. result: 传出参数，初始化结果，具体参考 ROBOT_SERVICE_STATE 类型。机械臂启动结果只有 result == ROBOT_SERVICE_WORKING 表示机械臂启动成功，否则表示启动失败。 7. IsBolck: 是否阻塞，默认 true。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.1.4 关机

<pre>int robotServiceRobotShutdown(bool IsBolck = true);</pre>	
功能描述:	机械臂断电。
参数说明:	IsBolck: 是否阻塞，默认为 true。 设置阻塞时，直到机械臂关机后函数返回；设置非阻塞时，立即返回，结果通过事件推送来返回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2 状态推送

2.2.1 设置是否允许实时关节状态推送

int robotServiceSetRealTimeJointStatusPush(bool enable);	
功能描述:	设置是否允许实时关节状态推送。
参数说明:	enable: true 表示允许, false 表示不允许。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.2 注册用于获取关节状态的回调函数

int robotServiceRegisterRealTimeJointStatusCallback(RealTimeJointStatusCallback ptr, void *arg);	
功能描述:	注册用于获取关节状态的回调函数。 注册回调函数后, 服务器实时推送当前的关节状态信息。 频率 30ms。
参数说明:	1. ptr: 获取实时关节状态信息的回调函数指针, 当 ptr==NULL 时, 相当于取消回调函数的注册, 取消该推送信息也可以通过该接口 robotServiceSetRealTimeJointStatusPush 进行。 2. arg: 这个参数系统不做任何处理, 只是进行了缓存, 当系统调用已注册回调函数时, 该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.3 设置是否允许实时路点信息推送

int robotServiceSetRealTimeRoadPointPush(bool enable);	
功能描述:	设置是否允许实时路点信息推送。
参数说明:	enable: true 表示允许, false 表示不允许。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.4 注册用于获取实时路点的回调函数

int robotServiceRegisterRealTimeRoadPointCallback(const RealTimeRoadPointCallback ptr, void *arg);	
功能描述:	注册用于获取实时路点信息的回调函数。 注册回调函数后, 服务器实时推送当前的路点信息。 频率 30ms。

参数说明:	1. ptr: 获取实时路点信息的回调函数指针, 当 ptr==NULL 时, 相当于取消回调函数的注册, 取消该推送信息也可以通过该接口 robotServiceSetRealTimeRoadPointPush 进行。 2. arg: 这个参数系统不做任何处理, 只是进行了缓存, 当系统调用已注册回调函数时, 该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.5 设置是否允许实时末端速度推送

int robotServiceSetRealTimeEndSpeedPush(bool enable);	
功能描述:	设置是否允许实时末端速度推送。
参数说明:	enable: true 表示允许, false 表示不允许。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.6 注册用于获取实时末端速度的回调函数

int robotServiceRegisterRealTimeEndSpeedCallback(const RealTimeEndSpeedCallback ptr, void *arg);	
功能描述:	注册用于获取实时末端速度的回调函数。 注册回调函数后, 服务器会实时推送当前的末端速度。 频率 30ms。
参数说明:	1. ptr: 获取实时末端速度信息的回调函数指针。当 ptr==NULL 时, 相等于取消回调函数的注册, 取消该推送信息也可以通过该接口 robotServiceSetRealTimeEndSpeedPush 进行。 2. arg: 这个参数系统不做任何处理, 只是进行了缓存, 当系统调用已注册回调函数时, 该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.7 注册用于获取机械臂事件信息的回调函数

int robotServiceRegisterRobotEventInfoCallback(RobotEventCallback ptr, void *arg);	
功能描述:	注册用于获取机械臂事件信息的回调函数。 注册回调函数后, 服务器会实时推送机械臂事件信息。 频率 30ms。 注意: 关于事件信息的推送没有提供是否允许推送的接口, 因为机械臂的很多重要通知都是通过事件推送实现的, 所以事件信息是系统默认推送的, 不允许取消的。
参数说明:	1. ptr: 获取机械臂事件信息的函数指针。当 ptr==NULL 时, 相

	等于取消回调函数的注册。 2. arg : 这个参数系统不做任何处理，只是进行了缓存，当系统调用已注册回调函数时，该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.8 注册 movep 进度通知的回调函数

int robotServiceRegisterMovepStepNumNotifyCallback(RealTimeMovepStepNumNotifyCallback ptr, void *arg);	
功能描述:	注册 movep 进度通知的回调函数。
参数说明:	1. ptr : 获取机械臂 movep 进度通知的函数指针，当 ptr==NULL 时，相等于取消回调函数的注册。 2. arg : 这个参数系统不做任何处理，只是进行了缓存，当回调函数触发时，该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.2.9 注册日志输出回调函数

int robotServiceRegisterLogPrintCallback(RobotLogPrintCallback ptr, void *arg);	
功能描述:	注册日志输出回调函数。
参数说明:	1. ptr : 获取机械臂日志输出的函数指针，当 ptr==NULL 时，相等于取消回调函数的注册。 2. arg : 这个参数系统不做任何处理，只是进行了缓存，当回调函数触发时，该参数会通过回调函数的参数传回。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3 机械臂运动相关的接口

本部分接口是关于运动相关的接口，包括运动属性的设置和多种形式的运动。

注意：在运动之前应该设置好对应的运动属性。

本机械臂接口支持下面几种运动：

1. 关节运动：对应接口函数为 `robotServiceJointMove()`;
2. 直线运动：对应接口函数为 `robotServiceLineMove()`;
3. 轨迹运动：对应接口函数为 `robotServiceTrackMove()`;
4. 旋转运动：对应接口函数为 `robotServiceRotateMove()`;

根据上面运动的扩展运动：

1. 关节运动到目标位置：对应接口函数为 `robotMoveJointToTargetPositionByRelative()`, `robotMoveJointToTargetPosition()`;
2. 直线运动到目标位置：对应接口函数为 `robotMoveLineToTargetPositionByRelative()`, `robotMoveLineToTargetPosition()`;

注意：如果用到偏移量的话需要设置偏移量属性。如果用到了工具需要在运动之前设置工具的运动学参数（参见接口 `robotServiceSetToolKinematicsParam`）和动力学参数(参见接口 `robotServiceSetToolDynamicsParam`)。

2.3.1 初始化全局的运动属性

<code>int robotServiceInitGlobalMoveProfile();</code>	
功能描述:	初始化全局的运动属性。对运动属性进行初始化，将各个属性设置为初始值。调用此函数时机械臂不运动，该函数初始化各个运动属性为默认值。 初始化后各属性的默认值为： 0:关节型运动的最大速度和最大加速度属性：关节最大速度默认为 25 度每秒；关节最大加速度默认为 25 度每秒方。当运动类型为关节型运动时有效。 1:末端型运动的最大线速度和最大线加速度属性：末端最大速度默认为 0.03 米每秒；末端最大加速度默认为 0.03 米每秒方； 末端型运动的最大角速度和最大角加速度属性：末端最大速度默认为 100 度每秒；末端最大加速度默认为 100 度每秒方； 当运动类型为末端型运动时有效。 2:路点信息缓存属性：默认路点缓存为空，轨迹运动时使用。 3:交融半径属性：默认为 0.02 米。轨迹运动子类型为 MOVEP 时使用。 4:轨迹运动中圆轨迹的圈数属性：默认为 0，当轨迹运动类型等于 ARC_CIR 时该属性生效；且圆轨迹的圈数等于零时，ARC_CIR 代表圆弧，圆轨迹的圈数大于零时，ARC_CIR 代表圆。

	5:运动属性之偏移量属性: 默认没有偏移, 除示教运动外的所有运动有效。 6:工具参数属性属性: 无工具即工具的位置为 0 0 0。 7:示教坐标系属性: 示教坐标系为基座标系, 仅适用于示教运动。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。 失败: 返回错误号。

2.3.2 设置关节型运动的最大加速度

int robotServiceSetGlobalMoveJointMaxAcc(const aubo_robot_namespace::JointVelcAccParam &moveMaxAcc);	
功能描述:	设置关节型运动的最大加速度。 关节型运动包含: 1. 关节运动; 2. 示教运动中的关节示教 (JOINT1, JOINT2, JOINT3, JOINT4, JOINT5, JOINT6) 3. 轨迹运动下的 (JOINT_CUBICSPLINE, JOINT_UBSPLINEINPUT) 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxAcc: 关节运动的最大加速度, 单位 rad/s^2 。
返回值:	成功: 返回 ErrnoSucc。 失败: 返回错误号。

2.3.3 设置关节型运动的最大速度

int robotServiceSetGlobalMoveJointMaxVelc(const aubo_robot_namespace::JointVelcAccParam &moveMaxVelc);	
功能描述:	设置关节型运动的最大速度。 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxVelc: 关节运动的最大速度, 单位 rad/s 。
返回值:	成功: 返回 ErrnoSucc。 失败: 返回错误号。

2.3.4 获取关节型运动的最大加速度

void robotServiceGetGlobalMoveJointMaxAcc(aubo_robot_namespace::JointVelcAccParam &moveMaxAcc);	
功能描述:	获取关节型运动的最大加速度。
参数说明:	moveMaxAcc: 传出参数, 表示关节运动最大加速度, 单位 rad/s^2

返回值:	无。
------	----

2.3.5 获取关节型运动的最大速度

void robotServiceGetGlobalMoveJointMaxVelc(aubo_robot_namespace::JointVelcAccParam &moveMaxVelc);	
功能描述:	获取关节型运动的最大速度。
参数说明:	moveMaxVelc: 传出参数, 表示关节运动最大速度, 单位 rad/s 。
返回值:	无。

2.3.6 设置末端型运动的最大线加速度

int robotServiceSetGlobalMoveEndMaxLineAcc(double moveMaxAcc);	
功能描述:	设置末端型运动的最大加速度。 末端型包含: 1. 直线运动 (MODEL); 2. 示教运动中的位置示教和姿态示教 (MOV_X, MOV_Y, MOV_Z, ROT_X, ROT_Y, ROT_Z); 3. 轨迹运动下的 (ARC_CIR, CARTESIAN_MOVEP, CARTESIAN_CUBICSPLINE, CARTESIAN_UBSPLINEINTP) 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxAcc: 末端型运动的最大加速度, 单位 m/s^2 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.7 设置末端型运动的最大线速度

int robotServiceSetGlobalMoveEndMaxLineVelc(double moveMaxVelc);	
功能描述:	设置末端型运动的最大线速度。 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxVelc: 末端型运动的最大线速度, 单位 m/s 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.8 获取末端型运动的最大线加速度

void robotServiceGetGlobalMoveEndMaxLineAcc(double &moveMaxAcc);	
功能描述:	获取末端型运动最大线加速度。
参数说明:	moveMaxAcc: 传出参数, 末端型运动的最大线加速度, 单位 m/s^2
返回值:	无。

2.3.9 获取末端型运动的最大线速度

void robotServiceGetGlobalMoveEndMaxLineVelc(double &moveMaxVelc);	
功能描述:	获取末端型运动的最大线速度。
参数说明:	moveMaxVelc: 传出参数, 末端型运动最大速度, 单位 m/s 。
返回值:	无。

2.3.10 设置末端型运动的最大角加速度

int robotServiceSetGlobalMoveEndMaxAngleAcc(double moveMaxAcc);	
功能描述:	设置末端型运动的最大角加速度。 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxAcc: 末端型运动的最大角加速度, 单位 rad/s^2 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.11 设置末端型运动的最大角速度

int robotServiceSetGlobalMoveEndMaxAngleVelc(double moveMaxVelc);	
功能描述:	设置末端型运动的最大角速度。 注意: 用户在设置速度和加速度时, 需要根据运动的类型设置。
参数说明:	moveMaxVelc: 末端型运动的最大角速度, 单位 rad/s 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.12 获取末端型运动的最大角加速度

void robotServiceGetGlobalMoveEndMaxAngleAcc(double &moveMaxAcc);	
功能描述:	获取末端型运动的最大角加速度。
参数说明:	moveMaxAcc: 传出参数, 末端型运动的最大角加速度, 单位 rad/s^2 。
返回值:	无。

2.3.13 获取末端型运动的最大角速度

void robotServiceGetGlobalMoveEndMaxAngleVelc(double &moveMaxVelc);	
功能描述:	获取末端型运动的最大角速度。
参数说明:	moveMaxVelc: 传出参数, 末端型运动的最大角速度, 单位 rad/s 。
返回值:	无。

2.3.14 设置加加速度

int robotServiceSetJerkAccRatio(double acc);	
功能描述:	设置加加速度。
参数说明:	acc: 加加速度。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.15 获取加加速度

void robotServiceGetJerkAccRatio(double &acc);	
功能描述:	获取加加速度。
参数说明:	acc: 传出参数, 加加速度。
返回值:	无。

2.3.16 清除路点容器

void robotServiceClearGlobalWayPointVector();	
功能描述:	清除路点容器, 通常用于新的轨迹运动之前。
参数说明:	无。
返回值:	无。

2.3.17 添加路点

int robotServiceAddGlobalWayPoint(const aubo_robot_namespace::wayPoint_S &wayPoint);	
功能描述:	添加路点, 用于轨迹运动 robotServiceTrackMove 中。
参数说明:	wayPoint: 法兰盘中心基于基座标系的路点信息。 注意: wayPoint_S 可只传 jointpos (系统可计算出 position 和 orientation);而不能只传 position 和 orientation, 则表示零位。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceAddGlobalWayPoint(const double jointAngle[aubo_robot_namespace::ARM_DOF]);	
功能描述:	添加路点, 用于轨迹运动 robotServiceTrackMove 中。
参数说明:	jointAngle: 关节角。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.18 获取路点容器

void robotServiceGetGlobalWayPointVector(std::vector<aubo_robot_namespace::wayPoint_S> &wayPointVector);	
功能描述:	获取路点容器。
参数说明:	wayPointVector: 传出参数, 表示全局路点容器。
返回值:	无。

2.3.19 设置交融半径

int robotServiceSetGlobalBlendRadius(float value);	
功能描述:	设置交融半径。 交融半径的范围: 0.0m~0.05m。 注意: 交融半径必须大于 0.0。
参数说明:	value: 交融半径, 单位 <i>m</i> 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.20 获取交融半径

float robotServiceGetGlobalBlendRadius();	
功能描述:	获取交融半径。
参数说明:	无。
返回值:	返回交融半径值, 单位 <i>m</i> 。

2.3.21 获取圆轨迹时圆的圈数

int robotServiceGetGlobalCircularLoopTimes();	
功能描述:	运动属性之圆轨迹时圆的圈数。 注意: 当轨迹类型为 ARC_CIR 时有效。 当圆的圈数属性 (CircularLoopTimes) 为 0 时, 表示圆弧轨迹; 当圆的圈数属性 (CircularLoopTimes) 大于 0 时, 表示圆轨迹。
参数说明:	无。
返回值:	返回圆的圈数。 times = 0 时, 表示圆弧运动; times > 0 时, 表示圆运动, 且表示圆的圈数

2.3.22 设置圆轨迹时圆的圈数

void robotServiceSetGlobalCircularLoopTimes(int times);	
---	--

功能描述:	设置圆运动圈数。 注意：当轨迹类型为 ARC_CIR 时有效。
参数说明:	times: 圆的圈数。 times = 0 时，表示圆弧运动； times > 0 时，表示圆运动，且表示圆的圈数
返回值:	无。

2.3.23 设置运动属性中的偏移属性

int robotServiceSetMoveRelativeParam(const aubo_robot_namespace::MoveRelative &relativeMoveOnBase);	
功能描述:	设置基于基坐标系下的相对偏移属性。
参数说明:	relativeMoveOnBase: 基于基坐标系的偏移。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceSetMoveRelativeParam(const aubo_robot_namespace::MoveRelative &relativeMoveOnUserCoord, const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord);	
功能描述:	设置基于自定义坐标系下的相对偏移属性。
参数说明:	1. relativeMoveOnUserCoord: 基于下面参数 userCoord 下的偏移，包括使能、位置偏移和姿态偏移。 2. userCoord: 自定义坐标系。
返回值:	成功: 返回 ErrnoSucc
	失败: 返回错误号

2.3.24 设置无提前到位

int robotServiceSetNoArrivalAhead();	
功能描述:	设置无提前到位。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.25 设置提前到位距离模式

int robotServiceSetArrivalAheadDistanceMode(double distance);	
功能描述:	设置提前到位距离模式。 注意：跟随模式之提前到位，当前仅适用于关节运动。
参数说明:	distance: 距离，单位 m。
返回值:	成功: 返回 ErrnoSucc。

	失败: 返回错误号。
--	------------

2.3.26 设置提前到位时间模式

int robotServiceSetArrivalAheadTimeMode(double second);	
功能描述:	设置提前到位时间模式。 注意: 跟随模式之提前到位, 当前仅适用于关节运动。
参数说明:	second: 时间, 单位 s。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.27 设置提前到位交融半径模式

int robotServiceSetArrivalAheadBlendDistanceMode(double distance);	
功能描述:	设置提前到位交融半径模式。 注意: 跟随模式之提前到位, 当前仅适用于关节运动。
参数说明:	distance: 交融半径, 单位 m。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.28 关节运动

int robotServiceJointMove(aubo_robot_namespace::wayPoint_S &wayPoint, bool IsBolck);	
功能描述:	运动接口之关节运动。
参数说明:	- wayPoint: 路点信息。 - IsBolck: 是否阻塞。 IsBolck==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞, 立即返回, 运动指令发送成功就返回, 函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceJointMove(double jointAngle[aubo_robot_namespace::ARM_DOF], bool IsBolck);	
功能描述:	运动接口之关节运动。
参数说明:	- jointAngle: 关节角。 - IsBlock: 是否阻塞。 IsBolck==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。

	IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

<pre>int robotServiceJointMove(aubo_robot_namespace::MoveProfile_t &moveProfile, double jointAngle[aubo_robot_namespace::ARM_DOF], bool IsBolck);</pre>	
功能描述:	运动接口之关节运动。
参数说明:	1. moveProfile: 偏移属性。 2. jointAngle: 关节角。 3. IsBlock: 是否阻塞。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.29 保持当前位姿通过关节运动的方式运动到目标位置, 其中目标位置是通过相对当前位置的偏移给出的

<pre>int robotMoveJointToTargetPositionByRelative(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::MoveRelative &relativeMoveOnUserCoord, bool IsBolck = false);</pre>	
功能描述:	保持当前位姿通过关节运动的方式运动到目标位置，其中目标位置是通过相对当前位置的偏移给出。
参数说明:	1. userCoord: 参考坐标系。 2. relativeMoveOnUserCoord: 参考坐标系下的偏移。 3. IsBolck: 是否阻塞。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.30 保持当前位姿通过关节运动的方式运动到目标位置

```
int robotMoveJointToTargetPosition(
```

<pre>const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::Pos &toolEndPositionOnUserCoord, const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc, bool IsBolck = false);</pre>	
功能描述:	保持当前姿态通过轴动运动的方式运动到目标位置。
参数说明:	1. userCoord: 参考坐标系。 2. toolEndPositionOnUserCoord: 工具坐标系在参考坐标系下的位置参数。 3. toolInEndDesc: 工具参数, 当没有使用工具时, 将此参数设置为 0。 4. IsBolck: 是否阻塞。 IsBolck==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞, 立即返回, 运动指令发送成功就返回, 函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.31 基于跟随模式的轴动

<pre>int robotServiceFollowModeJointMove(double jointAngle[aubo_robot_namespace::ARM_DOFS]);</pre>	
功能描述:	基于跟随模式的轴动接口。
参数说明:	jointAngle: 关节角。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.32 直线运动

<pre>int robotServiceLineMove(aubo_robot_namespace::wayPoint_S &wayPoint, bool IsBolck);</pre>	
功能描述:	运动接口之直线运动, 属于末端型运动。
参数说明:	1. wayPoint: 路点。 2. IsBlock: 是否阻塞。 IsBolck==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞, 立即返回, 运动指令发送成功就返回, 函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

```
int robotServiceLineMove(double jointAngle[aubo_robot_namespace::ARM_DOFS], boo
```

l IsBlock);	
功能描述:	运动接口之直线运动，属于末端型运动。
参数说明:	1. jointAngle: 关节角。 2. IsBlock: 是否阻塞。 IsBlock==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBlock==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceLineMove(aubo_robot_namespace::MoveProfile_t &moveProfile, double jointAngle[aubo_robot_namespace::ARM_DOF], bool IsBlock);	
功能描述:	运动接口之直线运动，属于末端型运动。
参数说明:	1. moveProfile: 运动偏移量属性。 2. jointAngle: 关节角。 3. IsBlock: 是否阻塞。 IsBlock==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBlock==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.33 保持当前位姿通过直线运动的方式运动到目标位置, 其中目标位置是通过相对当前位置的偏移给出

int robotMoveLineToTargetPositionByRelative(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::MoveRelative &relativeMoveOnUserCoord, bool IsBlock);	
功能描述:	保持当前位姿通过直线运动的方式运动到目标位置，其中目标位置是通过相对当前位置的偏移给出
参数说明:	1. userCoord: 参考坐标系。 2. relativeMoveOnUserCoord: 参考坐标系 userCoord 下的偏移。 3. IsBlock: 是否阻塞。 IsBlock==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBlock==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。

失败: 返回错误号。

2.3.34 保持当前位姿通过直线运动的方式运动到目标位置

<pre>int robotMoveLineToTargetPosition(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::Pos &toolEndPositionOnUserCoord, const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc, bool IsBlock = false);</pre>	
功能描述:	保持当前位姿通过直线运动的方式运动到目标位置。
参数说明:	1. userCoord: 目标位置 (positionOnUserCoord) 的参考坐标系。该坐标系参数 (userCoord), 表示下面的目标位置(positionOnUserCoord)是基于该平面的。 2. positionOnUserCoord: 工具坐标系在参考坐标系下的位置参数。基于用户平面表示的目标位置。工具末端点的位置信息 (目标位置 x,y,z), 工具参数通过下个参数 (toolInEndDesc) 给出; 3. toolInEndDesc: 工具参数, 当没有使用工具时, 将此参数设置为 0; 4. IsBlock: 是否阻塞。 IsBlock==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。 IsBlock==false 代表非阻塞, 立即返回, 运动指令发送成功就返回, 函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.35 保持当前位置变换姿态做旋转运动

<pre>int robotServiceRotateMove(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const double rotateAxisOnUserCoord[3], double rotateAngle, bool IsBlock);</pre>	
功能描述:	运动接口之保持当前位置变换姿态做旋转运动, 即机械臂的末端点绕着指定转轴做旋转运动 (位置保持不变)。
参数说明:	1. userCoord: 为转轴 (rotateAxisOnUserCoord) 的参考坐标系。 2. rotateAxisOnUserCoord: 转轴$[x,y,z]$。 3. rotateAngle: 绕转轴转动的转角, 单位rad。 4. IsBlock: 是否阻塞。 IsBlock==true 代表阻塞, 机械臂运动直到到达目标位置或者出现故障后返回。 IsBlock==false 代表非阻塞, 立即返回, 运动指令发送成功就

	返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.36 根据当前位姿及基坐标系下表示的旋转轴、旋转角，获取目标位姿

<pre>int robotServiceGetRotateTargetWaypion(const aubo_robot_namespace::wayPoint_S &originateWayPointOnBaseCoord, const double rotateAxisOnBaseCoord[], double rotateAngle, aubo_robot_namespace::wayPoint_S &targetWayPointOnBaseCoord);</pre>	
功能描述:	根据当前位姿及基坐标系下表示的旋转轴、旋转角，获取目标位姿
参数说明:	1. originateWayPointOnBaseCoord: 起始路点信息。 2. rotateAxisOnBaseCoord: 基坐标系下表示的旋转轴。 3. rotateAngle: 旋转角。 4. targetWayPointOnBaseCoord: 传出参数，目标路点信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.37 将用户坐标系描述的旋转轴变换到基坐标系下描述

<pre>int robotServiceGetRotateAxisUserToBase(const aubo_robot_namespace::Ori &oriOnUserCoord, const double rotateAxisOnUserCoord[], double rotateAxisOnBaseCoord[]);</pre>	
功能描述:	将用户坐标系描述的旋转轴变换到基坐标系下描述的旋转轴。
参数说明:	1. oriOnUserCoord: 用户坐标系姿态。 2. rotateAxisOnUserCoord: 用户坐标系下描述的旋转轴。 3. rotateAxisOnBaseCoord: 传出参数，基坐标系下描述的旋转轴
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.38 旋转运动到目标路点

<pre>int robotServiceRotateMoveToWaypoint(const aubo_robot_namespace::waypoint_S &targetWayPointOnBaseCoord, bool IsBolck);</pre>	
功能描述:	旋转运动到目标路点。
参数说明:	1. targetWayPointOnBaseCoord: 在基坐标系下的目标路点。

	2. IsBlock: 是否阻塞。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。 失败: 返回错误号。

2.3.39 轨迹运动

<pre>int robotServiceTrackMove(aubo_robot_namespace::move_track subMoveMode, bool IsBolck);</pre>	
功能描述:	轨迹运动。
参数说明:	1. subMoveMode: 轨迹运动类型。 - 当 subMoveMode==ARC_CIR, CARTESIAN_MOVEP, CARTESIAN_CUBICSPLINE, CARTESIAN_UBSPLINEINTP 时, 该运动属于末端型运动。 - 当 subMoveMode==JIONT_CUBICSPLINE, JOINT_UBSPLINEINTP 时, 该运动属于关节型运动。 - 当 subMoveMode==ARC_CIR, 表示圆或者圆弧。当圆的圈数属性 (CircularLoopTimes) 为 0 时, 表示圆弧轨迹; 当圆的圈数属性 (CircularLoopTimes) 大于 0 时, 表示圆轨迹。 - 当 subMoveMode==CARTESIAN_MOVEP, 表示 MOVEP 轨迹, 需要用户这只交融半径的属性。 - 当 subMoveMode==JOINT_UBSPLINEINTP, 轨迹复现接口。 2. IsBolck: 是否阻塞。 IsBolck==true 代表阻塞，机械臂运动直到到达目标位置或者出现故障后返回。 IsBolck==false 代表非阻塞，立即返回，运动指令发送成功就返回，函数返回后机械臂开始运动。
返回值:	成功: 返回 ErrnoSucc。 失败: 返回错误号。

2.3.40 开始示教

<pre>int robotServiceTeachStart(aubo_robot_namespace::teach_mode mode, bool direction);</pre>	
功能描述:	开始示教。
参数说明:	1. mode: 示教模式。 示教关节: JOINT1, JOINT2, JOINT3, JOINT4, JOINT5, JOINT6。

	2) 位置示教: MOV_X, MOV_Y, MOV_Z。 3) 姿态示教: ROT_X, ROT_Y, ROT_Z。 2. direction: 运动方向, 正方向 true, 反方向 false。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.41 设置示教运动的坐标系

int robotServiceSetTeachCoordinateSystem(const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &coordSystem);	
功能描述:	设置示教运动的坐标系。
参数说明:	coordSystem: 通过该参数确定一个坐标系, 具体使用参考类型定义处的使用说明。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.42 结束示教

int robotServiceTeachStop();	
功能描述:	示教停止。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.43 清除离线轨迹路点

int robotServiceOfflineTrackWaypointClear ();	
功能描述:	清除服务器离线轨迹的路点。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.44 添加离线轨迹路点

int robotServiceOfflineTrackWaypointAppend(const std::vector<aubo_robot_namespace::wayPoint_S> &wayPointVector);	
功能描述:	通过路点容器添加离线轨迹路点到服务器。
参数说明:	wayPointVector: 路点容器, 容器最大允 4000 个路点。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceOfflineTrackWaypointAppend(const char *fileName);	
功能描述:	通过路点文件的形式添加离线轨迹路点到服务器。
参数说明:	fileName: 路点文件。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.45 开始离线轨迹运动

int robotServiceOfflineTrackMoveStartup (bool IsBolck);	
功能描述:	启动离线轨迹的运行。
参数说明:	IsBolck: 调用接口时是否阻塞。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.46 结束离线轨迹运动

int robotServiceOfflineTrackMoveStop();	
功能描述:	结束离线轨迹运动。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.47 进入 TCP 转 CAN 透传模式

int robotServiceEnterTcp2CanbusMode();	
功能描述:	进入 TCP 转 CAN 透传模式。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.48 发送坐标数据到关节 CAN 总线

int robotServiceSetRobotPosData2Canbus(double jointAngle[aubo_robot_namespace::ARM_DOF]);	
功能描述:	发送坐标数据到关节 CAN 总线。 通过透传将关节角度信息发送至驱动器。
参数说明:	jointAngle: 关节角, 单位 <i>rad</i> 。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceSetRobotPosData2Canbus(const std::vector<aubo_robot_namespace::wayPoint_S> &wayPointVector);	
功能描述:	发送坐标数据到关节 CAN 总线。 通过透传将路点容器发送至驱动器。
参数说明:	wayPointVector: 路点容器
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.3.49 退出 TCP 转 CAN 透传模式

int robotServiceLeaveTcp2CanbusMode();	
功能描述:	退出 TCP 转 CAN 透传模式。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4 工具接口

2.4.1 正解

int robotServiceRobotFk(const double *jointAngle, int size, aubo_robot_namespace::waypoint_S &wayPoint);	
功能描述:	正解, 此函数为正解函数, 已知关节角求对应位置的位置和姿态。
参数说明:	1. jointAngle: 六个关节的关节角, 单位 rad 。 2. size: 关节角数组长度, 规定为 6 个。 3. wayPoint: 传出参数, 正解得到得路点。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.2 逆解

int robotServiceRobotIk(const double *startPointJointAngle, const aubo_robot_namespace::Pos &position, const aubo_robot_namespace::Ori &ori, aubo_robot_namespace::wayPoint_S &wayPoint);	
功能描述:	逆解。根据位置信息(x,y,z)和对应位置的参考姿态(w,x,y,z)得到对应位置的关节角信息。 逆运动学问题: 对某个机器人, 当给出机器人手部(法兰盘中心)

	在基坐标系中所处的位置和姿态时，求出其对应的关节角信息。
参数说明:	1. startPointJointAngle: 参考点六个关节的关节角，通常为当前机械臂的位置，单位rad。 2. position: 目标路点的位置。 3. ori: 目标路点的参考姿态。 例如：可以获取当前位置位姿作为此参数，这样相当于保持当前姿态。 4. wayPoint: 传出参数，目标路点信息。
返回值:	成功: 返回 <code>ErrnoSucc</code> 。
	失败: 返回错误号。

<pre>int robotServiceRobotIk(const aubo_robot_namespace::Pos &position, const aubo_robot_namespace::Ori &ori, std::vector<aubo_robot_namespace::wayPoint_S> &wayPointVector);</pre>	
功能描述:	逆解，此函数为机械臂逆解函数，根据位置信息(x,y,z)和对应位置的参考姿态(w,x,y,z)得到对应位置的关节角信息。
参数说明:	1. position: 目标路点的位置。 2. ori: 目标路点的参考姿态。 3. wayPointVector: 传出参数，目标路点容器。
返回值:	成功: 返回 <code>ErrnoSucc</code> 。
	失败: 返回错误号。

2.4.3 基坐标系转用户坐标系

<pre>static int baseToUserCoordinate(const aubo_robot_namespace::Pos &flangeCenterPositionOnBase, const aubo_robot_namespace::Ori &flangeCenterOrientationOnBase, const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc, aubo_robot_namespace::Pos &toolEndPositionOnUserCoord, aubo_robot_namespace::Ori &toolEndOrientationOnUserCoord);</pre>	
功能描述:	将法兰盘中心基于基坐标系下的位置和姿态转成工具末端基于用户坐标系下的位置和姿态。 扩展 1: 法兰盘中心可以看成是一个特殊的工具，即工具的位置为(0,0,0)姿态为(1,0,0,0)。 因此当工具为(0,0,0)时，相当于将法兰盘中心基于基坐标系下的位置和姿态转成法兰盘中心基于用户坐标系下的位置和姿态。 扩展 2: 用户坐标系也可以选择成基坐标系，即：<code>userCoord.coordType = BaseCoordinate</code>。 因此当用户平面为基坐标系时，相当于将法兰盘中心基于基坐标系下的位置和姿态转成工具末端基于基坐标系下的位置和姿态，即在基坐标系加工具。

参数说明:	1. flangeCenterPositionOnBase: 法兰盘中心基于基坐标系下的位置信息 (x,y,z)。 2. flangeCenterOrientationOnBase: 法兰盘中心基于基坐标系下的姿态信息(w, x, y, z)。 3. userCoord: 用户坐标系。 4. toolInEndDesc: 工具参数。 5. toolEndPositionOnUserCoord: 传出参数, 工具末端基于用户坐标系下的位置信息。 6. toolEndOrientationOnUserCoord: 传出参数, 工具末端基于用户坐标系下的姿态信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.4 基坐标系转基坐标得到工具末端点的位置和姿态

<pre>static int baseToBaseAdditionalTool(const aubo_robot_namespace::Pos &flangeCenterPositionOnBase, const aubo_robot_namespace::Ori &flangeCenterOrientationOnBase, const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc, aubo_robot_namespace::Pos &toolEndPositionOnBase, aubo_robot_namespace::Ori &toolEndOrientationOnBase);</pre>	
功能描述:	法兰盘中心在基坐标系下的位置和姿态转工具末端在基坐标系下的位置和姿态。
参数说明:	1. flangeCenterPositionOnBase: 法兰盘中心基于基坐标系下的位置信息。 2. flangeCenterOrientationOnBase: 法兰盘中心基于基坐标系下的姿态信息。 3. toolInEndDesc: 工具参数。 4. toolEndPositionOnUserCoord: 传出参数, 工具末端基于基坐标系下的位置信息。 5. toolEndOrientationOnUserCoord: 传出参数, 工具末端基于基坐标系下的姿态信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.5 用户坐标系位置和姿态信息转基坐标系下的位置和姿态

<pre>static int userToBaseCoordinate(const aubo_robot_namespace::Pos &toolEndPositionOnUserCoord, const aubo_robot_namespace::Ori &toolEndOrientationOnUserCoord, const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoord, const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc,</pre>	
--	--

aubo_robot_namespace::Pos &flangeCenterPositionOnBase, aubo_robot_namespace::Ori &flangeCenterOrientationOnBase);	
功能描述:	将工具末端基于用户坐标系下的位置和姿态转成法兰盘中心基于基坐标系下的位置和姿态。 扩展 1: 法兰盘中心可以看成是一个特殊的工具, 即工具的位置为 (0,0,0) 姿态为 (1,0,0,0) 因此当工具的位置为 (0,0,0) 姿态为 (1,0,0,0) 时, 表示 toolEndPositionOnUserCoord 和 toolEndOrientationOnUserCoord 是无工具的。 扩展 2: 用户坐标系也可以选择成基坐标系, 即: userCoord.coordType = BaseCoordinate。 因此当用户平面为基坐标系时, 相当于将工具末端基于基坐标系下的位置和姿态转成法兰盘中心基于基坐标系下的位置和姿态。 扩展 3: 利用该函数和逆解组合实现。当用户提供自定义坐标系 (特殊为基坐标系) 下工具末端的位置和姿态得到基坐标系下法兰盘中心的位置和姿态, 然后在逆解, 得到目标路点。
参数说明:	1. toolEndPositionOnUserCoord: 工具末端基于用户坐标系下的位置信息。 2. toolEndOrientationOnUserCoord: 工具末端基于用户坐标系下的姿态信息。 3. userCoord: 参考坐标系, 通过该参数确定一个坐标系。 4. toolInEndDesc: 工具参数。 5. flangeCenterPositionOnBase: 传出参数, 法兰盘中心基于基坐标系下的位置信息。 6. flangeCenterOrientationOnBase: 传出参数, 法兰盘中心基于基坐标系下的姿态信息。 注意: 这个的坐标系转换不支持末端系, 即不支持 userCoord==EndCoordinate, 如果 userCoord==EndCoordinate 会报参数错误(Error Code_ParamError)。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.6 将空间内一个基于用户坐标系的位置信息 (x, y, z) 转换成基于基坐标系下的位置信息 (x, y, z)

static int userCoordPointToBasePoint(const aubo_robot_namespace::Pos &userCoordPoint, const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoordSystem, aubo_robot_namespace::Pos &basePoint);	
功能描述:	将空间内一个基于用户坐标系的位置信息(x,y,z)转换成基于基坐标系下的位置信息(x,y,z)。同一个工具参数。
参数说明:	1. userCoordPoint: 用户坐标系下的位置信息 x,y,z (必须的基于

	下面参数提供的坐标系下的) 2. userCoordSystem: 用户坐标系描述, 通过该参数确定一个坐标系。 3. basePoint: 传出参数, 基于基坐标系下的位置参数。 注意: 这个的坐标系转换不支持末端系, 即不支持 <code>userCoord==EndCoordinate</code>, 如果 <code>userCoord==EndCoordinate</code> 会报参数错误(<code>ErrCode_ParamError</code>)。
返回值:	成功: 返回 <code>ErrnoSucc</code>。 失败: 返回错误号。

2.4.7 法兰盘姿态转成工具姿态

<pre>static int endOrientation2ToolOrientation(aubo_robot_namespace::Ori &tcpOriInEnd, const aubo_robot_namespace::Ori &endOri, aubo_robot_namespace::Ori &toolOri);</pre>	
功能描述:	将法兰盘中心的姿态转为工具末端的姿态。
参数说明:	1. tcpOriInEnd: 工具姿态参数。 2. endOri: 法兰盘中心姿态。 3. toolOri: 传出参数, 工具姿态。
返回值:	成功: 返回 <code>ErrnoSucc</code>。 失败: 返回错误号。

2.4.8 工具姿态转成法兰盘姿态

<pre>static int toolOrientation2EndOrientation(aubo_robot_namespace::Ori &tcpOriInEnd, const aubo_robot_namespace::Ori &toolOri, aubo_robot_namespace::Ori &endOri);</pre>	
功能描述:	将工具姿态转为法兰盘中心姿态。
参数说明:	1. tcpOriInEnd: 工具姿态参数。 2. toolOri: 工具姿态 3. endOri: 传出参数, 法兰盘中心姿态。
返回值:	成功: 返回 <code>ErrnoSucc</code>。 失败: 返回错误号。

2.4.9 根据位置获取目标路点信息

<pre>static int getTargetWaypointByPosition(const aubo_robot_namespace::wayPoint_S &sourceWayPointOnBaseCoord, const aubo_robot_namespace::CoordCalibrateByJointAngleAndTool &userCoordSystem, const aubo_robot_namespace::Pos &toolEndPosition,</pre>	
---	--

const aubo_robot_namespace::ToolInEndDesc &toolInEndDesc, aubo_robot_namespace::wayPoint_S &targetWayPointOnBaseCoord);	
功能描述:	根据位置参数求得目标路点信息（姿态同源路点）
参数说明:	1. sourceWayPointOnBaseCoord: 源路点。 2. userCoordSystem: 坐标系。 3. toolEndPosition: 末端工具位置。 4. toolInEndDesc: 末端工具参数。 5. targetWayPointOnBaseCoord: 传出参数，目标路点。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.10 四元数转欧拉角

int quaternionToRPY(const aubo_robot_namespace::Ori &ori, aubo_robot_namespace::Rpy &rpy);	
功能描述:	四元数转欧拉角。
参数说明:	1. ori: 姿态的四元数表示方法。 2. rpy: 传出参数，姿态的欧拉角表示方法。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.11 欧拉角转四元数

int RPYToQuaternion(const aubo_robot_namespace::Rpy &rpy, aubo_robot_namespace::Ori &ori);	
功能描述:	欧拉角转四元数。
参数说明:	1. rpy: 姿态的欧拉角表示方法。 2. ori: 传出参数，姿态的四元素表示方法。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.4.12 根据错误号获取错误信息

std::string getErrDescByCode(aubo_robot_namespace::RobotErrorCode code);	
功能描述:	根据错误号获取错误信息
参数说明:	code: 错误号。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.5 机械臂控制接口

2.5.1 机械臂运动控制：停止、暂停、继续

int rootServiceRobotMoveControl(aubo_robot_namespace::RobotMoveControlCommand cmd);	
功能描述:	机械臂运动控制：停止，暂停，继续。
参数说明:	cmd: 控制命令。 注意：RobotMoveControl 需要在与 move 不同的线程中调用。且 robotMoveStop 调用后需要停止 move 线程，否则会接着执行下一个 move 指令。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.5.2 机械臂控制

int rootServiceRobotControl(const aubo_robot_namespace::RobotControlCommand cmd);	
功能描述:	机械臂控制。
参数说明:	cmd: 控制命令。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.5.3 机械臂快速停止

int robotMoveFastStop();	
功能描述:	机械臂快速停止。 注意：robotMoveFastStop 需要在与 move 不同的线程中调用。且 robotMoveFastStop 调用后需要停止 move 线程，否则会接着执行下一个 move 指令。
参数说明:	无
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.5.4 机械臂运动停止

int robotMoveStop();	
功能描述:	机械臂运动停止。 注意：robotMoveStop 需要在与 move 不同的线程中调用。
参数说明:	无
返回值:	成功: 返回 ErrnoSucc。

失败: 返回错误号。

2.5.5 设置机械臂的电源状态

int robotServicePowerControl(bool value);	
功能描述:	设置机械臂的电源状态。
参数说明:	value: 机械臂电源状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.5.6 松刹车

int robotServiceReleaseBrake();	
功能描述:	机械臂松刹车。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6 末端工具接口

2.6.1 设置无工具的动力学参数

int robotServiceSetNoneToolDynamicsParam();	
功能描述:	设置无工具的动力学参数。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6.2 设置工具的动力学参数

int robotServiceSetToolDynamicsParam(const aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam);	
功能描述:	设置工具的动力学参数。
参数说明:	toolDynamicsParam: 工具动力学参数。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6.3 获取工具的动力学参数

int robotServiceGetToolDynamicsParam(aubo_robot_namespace::ToolDynamicsParam &toolDynamicsParam);	
功能描述:	获取工具的动力学参数。
参数说明:	toolDynamicsParam: 传出参数, 工具动力学参数。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6.4 设置无工具的运动学参数

int robotServiceSetNoneToolKinematicsParam();	
功能描述:	设置无工具运动学参数。
参数说明:	无。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6.5 设置工具的运动学参数

int robotServiceSetToolKinematicsParam(const aubo_robot_namespace::ToolKinematicsParam &toolKinematicsParam);	
功能描述:	设置工具的运动学参数。
参数说明:	toolKinematicsParam: 工具运动学参数。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.6.6 获取工具的运动学参数

int robotServiceGetToolKinematicsParam(aubo_robot_namespace::ToolKinematicsParam &toolKinematicsParam);	
功能描述:	获取工具的运动学参数
参数说明:	toolKinematicsParam: 传出参数, 工具运动学参数。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7 设置和获取机械臂相关参数接口

2.7.1 获取当前的连接状态

void robotServiceGetConnectStatus(bool &connectStatus);	
功能描述:	获取当前的连接状态。 该函数用于查看与机械臂服务器的连接状态。
参数说明:	connectStatus: 传出参数, 网络处于连通状态返回 true; 否则返回 false。
返回值:	无。

2.7.2 设置当前机械臂模式：仿真或真实

int robotServiceSetRobotWorkMode(aubo_robot_namespace::RobotWorkMode mode);	
功能描述:	设置当前机械臂模式：仿真或真实。
参数说明:	mode: 仿真或真实的枚举类型。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.3 获取当前机械臂模式

int robotServiceGetRobotWorkMode(aubo_robot_namespace::RobotWorkMode &mode);	
功能描述:	获取机械臂当前工作模式。
参数说明:	mode: 传出参数, 仿真或真实的枚举类型, 表示机械臂当前模式。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.4 获取重力分量

int robotServiceGetRobotGravityComponent(aubo_robot_namespace::RobotGravityComponent &gravityComponent);	
功能描述:	获取重力分量。
参数说明:	gravityComponent: 传出参数, 重力分量, 需连接真实机器人。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.5 获取当前碰撞等级

int robotServiceGetRobotCollisionCurrentService(int &collisionGrade);	
功能描述:	获取碰撞等级。
参数说明:	collisionGrade: 传出参数, 碰撞等级, 需连接真实机器人。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.6 设置碰撞等级

int robotServiceSetRobotCollisionClass(int grade);	
功能描述:	设置碰撞等级。
参数说明:	grade: 碰撞等级: 0~10。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

int robotServiceSetRobotCollisionClass(int grade, aubo_robot_namespace::CollisionMode collisionMode);	
功能描述:	设置碰撞等级。
参数说明:	1. grade: 碰撞等级: 0~10。 2. collisionMode: 碰撞模式。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.7 获取设备信息

int robotServiceGetRobotDevInfoService(aubo_robot_namespace::RobotDevInfo &devInfo);	
功能描述:	获取机器人设备信息, 需连接真实机器人。
参数说明:	devInfo: 设备信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.8 获取是否存在真实机械臂

int robotServiceGetIsRealRobotExist(bool &value);	
功能描述:	获取是否存在真实机械臂
参数说明:	value: 传出参数, true: 存在真实机械臂; false: 不存在真实机械臂。
返回值:	成功: 返回 ErrnoSucc。

	失败: 返回错误号。
--	------------

2.7.9 获取 6 关节旋转 360 使能标志

int robotServiceGetJoint6Rotate360EnableFlag(bool &value);	
功能描述:	获取 6 关节旋转 360 使能标志。
参数说明:	value: 传出参数, J6 是否为 360 度版本。true: 360 度版本; false: 普通版本。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.10 获取机械臂关节状态

int robotServiceGetRobotJointStatus(aubo_robot_namespace::JointStatus *jointStatus, int size);	
功能描述:	获取机械臂关节状态
参数说明:	1. jointStatus: 传出参数, 关节状态。 2. size: 关节角缓冲区长度, 6。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.11 获取机械臂诊断信息

int robotServiceGetRobotDiagnosisInfo(aubo_robot_namespace::RobotDiagnosis &robotDiagnosisInfo);	
功能描述:	获取机械臂诊断信息。
参数说明:	robotDiagnosisInfo: 传出参数, 机械臂诊断信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.12 获取机械臂当前关节角信息

int robotServiceGetJointAngleInfo(aubo_robot_namespace::JointParam &jointParam);	
功能描述:	获取机械臂当前关节角信息。
参数说明:	jointParam: 传出参数, 关节角信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.13 获取机械臂当前路点信息

int robotServiceGetCurrentWaypointInfo(aubo_robot_namespace::wayPoint_S &waypoint);	
功能描述:	获取机械臂当前路点信息。
参数说明:	wayPoint: 传出参数, 当前路点信息。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.14 当前机械臂是否运行在联机模式

int robotServiceIsOnlineMode(bool &isOnlineMode);	
功能描述:	返回当前机械臂是否运行在联机模式。
参数说明:	isOnlineMode: 传出参数, true: 联机 false: 非联机。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.15 当前机械臂是否运行在联机主模式

int robotServiceIsOnlineMasterMode(bool &isOnlineMasterMode);	
功能描述:	返回当前机械臂是否运行在联机主模式。
参数说明:	isOnlineMode: 传出参数, true: 联机主模式/手动模式 false: 联机从模式。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.16 获取机械臂当前运行状态

int robotServiceGetRobotCurrentState(aubo_robot_namespace::RobotState &state);	
功能描述:	获取机械臂当前运行状态。 注意: 需要与 move 在不同线程里。
参数说明:	state: 传出参数, 运行状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.17 获取 MAC 通信状态

int robotServiceGetMacCommunicationStatus(bool &value);	
功能描述:	获取 MAC 通信状态。
参数说明:	value: 传出参数, 通信状态. true: 连接成功 false: 连接失败。

返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.7.18 获取机械臂安全配置

int robotServiceGetRobotSafetyConfig(aubo_robot_namespace::RobotSafetyConfig &safetyConfig);	
功能描述:	获取机械臂安全配置。
参数说明:	safetyConfig: 安全配置。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.8 接口板 IO 相关的接口

接口板 IO 主要分为两大块，分别是控制器 IO 和用户 IO，每个 IO 都有对应的名称和地址，在使用是可以通过名称或地址两种形式获取和设置 IO 的状态。

2.8.1 获取接口板指定 IO 集合的配置信息

int robotServiceGetBoardIOConfig(const std::vector<aubo_robot_namespace::RobotIoType> &ioType, std::vector<aubo_robot_namespace::RobotIoDesc> &configVector);	
功能描述:	获取接口板指定 IO 集合的配置信息。
参数说明:	1. ioType: IO 类型的集合。 2. configVector: 传出参数，IO 配置信息的集合。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.8.2 获取接口板指定 IO 集合的状态信息

int robotServiceGetBoardIOStatus(const std::vector<aubo_robot_namespace::RobotIoType> ioType, std::vector<aubo_robot_namespace::RobotIoDesc> &statusVector);	
功能描述:	获取接口板指定 IO 集合的状态信息。
参数说明:	1. ioType: IO 类型的集合。 2. statusVector: 传出参数，IO 状态信息的集合。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.8.3 设置接口板 IO 状态

<pre>int robotServiceSetBoardIOStatus(aubo_robot_namespace::RobotIoType type, std::string name, double value);</pre>	
功能描述:	根据接口板 IO 类型和名称设置 IO 状态。
参数说明:	1. type: IO 类型。 2. name: IO 名称。 3. value: IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

<pre>int robotServiceSetBoardIOStatus(aubo_robot_namespace::RobotIoType type, int addr, double value);</pre>	
功能描述:	根据接口板 IO 类型和地址设置 IO 状态。
参数说明:	1. type: IO 类型。 2. addr: IO 地址。 3. value: IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

<pre>int robotServiceGetBoardIOStatus(aubo_robot_namespace::RobotIoType type, std::string name, double &value);</pre>	
功能描述:	根据接口板 IO 类型和名称获取 IO 状态。
参数说明:	1. type: IO 类型。 2. name: IO 名称。 3. value: 传出参数, IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

<pre>int robotServiceGetBoardIOStatus(aubo_robot_namespace::RobotIoType type, int addr, double &value);</pre>	
功能描述:	根据接口板 IO 类型和地址获取 IO 状态。
参数说明:	1. type: IO 类型。 2. addr: IO 地址。 3. value: 传出参数, IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9 工具 IO 相关的接口

2.9.1 设置工具端电源电压类型

int robotServiceSetToolPowerVoltageType(aubo_robot_namespace::ToolPowerType type);	
功能描述:	设置工具端电源电压类型。
参数说明:	type: 工具电源电压类型。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.2 获取工具端电源电压类型

int robotServiceGetToolPowerVoltageType(aubo_robot_namespace::ToolPowerType &type);	
功能描述:	获取工具端电源电压类型。
参数说明:	type: 传出参数, 工具电源电压类型。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.3 获取工具端的电源电压

int robotServiceGetToolPowerVoltageStatus(double &value);	
功能描述:	获取工具端的电源电压。
参数说明:	value: 传出参数, 工具端的电源电压。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.4 设置工具端电源电压类型和所有数字量 IO 的类型

int robotServiceSetToolPowerTypeAndDigitalIOType(aubo_robot_namespace::ToolPowerType type, aubo_robot_namespace::ToolIOType io0, aubo_robot_namespace::ToolIOType io1, aubo_robot_namespace::ToolIOType io2);	
功能描述:	设置工具端电源电压类型 and 所有数字量 IO 的类型。
参数说明:	1. type 电源电压类型。 2. io0 Tool IO 0 的类型。 3. io1 Tool IO 1 的类型。 4. io2 Tool IO 2 的类型。

	5. io3 Tool IO 3 的类型。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.5 设置工具端数字量 IO 的类型：输入或者输出

int robotServiceSetToolDigitalIOType(aubo_robot_namespace::ToolDigitalIOAdd addr, aubo_robot_namespace::ToolIOType type);	
功能描述:	设置工具端数字量 IO 的类型：输入或者输出。
参数说明:	1. addr: IO 地址。 2. type: IO 类型：输入或者输出。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.6 获取工具端所有数字量 IO 的状态

int robotServiceGetAllToolDigitalIOStatus(std::vector<aubo_robot_namespace::RobotIO Desc> &statusVector);	
功能描述:	获取工具端所有数字量 IO 的状态。
参数说明:	statusVector: 传出参数，IO 状态容器。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.7 根据地址设置工具端数字量 IO 的状态

int robotServiceSetToolIOStatus(aubo_robot_namespace::ToolDigitalIOAddr addr, aubo_robot_namespace::IO_STATUS value);	
功能描述:	根据地址设置工具端数字量 IO 的状态。
参数说明:	1. addr: IO 地址。 2. value: IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.8 根据名称设置工具端数字量 IO 的状态

int robotServiceSetToolIOStatus(std::string name, aubo_robot_namespace::IO_STATUS value);	
功能描述:	根据名称设置工具端数字量 IO 的状态。
参数说明:	1. name: IO 名称。 2. value: IO 状态值。

返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.9 根据名称获取工具端 IO 的状态

int robotServiceGetToolIoStatus(std::string name, double &value);	
功能描述:	根据名称获取工具端 IO 的状态。
参数说明:	1. name: IO 名称。 2. value: 传出参数, IO 状态。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

2.9.10 获取工具端所有 AI 的状态

int robotServiceGetAllToolAIStatus (std::vector<aubo_robot_namespace::RobotIoDesc> &statusVector);	
功能描述:	获取工具端所有 AI 的状态。
参数说明:	statusVector: 传出参数, 工具端 AI 容器。
返回值:	成功: 返回 ErrnoSucc。
	失败: 返回错误号。

3 错误码

3.1 接口函数错误码定义

错误号	错误代码	错误信息
0	InterfaceCallSuccCode	成功
10000	ErrCode_Base	
10001	ErrCode_Failed	通用失败
10002	ErrCode_ParamError	参数错误
10003	ErrCode_ConnectSocketFailed	Socket 连接失败
10004	ErrCode_SocketDisconnect	Socket 断开连接
10005	ErrCode_CreateRequestFailed	创建请求失败
10006	ErrCode_RequestRelatedVariableError	请求相关的内部变量出错
10007	ErrCode_RequestTimeout	请求超时
10008	ErrCode_SendRequestFailed	发送请求信息失败
10009	ErrCode_ResponseInfoIsNull	响应信息为空
10010	ErrCode_ResolveResponseFailed	解析响应失败
10011	ErrCode_FkFailed	正解出错
10012	ErrCode_IkFailed	逆解出错
10013	ErrCode_ToolCalibrateError	工具标定参数有错
10014	ErrCode_ToolCalibrateParamError	工具标定参数有错
10015	ErrCode_CoordinateSystemCalibrateError	坐标系标定失败
10016	ErrCode_BaseToUserConvertFailed	基坐标系转用户坐标系失败
10017	ErrCode_UserToBaseConvertFailed	用户坐标系转基坐标系失败
10018	ErrCode_MotionRelatedVariableError	运动相关的内部变量出错
10019	ErrCode_MotionRequestFailed	运动请求失败
10020	ErrCode_CreateMotionRequestFailed	生成运动请求失败
10021	ErrCode_MotionInterruptedByEvent	运动被事件中断
10022	ErrCode_MotionWaypointVetorSizeError	运动相关的路点容器的长度不符合规定
10023	ErrCode_ResponseReturnError	服务器响应返回错误

10024	ErrCode_RealRobotNoExist	真实机械臂不存在，因为有些接口只有在真是机械臂存在的情况下才可以被调用
10025	ErrCode_moveControlSlowStopFailed	调用缓停接口失败
10026	ErrCode_moveControlFastStopFailed	调用急停接口失败
10027	ErrCode_moveControlPauseFailed	调用暂停接口失败
10028	ErrCode_moveControlContinueFailed	调用继续接口失败

3.2 由于控制器异常事件导致的错误码

错误号	错误代码	错误信息
21001	ErrCodeMoveJConfigError	关节运动属性配置错误
21002	ErrCodeMoveLConfigError	直线运动属性配置错误
21003	ErrCodeMovePConfigError	轨迹运动属性配置错误
21004	ErrCodeInvailConfigError	无效的运动属性配置
21005	ErrCodeWaitRobotStopped	等待机器人停止
21006	ErrCodeJointOutOfRange	超出关节运动范围
21007	ErrCodeFirstWaypointSetError	请正确设置MOVEP 第一个路点
21008	ErrCodeConveyorTrackConfigError	传送带跟踪配置错误
21009	ErrCodeConveyorTrackTrajectoryTypeError	传送带轨迹类型错误
21010	ErrCodeRelativeTransformIKFailed	相对坐标变换逆解失败
21011	ErrCodeTeachModeCollision	示教模式发生碰撞
21012	ErrCodeextErnalToolConfigError	运动属性配置错误,外部工具或手持工件配置错误

21101	ErrCodeTrajectoryAbnormal	轨迹异常
21102	ErrCodeOnlineTrajectoryPlanError	轨迹规划错误
21103	ErrCodeOnlineTrajectoryTypeIIError	二型在线轨迹规划失败
21104	ErrCodeIKFailed	逆解失败
21105	ErrCodeAbnormalLimitProtect	动力学限制保护
21106	ErrCodeConveyorTrackingFailed	传送带跟踪失败
21107	ErrCodeConveyorOutWorkingRange	超出传送带工作范围
21108	ErrCodeTrajectoryJointOutOfRange	关节超出范围
21109	ErrCodeTrajectoryJointOverspeed	关节超速
21110	ErrCodeOfflineTrajectoryPlanFailed	离线轨迹规划失败
21111	ErrCodeTrajectoryJointAccOutOfRange	轨迹异常,关节加速度超限
21120	ErrCodeForceModeException	力控模式异常
21121	ErrCodeForceModeIKFailed	轨迹异常, 力控模式下失败
21122	ErrCodeForceModeTrackJointverspeed	关节超速
21200	ErrCodeControllerIKFailed	控制器异常, 逆解失败
21201	ErrCodeControllerStatusException	控制器异常, 状态异常
21202	ErrCodeControllerTrackingLost	关节跟踪误差过大
21203	ErrCodeMonitorErrTrackingLost	关节跟踪误差过大
21204	ErrCodeMonitorErrNoArrivalInTime	预留
21205	ErrCodeMonitorErrCurrentOverload	预留
21206	ErrCodeMonitorErrJointOutOfRange	机械臂关节超出限制范围
21207	ErrCodeFifoDataTimeNotRead	缓存区超时未更新
21300	ErrCodeMoveEnterStopState	运动进入到 stop 阶段
21301	ErrCodeMoveInterruptedByEvent	运动被未知事件中断

3.3 由于硬件层异常事件导致的错误码

错误号	错误代码	错误信息
22001	ErrCodeHardwareErrorNotify	机械臂硬件错误

		不能区分是哪种硬件异常才会返回该错误
22101	ErrCodeJointError	机械臂关节错误
22102	ErrCodeJointOverCurrent	机械臂关节过流
22103	ErrCodeJointOverVoltage	机械臂关节过压
22104	ErrCodeJointLowVoltage	机械臂关节欠压
22105	ErrCodeJointOverTemperature	机械臂关节过温
22106	ErrCodeJointHallError	机械臂关节霍尔错误
22107	ErrCodeJointEncoderError	机械臂关节编码器错误
22108	ErrCodeJointAbsoluteEncoderError	机械臂关节绝对编码器错误
22109	ErrCodeJointCurrentDetectError	机械臂关节当前位置错误
22110	ErrCodeJointEncoderPollution	机械臂关节编码器污染。建议采取措施:警告性通知
22111	ErrCodeJointEncoderZSignalError	机械臂关节编码器 Z 信号错误
22112	ErrCodeJointEncoderCalibrateInvalid	机械臂关节编码器校准失效
22113	ErrCodeJoint_IMU_SensorInvalid	机械臂关节 IMU 传感器失效
22114	ErrCodeJointTemperatureSensorError	机械臂关节温度传感器出错
22115	ErrCodeJointCanBusError	机械臂关节 CAN 总线出错
22116	ErrCodeJointCurrentError	机械臂关节当前电流错误
22117	ErrCodeJointCurrentPositionError	机械臂关节当前位置错误
22118	ErrCodeJointOverSpeed	机械臂关节超速
22119	ErrCodeJointOverAccelerate	机械臂关节加速度过大错误
22120	ErrCodeJointTraceAccuracy	机械臂关节跟踪精度错误
22121	ErrCodeJointTargetPositionOutOfRange	机械臂关节目标位置超范围
22122	ErrCodeJointTargetSpeedOutOfRange	机械臂关节目标速度超范围

22123	ErrCodeJointCollision	建议采取措施:暂停当前运动
22200	ErrCodeDataAbnormal	机械臂信息异常
22201	ErrCodeRobotTypeError	机械臂类型错误
22202	ErrCodeAccelerationSensorError	机械臂加速度计芯片错误
22203	ErrCodeEncoderLineError	机械臂编码器线数错误
22204	ErrCodeEnterDragAndTeachModeError	机械臂进入拖动示教模式错误
22205	ErrCodeExitDragAndTeachModeError	机械臂退出拖动示教模式错误
22206	ErrCodeMACDataInterruptionError	机械臂 MAC 数据中断错误
22207	ErrCodeDriveVersionError	驱动器版本错误 (关节固件版本不一致)
22300	ErrCodeInitAbnormal	机械臂初始化异常
22301	ErrCodeDriverEnableFailed	机械臂驱动器使能失败
22302	ErrCodeDriverEnableAutoBackFailed	机械臂驱动器使能自动回应失败
22303	ErrCodeDriverEnableCurrentLoopFailed	机械臂驱动器使能电流环失败
22304	ErrCodeDriverSetTargetCurrentFailed	机械臂驱动器设置目标电流失败
22305	ErrCodeDriverReleaseBrakeFailed	机械臂释放刹车失败
22306	ErrCodeDriverEnablePostionLoopFailed	机械臂使能位置环失败
22307	ErrCodeSetMaxAccelerateFailed	设置最大加速度失败
22400	ErrCodeSafetyError	机械臂安全出错
22401	ErrCodeExternEmergencyStop	机械臂外部紧急停止
22402	ErrCodeSystemEmergencyStop	机械臂系统紧急停止
22403	ErrCodeTeachpendantEmergencyStop	机械臂示教器紧急停止
22404	ErrCodeControlCabinetEmergencyStop	机械臂控制柜紧急停止
22405	ErrCodeProtectionStopTimeout	机械臂保护停止

		超时
22406	ErrCodeEeducedModeTimeout	机械臂缩减模式 超时
22500	ErrCodeSystemAbnormal	机械臂系统异常
22501	ErrCode_MCU_CommunicationAbnormal	机械臂 mcu 通信 异常
22502	ErrCode485CommunicationAbnormal	机械臂 485 通信 异常
22550	ErrCodeSoftEmergency	软急停
22600	ErrCodeArmPowerOff	控制柜接触器断 开导致机械臂 48V 断电

4 使用案例

4.1 使用 SDK 构建一个最简单的机械臂的控制工程

本案例是使用 SDK 来构建一个最简单的机械臂的控制工程。程序的主要流程是：机械臂登录；初始化；模拟业务；机械臂关机；退出登录。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_0.h 代码如下：

```
#ifndef EXAMPLE_0_H
#define EXAMPLE_0_H

class Example_0
{
public:
    Example_0();

    /**
     * @brief demo
     *
     * 使用 SDK 构建一个最简单的机械臂的控制工程
     */
    static void demo();
};

#endif // EXAMPLE_0_H
```

example_0.cpp 代码如下：

```
#include "example_0.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>

#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

#define SERVER_HOST "127.0.0.1"
#define SERVER_PORT 8899
```

```

Example_0::Example_0()
{
}

void Example_0::demo()
{

    ServiceInterface robotService;

    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化**/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr<<"机械臂初始化成功."<<std::endl;
    }
}

```

```

else
{
    std::cerr<<"机械臂初始化失败."<<std::endl;
}

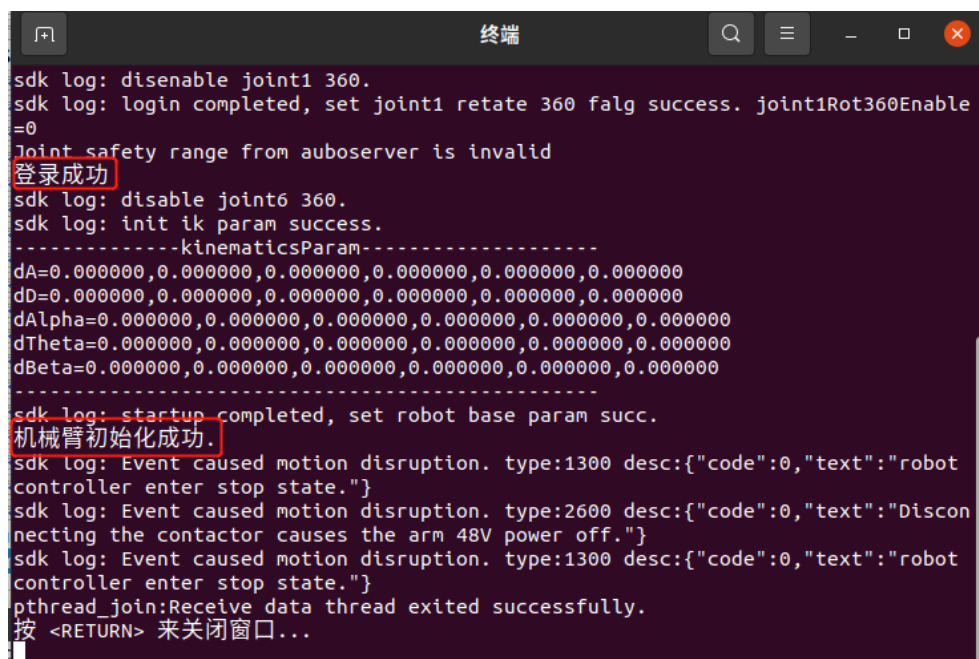
/** 模拟业务 **/
sleep(5);

/** 机械臂 Shutdown **/
robotService.robotServiceRobotShutdown();

/** 接口调用: 退出登录**/
robotService.robotServiceLogout();
}

```

如果通讯成功，则会打印出“登录成功”和“机械臂初始化成功”的语句。



```

sdk log: disable joint1 360.
sdk log: login completed, set joint1 rotate 360 flag success. joint1Rot360Enable=0
Joint safety range from auboserver is invalid
登录成功
sdk log: disable joint6 360.
sdk log: init ik param success.
-----kinematicsParam-----
dA=0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
dB=0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
dAlpha=0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
dTheta=0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
dBeta=0.000000,0.000000,0.000000,0.000000,0.000000,0.000000
-----
sdk log: startup completed, set robot base param succ.
机械臂初始化成功.
sdk log: Event caused motion disruption. type:1300 desc:{"code":0,"text":"robot controller enter stop state."}
sdk log: Event caused motion disruption. type:2600 desc:{"code":0,"text":"Disconnecting the contactor causes the arm 48V power off."}
sdk log: Event caused motion disruption. type:1300 desc:{"code":0,"text":"robot controller enter stop state."}
pthread_join:Receive data thread exited successfully.
按 <RETURN> 来关闭窗口...

```

4.2 用回调函数的方式来获取实时路点、末端速度、机械臂事件、关节状态

本案例是用回调函数的方式来获取实时信息，其中包括了实时路点信息、实时末端速度、实时机械臂事件、实时关节状态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_1.h 代码如下:

```
#ifndef EXAMPLE_1_H
#define EXAMPLE_1_H
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

class Example_1
{
public:
    Example_1();

public:
    static void RealTimeWaypointCallback (const aubo_robot_namespace::wayPoint_S *wayPointPtr, void *arg); //用于获取实时路点回调函数

    static void RealTimeEndSpeedCallback (double speed, void *arg); //获取实时末端速度回调函数

    static void RealTimeEventInfoCallback(const aubo_robot_namespace::RobotEventInfo *pEventInfo, void *arg); //获取实时机械臂事件回调函数

    static void RealTimeJointStatusCallback(const aubo_robot_namespace::JointStatus *jointStatusPtr, int size, void *arg); //获取实时机械臂关节状态

    /**
     * @brief demo
     *
     * 回调函数的方式获取实时路点，末端速度，机械臂的事件
     */
    static void demo();
};

#endif // EXAMPLE_1_H
```

example_1.cpp 代码如下:

注意：代码中使用 Util 类的函数，请参考 4.15 章。

```
#include "example_1.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
```



```
#include <sstream>
#include <fstream>

#include "util.h"

#define SERVER_HOST "127.0.0.1"
#define SERVER_PORT 8899

Example_1::Example_1()
{

}

void Example_1::RealTimeWaypointCallback(const aubo_robot_namespace::wayPoint_S *wayPointPtr, void *arg)
{
    (void)arg;
    aubo_robot_namespace::wayPoint_S waypoint = *wayPointPtr;
    Util::printWaypoint(waypoint);
}

void Example_1::RealTimeEndSpeedCallback(double speed, void *arg)
{
    (void)arg;
    std::cout << "实时末端速度:" << speed << std::endl;
}

void Example_1::RealTimeEventInfoCallback(const aubo_robot_namespace::RobotEventInfo *pEventInfo, void *arg)
{
    (void)arg;
    Util::printEventInfo(*pEventInfo);
}

void Example_1::RealTimeJointStatusCallback(const aubo_robot_namespace::JointStatus *jointStatusPtr, int size, void *arg)
{
    (void)arg;
    Util::printJointStatus(jointStatusPtr, size);
}

void Example_1::demo()
{
```

```

ServiceInterface robotService;

int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "登录成功." << std::endl;
}
else
{
    std::cerr << "登录成功." << std::endl;
}

/** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));

ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位姿 默认为 true*/,
                                           true,       /*保留默认为 true */
                                           1000,      /*保留默认为 1000 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//获取实时路点信息

```

```

        robotService.robotServiceRegisterRealTimeRoadPointCallback(Example_1::RealTimeWaypointCallback, NULL);

        //获取实时末端速度信息
        robotService.robotServiceRegisterRealTimeEndSpeedCallback(Example_1::RealTimeEndSpeedCallback, NULL);

        //获取实时事件信息
        robotService.robotServiceRegisterRobotEventInfoCallback(Example_1::RealTimeEventInfoCallback, NULL);

        //获取实时关节状态信息
        robotService.robotServiceRegisterRealTimeJointStatusCallback(Example_1::RealTimeJointStatusCallback, NULL);

        sleep(10);

    }

```

4.3 正逆解

本案例是用 SDK 来实现机器人运动学正解与逆解的功能。

该程序的主要流程是：(1) 机械臂登录 (2) 初始化 (3) 获取当前的路点信息 (4) 根据当前路点的关节角, 正解得到目标位置和姿态 (5) 根据正解得到的位置姿态, 调用逆解函数来获得逆解集和最优逆解。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_2.h 代码如下：

```

#ifndef EXAMPLE_2_H
#define EXAMPLE_2_H

class Example_2
{
public:
    Example_2();

    /**
     * @brief demo
     *
     * 正逆解
     */

```

```

public:
    static void demo();
};

#endif // EXAMPLE_2_H

```

example_2.cpp 代码如下:

```

#include "example_2.h"
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>
#include <vector>

using namespace std;

#define SERVER_HOST "192.168.221.13"
#define SERVER_PORT 8899

Example_2::Example_2()
{
}

void Example_2::demo()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }
}

```

```

    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //获取机械臂当前的路点信息
    aubo_robot_namespace::wayPoint_S currentWaypoint;
    robotService.robotServiceGetCurrentWaypointInfo(currentWaypoint);

    //根据当前路点的关节角，正解得到目标路点
    aubo_robot_namespace::wayPoint_S targetWaypoint;
    ret = robotService.robotServiceRobotFk(currentWaypoint.jointpos, aubo_robot_
namespace::ARM_DOF, targetWaypoint);
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "-----正解-----" << std::endl;
        std::cout << "正解得到的路点位置： "
            << " x = " << targetWaypoint.cartPos.position.x
            << " y = " << targetWaypoint.cartPos.position.y
            << " z = " << targetWaypoint.cartPos.position.z
            << std::endl;
        std::cout << "正解得到的目标路点的姿态（四元数）： "

```

```

        << " w = " << targetWaypoint.orientation.w
        << " x = " << targetWaypoint.orientation.x
        << " y = " << targetWaypoint.orientation.y
        << " z = " << targetWaypoint.orientation.z
        << std::endl;
//四元数转欧拉角
aubo_robot_namespace::Rpy rpy;
robotService.quaternionToRPY(targetWaypoint.orientation, rpy);
std::cout << "正解得到的目标路点的姿态（欧拉角）:"
        << " RX = " << rpy.rx*180/M_PI
        << " RY = " << rpy.ry*180/M_PI
        << " RZ = " << rpy.rz*180/M_PI
        << std::endl;
    }
    else
    {
        std::cerr << "调用正解函数失败" << std::endl;
    }

//根据正解得到的位置姿态，来获取逆解集
aubo_robot_namespace::Pos position = targetWaypoint.cartPos.position; //正解
得到的路点的位置
aubo_robot_namespace::Ori orientation = targetWaypoint.orientation; //正解得到
的路点的姿态
std::vector<aubo_robot_namespace::wayPoint_S> wayPointVector;
ret = robotService.robotServiceRobotIk(position, orientation, wayPointVector);

std::cout << std::endl;
std::cout << "-----逆解集-----" << std::endl;
std::cout << "逆解集的大小: " << wayPointVector.size() << std::endl;
for(int i = 0; i < wayPointVector.size(); i++)
{
    std::cout << "第" << i+1 << "组解: " << std::endl;
    for(int j = 0; j < 6; j++)
    {
        std::cout << "关节" << j+ 1 << ": " << wayPointVector[i].jointpos
[j]*180/M_PI << std::endl;
    }
}

//根据当前路点，获取最优逆解
aubo_robot_namespace::wayPoint_S wayPoint;

```

```

robotService.robotServiceGetCurrentWaypointInfo(currentWaypoint);
ret = robotService.robotServiceRobotIk(currentWaypoint.jointpos, position, orientation, wayPoint);
std::cout << std::endl;
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "-----最优逆解-----" << std::endl;
    for(int i = 0; i < 6; i++)
    {
        std::cout << "关节" << i+ 1 << ": " << wayPoint.jointpos[i]*180/M_PI << std::endl;
    }
}
else
{
    std::cerr << "调用逆解函数失败" << std::endl;
}
}

```

4.4 坐标系转换

4.4.1 基坐标系转用户坐标系 baseToUserCoordinate()

baseToUserCoordinate 函数示例 1

本示例是将法兰盘中心在基坐标系下的位置和姿态转成工具在用户坐标系下的位置和姿态。

本程序的主要流程是：(1) 机械臂登录 (2) 初始化 (3) 设置法兰盘中心在基坐标系下的位置和姿态 (3) 设置用户坐标系及坐标系的工具参数 (4) 设置工具参数 (5) 调用 baseToUserCoordinate 函数来获得工具在用户坐标系下的位置和姿态

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 baseToUserCoordinate()代码如下：

```

void Example_3::baseToUserCoordinate1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub

```

```

o", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位
姿 默认为 true*/,
                                                true,        /*保留默认为 true
*/
                                                1000,        /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系
下的位置
    flangeCenterPosOnBase.x = 0.247248;
    flangeCenterPosOnBase.y = 0.280197;
    flangeCenterPosOnBase.z = 0.322796;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
    flangeCenterRpyOnBase.rx = -101.335228*M_PI/180;

```



```

flangeCenterRpyOnBase.ry = 8.768851*M_PI/180;
flangeCenterRpyOnBase.rz = 93.718178*M_PI/180;

aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下的姿态（四元数）
robotService.RPYToQuaternion(flangeCenterRpyOnBase, flangeCenterOriOnBase);

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;//用户坐标系
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = 0;
toolUserCoord.toolInEndPosition.y = 0;
toolUserCoord.toolInEndPosition.z = 0.45;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

```

```

aubo_robot_namespace::ToolInEndDesc toolInEndDesc;
toolInEndDesc.toolInEndPosition.x = 0;
toolInEndDesc.toolInEndPosition.y = 0;
toolInEndDesc.toolInEndPosition.z = 0.45;
toolInEndDesc.toolInEndOrientation.w = 1;
toolInEndDesc.toolInEndOrientation.x = 0;
toolInEndDesc.toolInEndOrientation.y = 0;
toolInEndDesc.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos toolEndPosOnUser;//工具末端在用户坐标系下的位置
aubo_robot_namespace::Ori toolEndOriOnUser;//工具末端在用户坐标系下的姿态

//将法兰盘中心在基坐标系下的位置姿态转化为工具末端在用户坐标系下的位置和姿态
robotService.baseToUserCoordinate(flangeCenterPosOnBase, flangeCenterOriOnBase, userCoord, toolInEndDesc, toolEndPosOnUser, toolEndOriOnUser);

std::cout << "工具末端在用户坐标系下的位置: ";
std::cout << "(" << toolEndPosOnUser.x << ", " << toolEndPosOnUser.y << ", " << toolEndPosOnUser.z << ")";
std::cout << std::endl;

std::cout << "工具末端在用户坐标系下的姿态（四元数）: ";
std::cout << "(" << toolEndOriOnUser.w << ", " << toolEndOriOnUser.x << ", " << toolEndOriOnUser.y << ", " << toolEndOriOnUser.z << ")";
std::cout << std::endl;

aubo_robot_namespace::Rpy toolEndRpyOnUser;
robotService.quaternionToRPY(toolEndOriOnUser, toolEndRpyOnUser);
std::cout << "工具末端在用户坐标系下的姿态（欧拉角）: ";
std::cout << "(" << toolEndRpyOnUser.rx*180/M_PI << ", " << toolEndRpyOnUser.ry*180/M_PI << ", " << toolEndRpyOnUser.rz*180/M_PI << ")";
std::cout << std::endl;
}

```

baseToUserCoordinate 函数示例 2

本示例是将法兰盘中心在基坐标系下的位置和姿态转成法兰盘中心在用户坐标系下的位置和姿态。法兰盘中心可看成是一个位置(0,0,0)姿态(1,0,0,0)的特殊的工具。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置法兰盘中心在基坐标

系下的位置和姿态 (3) 设置用户坐标系及坐标系的工具参数 (4) 设置工具参数为法兰盘中心 (5) 调用 baseToUserCoordinate 函数来获得法兰盘中心在用户坐标系下的位置和姿态。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 baseToUserCoordinate2()代码如下：

```
void Example_3::baseToUserCoordinate2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
}
```

```

else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系
下的位置
flangeCenterPosOnBase.x = 0.247248;
flangeCenterPosOnBase.y = 0.280197;
flangeCenterPosOnBase.z = 0.322796;

aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
flangeCenterRpyOnBase.rx = -101.335228*M_PI/180;
flangeCenterRpyOnBase.ry = 8.768851*M_PI/180;
flangeCenterRpyOnBase.rz = 93.718178*M_PI/180;

aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下
的姿态（四元数）
robotService.RPYToQuaternion(flangeCenterRpyOnBase, flangeCenterOriOnBas
e);

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;//用户
坐标系
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;

```

```

userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = 0;
toolUserCoord.toolInEndPosition.y = 0;
toolUserCoord.toolInEndPosition.z = 0.45;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

aubo_robot_namespace::ToolInEndDesc toolInEndDesc;
toolInEndDesc.toolInEndPosition.x = 0;
toolInEndDesc.toolInEndPosition.y = 0;
toolInEndDesc.toolInEndPosition.z = 0;
toolInEndDesc.toolInEndOrientation.w = 1;
toolInEndDesc.toolInEndOrientation.x = 0;
toolInEndDesc.toolInEndOrientation.y = 0;
toolInEndDesc.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos flangeCenterPosOnUser;
aubo_robot_namespace::Ori flangeCenterOriOnUser;

//将法兰盘中心在基坐标系下的位置和姿态转成法兰盘中心在用户坐标系下的位置和姿态
robotService.baseToUserCoordinate(flangeCenterPosOnBase, flangeCenterOriOnBase, userCoord, toolInEndDesc, flangeCenterPosOnUser, flangeCenterOriOnUser);

std::cout << "法兰盘中心在用户坐标系下的位置: ";
std::cout << "(" << flangeCenterPosOnUser.x << ", " << flangeCenterPosOnUser.y << ", " << flangeCenterPosOnUser.z << ")";
std::cout << std::endl;

std::cout << "法兰盘中心在用户坐标系下的姿态（四元数）: ";
std::cout << "(" << flangeCenterOriOnUser.w << ", " << flangeCenterOriOnUser.x << ", " << flangeCenterOriOnUser.y << ", " << flangeCenterOriOnUser.z << ")";
std::cout << std::endl;

aubo_robot_namespace::Rpy flangeCenterRpyOnUser;

```

```

    robotService.quaternionToRPY(flangeCenterOriOnUser, flangeCenterRpyOnUser);
    std::cout << "法兰盘中心在用户坐标系下的姿态（欧拉角）：";
    std::cout << "(" << flangeCenterRpyOnUser.rx*180/M_PI << ", " << flangeCenterRpyOnUser.ry*180/M_PI << ", " << flangeCenterRpyOnUser.rz*180/M_PI <<
    ")";
    std::cout << std::endl;
}

```

baseToUserCoordinate 函数示例 3

本示例是将法兰盘中心在基坐标系下的位置和姿态转成工具在基坐标系下的位置和姿态。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置法兰盘中心在基坐标系下的位置和姿态（3）设置基坐标系（4）设置工具参数（5）调用 baseToUserCoordinate 函数来获得工具在基坐标系下的位置和姿态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 baseToUserCoordinate3()代码如下：

```

void Example_3::baseToUserCoordinate3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
}

```

```

    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系
下的位置
    flangeCenterPosOnBase.x = -0.374741;
    flangeCenterPosOnBase.y = -0.534099;
    flangeCenterPosOnBase.z = 0.198738;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
    flangeCenterRpyOnBase.rx = 179.956696*M_PI/180;
    flangeCenterRpyOnBase.ry = -5.516838*M_PI/180*M_PI/180;
    flangeCenterRpyOnBase.rz = -91.337303*M_PI/180;

    aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下
的姿态（四元数）
    robotService.RPYToQuaternion(flangeCenterRpyOnBase, flangeCenterOriOnBas
e); //欧拉角转四元数

    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;//基坐
标系
    baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    aubo_robot_namespace::ToolInEndDesc toolInEndDesc;//工具参数
    toolInEndDesc.toolInEndPosition.x = 0;
    toolInEndDesc.toolInEndPosition.y = 0;

```

```

    toolInEndDesc.toolInEndPosition.z = 0.45;
    toolInEndDesc.toolInEndOrientation.w = 1;
    toolInEndDesc.toolInEndOrientation.x = 0;
    toolInEndDesc.toolInEndOrientation.y = 0;
    toolInEndDesc.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos toolEndPosOnBase;//工具末端在基坐标系下的位置
    aubo_robot_namespace::Ori toolEndOriOnBase;//工具末端在基坐标系下的姿态

    //将法兰盘中心在基坐标系下的位置和姿态转成工具在基坐标系下的位置和姿态
    robotService.baseToUserCoordinate(flangeCenterPosOnBase, flangeCenterOriOnBase, baseCoord, toolInEndDesc, toolEndPosOnBase, toolEndOriOnBase);

    std::cout << "工具末端在基坐标系下的位置: ";
    std::cout << "(" << toolEndPosOnBase.x << ", " << toolEndPosOnBase.y << ", " << toolEndPosOnBase.z << ")";
    std::cout << std::endl;

    std::cout << "工具末端在基坐标系下的姿态（四元数）: ";
    std::cout << "(" << toolEndOriOnBase.w << ", " << toolEndOriOnBase.x << ", " << toolEndOriOnBase.y << ", " << toolEndOriOnBase.z << ")";
    std::cout << std::endl;

    aubo_robot_namespace::Rpy toolEndRpyOnBase;
    robotService.quaternionToRPY(toolEndOriOnBase, toolEndRpyOnBase);
    std::cout << "工具末端在基坐标系下的姿态（欧拉角）: ";
    std::cout << "(" << toolEndRpyOnBase.rx*180/M_PI << ", " << toolEndRpyOnBase.ry*180/M_PI << ", " << toolEndRpyOnBase.rz*180/M_PI << ")";
    std::cout << std::endl;
}

```

4.4.2 基坐标系转基坐标得到工具末端点的位置姿态 baseToBaseAdditionalTool()

baseToBaseAdditionalTool 函数示例

本示例是将法兰盘中心在基坐标系下的位置和姿态转成工具在基坐标系下的位置和姿态。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置法兰盘中心在基坐标系下的位置和姿态（4）设置工具参数（5）调用 baseToBaseAdditionalTool 函数来

获得工具在基坐标系下的位置和姿态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 baseToBaseAdditionalTool1()代码如下：

```
void Example_3::baseToBaseAdditionalTool1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位姿 默认为 true*/,
                                                true,      /*保留默认为 true */
                                                1000,     /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {

```

```

        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系
下的位置
    flangeCenterPosOnBase.x = 0.247248;
    flangeCenterPosOnBase.y = 0.280197;
    flangeCenterPosOnBase.z = 0.322796;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
    flangeCenterRpyOnBase.rx = -101.335228*M_PI/180;
    flangeCenterRpyOnBase.ry = 8.768851*M_PI/180;
    flangeCenterRpyOnBase.rz = 93.718178*M_PI/180;

    aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下
的姿态（四元数）
    robotService.RPYToQuaternion(flangeCenterRpyOnBase, flangeCenterOriOnBas
e);

    aubo_robot_namespace::ToolInEndDesc toolInEndDesc;//工具参数
    toolInEndDesc.toolInEndPosition.x = 0;
    toolInEndDesc.toolInEndPosition.y = 0;
    toolInEndDesc.toolInEndPosition.z = 0.45;
    toolInEndDesc.toolInEndOrientation.w = 1;
    toolInEndDesc.toolInEndOrientation.x = 0;
    toolInEndDesc.toolInEndOrientation.y = 0;
    toolInEndDesc.toolInEndOrientation.z = 0;

    aubo_robot_namespace::Pos toolEndPosOnBase;//工具末端在基坐标系下的位
置
    aubo_robot_namespace::Ori toolEndOriOnBase;//工具末端在基坐标系下的姿态

    //将法兰盘中心在基坐标系下的位置和姿态转成工具在基坐标系下的位置和
姿态
    robotService.baseToBaseAdditionalTool(flangeCenterPosOnBase, flangeCenterOriOnBase, toolInEndDesc, toolEndPosOnBase, toolEndOriOnBase);

    std::cout << "工具末端在基坐标系下的位置: ";
    std::cout << "(" << toolEndPosOnBase.x << ", " << toolEndPosOnBase.y <<
", " << toolEndPosOnBase.z << ")";
    std::cout << std::endl;

    std::cout << "工具末端在基坐标系下的姿态（四元数）: ";

```

```

        std::cout << "(" << toolEndOriOnBase.w << ", " << toolEndOriOnBase.x <<
        ", " << toolEndOriOnBase.y << ", " << toolEndOriOnBase.z << ")";
        std::cout << std::endl;

        aubo_robot_namespace::Rpy toolEndRpyOnBase;
        robotService.quaternionToRPY(toolEndOriOnBase, toolEndRpyOnBase);
        std::cout << "工具末端在基坐标系下的姿态（欧拉角）：";
        std::cout << "(" << toolEndRpyOnBase.rx*180/M_PI << ", " << toolEndRpy
        OnBase.ry*180/M_PI << ", " << toolEndRpyOnBase.rz*180/M_PI << ")";
        std::cout << std::endl;
    }

```

4.4.3 用户坐标系转基坐标系 userToBaseCoordinate()

userToBaseCoordinate 函数示例 1

本示例是将工具末端在用户坐标系下的位置和姿态转成法兰盘中心在基坐标系下的位置和姿态。

本程序的主要流程是：(1) 机械臂登录 (2) 初始化 (3) 设置工具末端在用户坐标系下的位置和姿态 (4) 设置用户坐标系及坐标系的工具参数 (5) 设置工具参数 (6) 调用 userToBaseCoordinate 函数来获得法兰盘中心在基坐标系下的位置和姿态

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 userToBaseCoordinate1()代码如下：

```

void Example_3::userToBaseCoordinate1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }
}

```

```

/** 如果是连接真实机械臂，需要对机械臂进行初始化 */
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true        /*是否允许读取位
姿 默认为 true*/,
                                true,      /*保留默认为 true
*/
                                1000,      /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

aubo_robot_namespace::Pos toolEndPosOnUser;//工具末端在用户坐标系下的
位置
toolEndPosOnUser.x = 0.108191;
toolEndPosOnUser.y = -0.319869;
toolEndPosOnUser.z = 0.595867;

aubo_robot_namespace::Ori toolEndOriOnUser;//工具末端在用户坐标系下的姿
态（四元数）
aubo_robot_namespace::Rpy toolEndRpyOnUser;//工具末端在用户坐标系下的
姿态（欧拉角）
toolEndRpyOnUser.rx = -101.335*M_PI/180;
toolEndRpyOnUser.ry = 8.76846*M_PI/180;
toolEndRpyOnUser.rz = 93.7183*M_PI/180;
robotService.RPYToQuaternion(toolEndRpyOnUser,toolEndOriOnUser);

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;//用户
坐标系
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis

```

```
_AnyPointOnFirstQuadrantOfXOYPlane;
```

```
userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;
```

```
userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;
```

```
userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;
```

```
aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = 0;
toolUserCoord.toolInEndPosition.y = 0;
toolUserCoord.toolInEndPosition.z = 0.45;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;
```

```
aubo_robot_namespace::ToolInEndDesc toolInEndDesc;
toolInEndDesc.toolInEndPosition.x = 0;
toolInEndDesc.toolInEndPosition.y = 0;
toolInEndDesc.toolInEndPosition.z = 0.45;
toolInEndDesc.toolInEndOrientation.w = 1;
toolInEndDesc.toolInEndOrientation.x = 0;
toolInEndDesc.toolInEndOrientation.y = 0;
toolInEndDesc.toolInEndOrientation.z = 0;
```

```
aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系下的位置
```

```

    aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下的
    姿态（四元数）

    //将工具末端在用户坐标系下的位置和姿态转成法兰盘中心在基坐标系下的
    位置和姿态
    robotService.userToBaseCoordinate(toolEndPosOnUser, toolEndOriOnUser, user
    Coord, toolInEndDesc, flangeCenterPosOnBase, flangeCenterOriOnBase);

    std::cout << "法兰盘中心在基坐标系下的位置: ";
    std::cout << "(" << flangeCenterPosOnBase.x << ", " << flangeCenterPosOn
    Base.y << ", " << flangeCenterPosOnBase.z << ")";
    std::cout << std::endl;

    std::cout << "法兰盘中心在基坐标系下的姿态（四元数）: ";
    std::cout << "(" << flangeCenterOriOnBase.w << ", " << flangeCenterOriOn
    Base.x << ", " << flangeCenterOriOnBase.y << ", " << flangeCenterOriOnBase.z
    << ")";
    std::cout << std::endl;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
    下的姿态（欧拉角）
    robotService.quaternionToRPY(flangeCenterOriOnBase, flangeCenterRpyOnBas
    e);
    std::cout << "法兰盘中心在基坐标系下的姿态（欧拉角）: ";
    std::cout << "(" << flangeCenterRpyOnBase.rx*180/M_PI << ", " << flange
    CenterRpyOnBase.ry*180/M_PI << ", " << flangeCenterRpyOnBase.rz*180/M_PI
    << ")";
    std::cout << std::endl;

}

```

userToBaseCoordinate 函数示例 2

本示例是将法兰盘中心在用户坐标系下的位置和姿态转成法兰盘中心在基坐标系下的位置和姿态。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置法兰盘中心在用户坐标系下的位置和姿态（4）设置用户坐标系及其工具参数（5）设置工具参数，为法兰盘中心（6）调用 userToBaseCoordinate 函数来获得法兰盘中心在基坐标系下的位置和姿态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 userToBaseCoordinate2()代码如下：

```

void Example_3::userToBaseCoordinate2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::Pos flangeCenterPosOnUser;//法兰盘中心在用户坐标系下的位置

```

```

    flangeCenterPosOnUser.x = 0.547609;
    flangeCenterPosOnUser.y = -0.2778;
    flangeCenterPosOnUser.z = 0.683283;

    aubo_robot_namespace::Ori flangeCenterOriOnUser;//法兰盘中心在用户坐标系
下的姿态（四元数）
    aubo_robot_namespace::Rpy flangeCenterRpyOnUser;//法兰盘中心在用户坐标
系下的姿态（欧拉角）
    flangeCenterRpyOnUser.rx = -101.335*M_PI/180;
    flangeCenterRpyOnUser.ry = 8.76846*M_PI/180;
    flangeCenterRpyOnUser.rz = 93.7183*M_PI/180;
    robotService.RPYToQuaternion(flangeCenterRpyOnUser,flangeCenterOriOnUse
r);

    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;//用户
坐标系
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

    userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc toolUserCoord;
    toolUserCoord.toolInEndPosition.x = 0;
    toolUserCoord.toolInEndPosition.y = 0;

```



```

toolUserCoord.toolInEndPosition.z = 0.45;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

aubo_robot_namespace::ToolInEndDesc toolInEndDesc;
toolInEndDesc.toolInEndPosition.x = 0;
toolInEndDesc.toolInEndPosition.y = 0;
toolInEndDesc.toolInEndPosition.z = 0;
toolInEndDesc.toolInEndOrientation.w = 1;
toolInEndDesc.toolInEndOrientation.x = 0;
toolInEndDesc.toolInEndOrientation.y = 0;
toolInEndDesc.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系
下的位置
aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下
的姿态（四元数）

//将法兰盘中心在用户坐标系下的位置和姿态转成法兰盘中心在基坐标系下
的位置和姿态
robotService.userToBaseCoordinate(flangeCenterPosOnUser, flangeCenterOriOn
User, userCoord, toolInEndDesc, flangeCenterPosOnBase, flangeCenterOriOnBase);

std::cout << "法兰盘中心在基坐标系下的位置: ";
std::cout << "(" << flangeCenterPosOnBase.x << ", " << flangeCenterPosOn
Base.y << ", " << flangeCenterPosOnBase.z << ")";
std::cout << std::endl;

std::cout << "法兰盘中心在基坐标系下的姿态（四元数）: ";
std::cout << "(" << flangeCenterOriOnBase.w << ", " << flangeCenterOriOn
Base.x << ", " << flangeCenterOriOnBase.y << ", " << flangeCenterOriOnBase.z
<< ")";
std::cout << std::endl;

aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
robotService.quaternionToRPY(flangeCenterOriOnBase, flangeCenterRpyOnBas
e);
std::cout << "法兰盘中心在基坐标系下的姿态（欧拉角）: ";
std::cout << "(" << flangeCenterRpyOnBase.rx*180/M_PI << ", " << flange
CenterRpyOnBase.ry*180/M_PI << ", " << flangeCenterRpyOnBase.rz*180/M_PI

```

```

<< ")\n";
    std::cout << std::endl;

}

```

userToBaseCoordinate 函数示例 3

本示例是将工具末端在基坐标系下的位置和姿态转成法兰盘中心在基坐标系下的位置和姿态。

本程序的主要流程是：(1) 机械臂登录 (2) 初始化 (3) 设置工具末端在基坐标系下的位置和姿态 (4) 设置基坐标系 (5) 设置工具参数 (6) 调用 userToBaseCoordinate 函数来获得法兰盘中心在基坐标系下的位置和姿态。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 userToBaseCoordinate3()代码如下：

```

void Example_3::userToBaseCoordinate3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,

                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位

```

```

姿 默认为 true*/,
                                true,    /*保留默认为 true
*/
                                1000,    /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

aubo_robot_namespace::Pos toolEndPosOnBase;//工具末端在基坐标系下的位置
toolEndPosOnBase.x = -0.375099;
toolEndPosOnBase.y = -0.534847;
toolEndPosOnBase.z = -0.251261;

aubo_robot_namespace::Ori toolEndOriOnBase;//工具末端在基坐标系下的姿态（四元数）
aubo_robot_namespace::Rpy toolEndRpyOnBase;//工具末端在基坐标系下的姿态（欧拉角）
toolEndRpyOnBase.rx = 179.957*M_PI/180;
toolEndRpyOnBase.ry = -0.096287*M_PI/180;
toolEndRpyOnBase.rz = -91.3373*M_PI/180;
robotService.RPYToQuaternion(toolEndRpyOnBase, toolEndOriOnBase);

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

aubo_robot_namespace::ToolInEndDesc toolInEndDesc;
toolInEndDesc.toolInEndPosition.x = 0;
toolInEndDesc.toolInEndPosition.y = 0;
toolInEndDesc.toolInEndPosition.z = 0.45;
toolInEndDesc.toolInEndOrientation.w = 1;
toolInEndDesc.toolInEndOrientation.x = 0;
toolInEndDesc.toolInEndOrientation.y = 0;
toolInEndDesc.toolInEndOrientation.z = 0;

aubo_robot_namespace::Pos flangeCenterPosOnBase;//法兰盘中心在基坐标系下的位置

```

```

    aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下的
    姿态（四元数）

    //将工具末端在基坐标系下的位置和姿态转成法兰盘中心在基坐标系下的位
    置和姿态
    robotService.userToBaseCoordinate(toolEndPosOnBase, toolEndOriOnBase, bas
    eCoord, toolInEndDesc, flangeCenterPosOnBase, flangeCenterOriOnBase);

    std::cout << "法兰盘中心在基坐标系下的位置: ";
    std::cout << "(" << flangeCenterPosOnBase.x << ", " << flangeCenterPosOn
    Base.y << ", " << flangeCenterPosOnBase.z << ")";
    std::cout << std::endl;

    std::cout << "法兰盘中心在基坐标系下的姿态（四元数）: ";
    std::cout << "(" << flangeCenterOriOnBase.w << ", " << flangeCenterOriOn
    Base.x << ", " << flangeCenterOriOnBase.y << ", " << flangeCenterOriOnBase.z
    << ")";
    std::cout << std::endl;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
    下的姿态（欧拉角）
    robotService.quaternionToRPY(flangeCenterOriOnBase, flangeCenterRpyOnBas
    e);
    std::cout << "法兰盘中心在基坐标系下的姿态（欧拉角）: ";
    std::cout << "(" << flangeCenterRpyOnBase.rx*180/M_PI << ", " << flange
    CenterRpyOnBase.ry*180/M_PI << ", " << flangeCenterRpyOnBase.rz*180/M_PI
    << ")";
    std::cout << std::endl;

}

```

4.4.4 用户坐标系下位置参数转基坐标系 userCoordPointToBasePo int()

userCoordPointToBasePoint 函数示例

本示例是将工具末端在用户坐标系下的位置转成工具末端在基坐标系下的位置。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置工具末端在用户坐标系下的位置（4）设置用户坐标系及其工具参数（5）调用 userCoordPointToBasePoint 函数来获得工具末端在基坐标系下的位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 userCoordPointToBasePoint1()代码如下：

```
void Example_3::userCoordPointToBasePoint1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,        /*保留默认为 true */
                                                1000,        /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }
}
```

```

    }

    aubo_robot_namespace::Pos toolEndPosOnUser;//工具末端在用户坐标系下的
位置
    toolEndPosOnUser.x = -0.074733;
    toolEndPosOnUser.y = -1.092842;
    toolEndPosOnUser.z = 0.109217;

    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;//用户
坐标系
    userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
    userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

    userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
    userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
    userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
    userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
    userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
    userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

    userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
    userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
    userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
    userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
    userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
    userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

    userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
    userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
    userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
    userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
    userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
    userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

    aubo_robot_namespace::ToolInEndDesc toolUserCoord;
    toolUserCoord.toolInEndPosition.x = 0;
    toolUserCoord.toolInEndPosition.y = 0;
    toolUserCoord.toolInEndPosition.z = 0.45;
    toolUserCoord.toolInEndOrientation.w = 1;
    toolUserCoord.toolInEndOrientation.x = 0;
    toolUserCoord.toolInEndOrientation.y = 0;
    toolUserCoord.toolInEndOrientation.z = 0;
    userCoord.toolDesc = toolUserCoord;

```

```

aubo_robot_namespace::Pos toolEndPosOnBase;//工具末端在基坐标系下的位置
//将法兰盘中心在基坐标系下的姿态转成工具末端在基坐标系下的姿态
robotService.userCoordPointToBasePoint(toolEndPosOnUser, userCoord, toolEndPosOnBase);

std::cout << "工具末端在基坐标系下的位置: ";
std::cout << "(" << toolEndPosOnBase.x << ", " << toolEndPosOnBase.y << ", " << toolEndPosOnBase.z << ")";
std::cout << std::endl;
}

```

4.4.5 法兰盘姿态转工具姿态 endOrientation2ToolOrientation()

endOrientation2ToolOrientation 函数示例

本示例是将法兰盘中心在基坐标系下的姿态转成工具末端在基坐标系下的姿态。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置法兰盘中心在基坐标系下的姿态（4）设置工具姿态参数（5）调用 endOrientation2ToolOrientation 函数来获得工具末端在基坐标系下的姿态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 endOrientation2ToolOrientation1()代码如下：

```

void Example_3::endOrientation2ToolOrientation1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
}

```

```

aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true       /*是否允许读取位
姿 默认为 true*/,
                                true,     /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下的
的姿态（四元数）
aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系
下的姿态（欧拉角）
flangeCenterRpyOnBase.rx = -176.317383*M_PI/180;
flangeCenterRpyOnBase.ry = 19.773378*M_PI/180;
flangeCenterRpyOnBase.rz = -169.362442*M_PI/180;
robotService.RPYToQuaternion(flangeCenterRpyOnBase, flangeCenterOriOnBas
e);

aubo_robot_namespace::Ori toolOriParam;//工具姿态参数（四元数）
aubo_robot_namespace::Rpy toolRpyParam;//工具姿态参数（欧拉角）
toolRpyParam.rx = -162.014703*M_PI/180;
toolRpyParam.ry = -25.569674*M_PI/180;
toolRpyParam.rz = -36.962539*M_PI/180;
robotService.RPYToQuaternion(toolRpyParam, toolOriParam);

aubo_robot_namespace::Ori toolEndOriOnBase;//工具末端在基坐标系下的姿态
（四元数）
//将法兰盘中心在基坐标系下的姿态转成工具末端在基坐标系下的姿态

```



```

    robotService.endOrientation2ToolOrientation(toolOriParam, flangeCenterOriOnBase, toolEndOriOnBase);

    std::cout << "工具末端在基坐标系下的姿态（四元数）：";
    std::cout << "(" << toolEndOriOnBase.w << ", " << toolEndOriOnBase.x <<
    ", " << toolEndOriOnBase.y << ", " << toolEndOriOnBase.z << ")";
    std::cout << std::endl;

    aubo_robot_namespace::Rpy toolEndRpyOnBase;//工具末端在基坐标系下的姿态（欧拉角）
    robotService.quaternionToRPY(toolEndOriOnBase, toolEndRpyOnBase);
    std::cout << "工具末端在基坐标系下的姿态（欧拉角）：";
    std::cout << "(" << toolEndRpyOnBase.rx*180/M_PI << ", " << toolEndRpyOnBase.ry*180/M_PI << ", " << toolEndRpyOnBase.rz*180/M_PI << ")";
    std::cout << std::endl;
}

```

4.4.6 工具姿态转法兰盘姿态 toolOrientation2EndOrientation()

toolOrientation2EndOrientation 函数示例

本示例是将工具末端在基坐标系下的姿态转成法兰盘中心在基坐标系下的姿态。

本程序的主要流程是：（1）机械臂登录（2）初始化（3）设置工具末端在基坐标系下的姿态（4）设置工具姿态参数（5）调用 toolOrientation2EndOrientation 函数来获得法兰盘中心在基坐标系下的姿态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_3.cpp 中的函数 toolOrientation2EndOrientation1()代码如下：

```

void Example_3::toolOrientation2EndOrientation1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {

```

```

        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 */
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,      /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/

    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::Ori toolEndOriOnBase;//工具末端在基坐标系下的姿态
（四元数）
    aubo_robot_namespace::Rpy toolEndRpyOnBase;//工具末端在基坐标系下的姿
态（欧拉角）
    toolEndRpyOnBase.rx = 36.627361*M_PI/180;
    toolEndRpyOnBase.ry = 38.051857*M_PI/180;
    toolEndRpyOnBase.rz = -123.093803*M_PI/180;
    robotService.RPYToQuaternion(toolEndRpyOnBase, toolEndOriOnBase);

    aubo_robot_namespace::Ori toolOriParam;//工具姿态参数（四元数）
    aubo_robot_namespace::Rpy toolRpyParam;//工具姿态参数（欧拉角）
    toolRpyParam.rx = -162.014703*M_PI/180;
    toolRpyParam.ry = -25.569674*M_PI/180;
    toolRpyParam.rz = -36.962539*M_PI/180;
    robotService.RPYToQuaternion(toolRpyParam, toolOriParam);

```

```

    aubo_robot_namespace::Ori flangeCenterOriOnBase;//法兰盘中心在基坐标系下的姿态（四元数）
    //将工具末端在基坐标系下的姿态转成法兰盘中心在基坐标系下的姿态
    robotService.toolOrientation2EndOrientation(toolOriParam, toolEndOriOnBase, flangeCenterOriOnBase);
    std::cout << "法兰盘中心在基坐标系下的姿态（四元数）：";
    std::cout << "(" << flangeCenterOriOnBase.w << ", " << flangeCenterOriOnBase.x << ", " << flangeCenterOriOnBase.y << ", " << flangeCenterOriOnBase.z << ")";
    std::cout << std::endl;

    aubo_robot_namespace::Rpy flangeCenterRpyOnBase;//法兰盘中心在基坐标系下的姿态（欧拉角）
    robotService.quaternionToRPY(flangeCenterOriOnBase, flangeCenterRpyOnBase);
    std::cout << "法兰盘中心在基坐标系下的姿态（欧拉角）：";
    std::cout << "(" << flangeCenterRpyOnBase.rx*180/M_PI << ", " << flangeCenterRpyOnBase.ry*180/M_PI << ", " << flangeCenterRpyOnBase.rz*180/M_PI << ")";
    std::cout << std::endl;
}

```

4.5 设置和获取机械臂相关参数

本示例是设置和获取机械臂相关的参数。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 获取机械臂关节状态 (4) 获取真实机械臂是否存在 (5) 获取机械臂诊断信息 (6) 获取连接状态 (7) 设置机械臂当前的工作模式 (8) 获取机械臂当前的工作模式 (9) 获取重力分量 (10) 设置机械臂的碰撞等级 (11) 获取机械臂的碰撞等级 (12) 获取设备信息 (13) 获取机械臂当前的运行状态 (14) 获取 MAC 通信状态 (15) 获取 6 关节旋转 360 使能标志 (16) 获取机械臂当前的关节角信息 (17) 获取当前的路点信息 (18) 机械臂是否在联机模式 (19) 机械臂是否在联机主模式 (20) 获取机械臂的安全配置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_4.cpp 代码如下：

注意：代码中使用 Util 类的函数，请参考 4.15 章。

```

#include "example_4.h"
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"
#include "util.h"

```

```

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>

#define SERVER_HOST "192.168.221.13"
#define SERVER_PORT 8899

Example_4::Example_4()
{

}

void Example_4::demo()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,

```

```

true,    /*保留默认为 true
*/
1000,    /*保留默认为 100
0 */
result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//1.获取机械臂关节状态
aubo_robot_namespace::JointStatus jointStatus[6];
ret = robotService.robotServiceGetRobotJointStatus(jointStatus, 6);
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "获取关节状态成功." << std::endl;

    Util::printJointStatus(jointStatus, 6);

}
else
{
    std::cerr << "获取关节状态失败." << std::endl;
}

//2.获取真实臂是否存在
bool IsRealRobotExist = false;
ret = robotService.robotServiceGetIsRealRobotExist(IsRealRobotExist);

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "真实机械臂是否存在: " << IsRealRobotExist << std::endl;
}
else
{
    std::cerr << "ERROR:获取机械臂真实臂是否存在失败." << std::endl;
}

//3.机械臂诊断信息
aubo_robot_namespace::RobotDiagnosis robotDiagnosisInfo;

```

```

ret = robotService.robotServiceGetRobotDiagnosisInfo(robotDiagnosisInfo);
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    Util::printRobotDiagnosis(robotDiagnosisInfo);
}
else
{
    std::cerr << "ERROR:获取机械臂诊断信息失败." << std::endl;
}

//4.获取连接状态
bool connectStatus;
robotService.robotServiceGetConnectStatus(connectStatus);
std::cout << "机械臂的连接状态: " << connectStatus << std::endl;

//5.设置机械臂当前工作模式
aubo_robot_namespace::RobotWorkMode workmode = aubo_robot_namespace::
RobotModeReal;
ret = robotService.robotServiceSetRobotWorkMode(workmode);
std::cout << "设置机械臂当前工作模式: " << workmode << std::endl;

//6.获取机械臂当前工作模式
aubo_robot_namespace::RobotWorkMode workmode2;
robotService.robotServiceGetRobotWorkMode(workmode2);
std::cout << "机械臂当前工作模式: " << workmode2 << std::endl;

//7.获取重力分量
aubo_robot_namespace::RobotGravityComponent gravity;
robotService.robotServiceGetRobotGravityComponent(gravity);
std::cout << "**** 重力分量 ****" << std::endl;
std::cout << "x = " << gravity.x << std::endl;
std::cout << "y = " << gravity.y << std::endl;
std::cout << "z = " << gravity.z << std::endl;

//8.设置机械臂碰撞等级
int collisionClass = 5;
ret = robotService.robotServiceSetRobotCollisionClass(collisionClass);
std::cout << "设置机械臂碰撞等级: " << collisionClass << std::endl;

//9.获取碰撞等级
int collisiongrade;
robotService.robotServiceGetRobotCollisionCurrentService(collisiongrade);
std::cout << "获取碰撞等级: " << collisiongrade << std::endl;

```

```

//10.获取设备信息
aubo_robot_namespace::RobotDevInfo robotdevinfo;
robotService.robotServiceGetRobotDevInfoService(robotdevinfo);
std::cout << "**** 机械臂设备信息 ****" << std::endl;
std::cout << "设备版本号 revision : " << robotdevinfo.revision << std::endl;
std::cout << "从设备版本号 slave version : " << robotdevinfo.slave_version
<< std::endl;
std::cout << "IO 扩展版本号 extern IO version : " << robotdevinfo.extio_ver
sion << std::endl;
std::cout << "厂家 ID manu_id : " << robotdevinfo.manu_id << std::endl;
std::cout << "机械臂类型 joint_type : " << robotdevinfo.joint_type << std::e
ndl;
for(int i = 0; i < 8; i++)
{
    if(i == 6)
    {
        std::cout <<"Tool" << " 硬件版本 hardware version is : " << robotd
evinfo.joint_ver[i].hw_version << std::endl;
        std::cout <<"Tool" << " 软件版本 software version is : " << robotde
vinfo.joint_ver[i].sw_version << std::endl;
    } else if(i == 7)
    {
        std::cout <<"Base" << " 硬件版本 hardware version is : " << robo
tdevinfo.joint_ver[i].hw_version << std::endl;
        std::cout <<"Base" << " 软件版本 software version is : " << robot
devinfo.joint_ver[i].sw_version << std::endl;
    } else
    {
        std::cout <<"关节 joint[" << i+1 << "]" 硬件版本 hardware version
is : " << robotdevinfo.joint_ver[i].hw_version << std::endl;
        std::cout <<"关节 joint[" << i+1 << "]" 软件版本 software version
is : " << robotdevinfo.joint_ver[i].sw_version << std::endl;
    }
}
for(int i = 0; i < 8; i++)
{
    if(i == 0)
    {
        std::cout <<"Interface Board ID is : " << robotdevinfo.jointProductI
D[i].productID << std::endl;
    } else if(i == 7)
    {
        std::cout <<"Tool ID is : " << robotdevinfo.jointProductID[i].produc
tID << std::endl;
    }
}

```

```

        } else
        {
            std::cout <<"关节 joint[" << i << "]" ID is : " << robotdevinfo.jointPro
ductID[i].productID << std::endl;
        }
    }

//11.获取机械臂当前运行状态
aubo_robot_namespace::RobotState robotstate;
robotService.robotServiceGetRobotCurrentState(robotstate);
std::cout << "机械臂当前运行状态: " << robotstate << std::endl;

//12.获取 MAC 通信状态
bool macconnectstate;
robotService.robotServiceGetMacCommunicationStatus(macconnectstate);
std::cout << "MAC 通信状态: " << macconnectstate << std::endl;

//13.获取 6 关节旋转 360 使能标志
bool j6_360_flag;
robotService.robotServiceGetJoint6Rotate360EnableFlag(j6_360_flag);
std::cout << "joint 6 360 flag : " << j6_360_flag << std::endl;

//14.获取机械臂当前关节角信息
aubo_robot_namespace::JointParam jointangle;
robotService.robotServiceGetJointAngleInfo(jointangle);
//关节信息
std::cout<<"关节角: "<<std::endl;
for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
{
    std::cout << "关节" << i+1 << ": " << jointangle.jointPos[i] << " ~ " <
<
    jointangle.jointPos[i]*180.0/M_PI << std::endl;
}

//15.获取当前路点信息
aubo_robot_namespace::wayPoint_S wayPoint;
robotService.robotServiceGetCurrentWaypointInfo(wayPoint);
std::cout<<std::endl<<"-----当前路点信息-----"<<std::endl;
//位置信息
std::cout<<"位置 Pos: ";
std::cout<<"x:"<<wayPoint.cartPos.position.x<<" ";
std::cout<<"y:"<<wayPoint.cartPos.position.y<<" ";
std::cout<<"z:"<<wayPoint.cartPos.position.z<<std::endl;

```



```

//姿态信息
std::cout<<"姿态 Ori: ";
std::cout<<"w:"<<wayPoint.orientation.w<<" ";
std::cout<<"x:"<<wayPoint.orientation.x<<" ";
std::cout<<"y:"<<wayPoint.orientation.y<<" ";
std::cout<<"z:"<<wayPoint.orientation.z<<std::endl;
aubo_robot_namespace::Rpy tempRpy;
robotService.quaternionToRPY(wayPoint.orientation,tempRpy);
std::cout<<"欧拉角 Rpy: ";
std::cout<<"RX:"<<tempRpy.rx*180.0/M_PI<<" RY:"<<tempRpy.ry*180.0/M_P
I
    <<" RZ:"<<tempRpy.rz*180.0/M_PI<<std::endl;
//关节信息
std::cout<<"关节位置: "<<std::endl;
for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
{
    std::cout<<"关节"<<i+1<<": "<<wayPoint.jointpos[i]<<" ~ "<<wayPoint.jo
intpos[i]*180.0/M_PI<<std::endl;
}

//16.是否在联机模式
bool isonlinemode;
robotService.robotServiceIsOnlineMode(isonlinemode);
std::cout << "机械臂是否在联机模式 : " << isonlinemode << std::endl;

//17.是否在联机主模式
bool isonlinemastermode;
robotService.robotServiceIsOnlineMasterMode(isonlinemastermode);
std::cout << "机械臂是否在联机主模式: " << isonlinemastermode << std::en
dl;

//18.获取机械臂安全配置
aubo_robot_namespace::RobotSafetyConfig safeconfig;
robotService.robotServiceGetRobotSafetyConfig(safeconfig);
std::cout << "缩减模式的关节速度限制: " << std::endl;
for(int i = 0; i < 6 ;i++)
{
    std::cout << "关节[" << i+1 << "]" 速度 = " << safeconfig.robotReduce
dConfigJointSpeed[i] << std::endl;
}
std::cout << "缩减模式的 TCP 速度限制 = " << safeconfig.robotReducedConf
igTcpSpeed << std::endl;
std::cout << "缩减模式的 TCP 力 = " << safeconfig.robotReducedConfigTcpF
orce << std::endl;

```

```

        std::cout << "缩减模式的动量 = " << safeconfig.robotReducedConfigMoment
um << std::endl;
        std::cout << "缩减模式的功率 = " << safeconfig.robotReducedConfigPower
<< std::endl;
    }

```

4.6 IO

4.6.1 工具 IO

本示例是关于工具 IO。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置工具端电源电压类型（4）获取工具端电源电压类型（5）获取工具端电源电压状态（6）获取工具端所有数字量 IO 的状态（7）设置工具端数字量 IO 的类型：输入或者输出（8）根据名称设置工具端数字量 IO 的状态（9）根据地址设置工具端数字量 IO 的状态（10）获取工具端所有 AI 的状态（11）设置工具端电源电压类型和所有数字量 IO 的类型。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_5.cpp 中的 toolio()函数代码如下：

```

#include "example_5.h"
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>

#define SERVER_HOST "192.168.221.13"
#define SERVER_PORT 8899

Example_5::Example_5()
{

}

void Example_5::toolio()
{

```

```

ServiceInterface robotService;
int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//1.设置工具端电源电压类型
aubo_robot_namespace::ToolPowerType type = aubo_robot_namespace::OUT_2
4V;
ret = robotService.robotServiceSetToolPowerVoltageType(type);

```

```

        if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
        {
            std::cout<<"INFO:设置工具端电压类型成功. 当前设置的类型:"<<type<<std::endl;
        }
        else
        {
            std::cerr<<"ERROR:设置工具端电压类型失败."<<std::endl;
        }

        //2.获取工具端电源电压类型
        aubo_robot_namespace::ToolPowerType type2;
        ret = robotService.robotServiceGetToolPowerVoltageType(type2);

        if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
        {
            std::cout<<"INFO:获取电源电压类型成功. 结果为:"<<type2<<std::endl;
        }
        else
        {
            std::cerr<<"ERROR:设置工具端电压类型失败."<<std::endl;
        }

        //3.获取工具端电源电压状态
        double ToolPowerVoltageStatus;

        ret = robotService.robotServiceGetToolPowerVoltageStatus(ToolPowerVoltageStatus);

        if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
        {
            std::cout<<"INFO:获取工具端电源电压成功 电源电压:"<<ToolPowerVoltageStatus<<std::endl;
        }
        else
        {
            std::cout<<"ERROR:获取工具端电源电压失败."<<std::endl;
        }

        //4.获取工具端所有数字量 IO 的状态
        std::vector<aubo_robot_namespace::RobotIoDesc> statusVector;
        ret = robotService.robotServiceGetAllToolDigitalIOStatus(statusVector);

```

```

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout<<"INFO:工具端数字量 IO 的状态:"<<std::endl;

    for(int i=0;i<(int)statusVector.size();i++)
    {
        std::cout <<"名称:"<<statusVector[i].ioName <<" 类型:"<<statusVect
or[i].ioType <<" 地址:"<<statusVector[i].ioAddr <<" 状态:"<<statusVector[i].i
oValue <<std::endl;
    }

    std::cout<<"INFO:获取工具端数字量 IO 的状态成功."<<std::endl;
}
else
{
    std::cerr<<"ERROR:获取工具端数字量 IO 的状态失败."<<std::endl;
}

//5.设置工具端数字量 IO 的类型：输入或者输出
aubo_robot_namespace::ToolDigitalIOAddr addr;
aubo_robot_namespace::ToolIOType value;

addr = aubo_robot_namespace::TOOL_DIGITAL_IO_1;
value = aubo_robot_namespace::IO_OUT;

ret = robotService.robotServiceSetToolDigitalIOType(addr, value);
if( ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout<<"INFO:设置工具端数字量 IO 的类型成功. 当前设置 addr:"<<a
ddr<<" 类型:"<<value<<std::endl;
}
else
{
    std::cerr<<"ERROR:设置工具端数字量 IO 的类型失败. addr:"<<addr<<st
d::endl;
}

//6.根据名称设置工具端数字量 IO 的状态
std::string name = "T_DI/O_01";
aubo_robot_namespace::IO_STATUS value2 = aubo_robot_namespace::IO_ST
ATUS_VALID;
robotService.robotServiceSetToolDOStatus(name, value2);
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{

```

```

        std::cout<<"INFO:设置工具端数字量 IO 的状态成功. 当前设置 name:"<<
name<<"  状态:"<<value2<<std::endl;
    }
    else
    {
        std::cerr<<"ERROR:设置工具端数字量 IO 的状态失败. name:"<<name<<s
td::endl;
    }

    sleep(1);

    //7.根据地址设置工具端数字量 IO 的状态
    aubo_robot_namespace::ToolDigitalIOAddr addr2;
    aubo_robot_namespace::IO_STATUS value3;

    addr2 = aubo_robot_namespace::TOOL_DIGITAL_IO_2;
    value3 = aubo_robot_namespace::IO_STATUS_INVALID;

    robotService.robotServiceSetToolDOStatus(addr2, value3);
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout<<"INFO:设置工具端数字量 IO 的状态成功. 当前设置 addr:"<
<addr2<<"  状态:"<<value3<<std::endl;
    }
    else
    {
        std::cerr<<"ERROR:设置工具端数字量 IO 的状态失败. addr:"<<addr2<<st
d::endl;
    }

    sleep(1);

    //8.获取工具端所有 AI 的状态
    std::vector<aubo_robot_namespace::RobotIoDesc> toolAIStatusVector;
    ret = robotService.robotServiceGetAllToolAIStatus(toolAIStatusVector);
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout<<"INFO:工具端所有 AI 的状态:"<<std::endl;
        for(int i=0;i<(int)toolAIStatusVector.size();i++)
        {
            std::cout <<"名称:"<<toolAIStatusVector[i].ioName <<" 类型:"<<tool
AIStatusVector[i].ioType <<"  地址:"<<toolAIStatusVector[i].ioAddr <<"  状态:
"<<toolAIStatusVector[i].ioValue <<std::endl;
        }
    }

```

```

        std::cout<<"INFO:获取工具端所有 AI 的状态成功."<<std::endl;
    }
    else
    {
        std::cerr<<"ERROR:获取工具端所有 AI 的状态失败."<<std::endl;
    }

    //9. 设置工具端电源电压类型和所有数字量 IO 的类型
    ret = robotService.robotServiceSetToolPowerTypeAndDigitalIOType(aubo_robot
_namespace::OUT_12V,
                                                                    aubo_
robot_namespace::IO_OUT, aubo_robot_namespace::IO_OUT,
                                                                    aubo_
robot_namespace::IO_OUT, aubo_robot_namespace::IO_IN);
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr<<"设置工具端 IO 类型 SUCC."<<std::endl;
    }
    else
    {
        std::cerr<<"设置工具端 IO 类型 Failed."<<std::endl;
    }
}

```

4.6.2 用户 IO

本示例是关于工具 IO。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）获取接口板 IO 配置（4）获取接口板 IO 状态（5）根据接口板 IO 类型和名称设置 IO 状态（6）根据接口板 IO 类型和地址设置 IO 状态（7）根据接口板 IO 类型和名称获取 IO 状态（8）根据接口板 IO 类型和地址获取 IO 状态。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_5.cpp 中的 userio()函数代码如下：

```

void Example_5::userio()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
}

```

```

    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位
姿 默认为 true*/,
                                                true,        /*保留默认为 true
*/
                                                1000,        /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //1.获取接口板 IO 配置
    ret = aubo_robot_namespace::InterfaceCallSuccCode;
    std::vector<aubo_robot_namespace::RobotIoType> ioType;
    std::vector<aubo_robot_namespace::RobotIoDesc> configVector;
    ioType.push_back(aubo_robot_namespace::RobotBoardControllerDI);
    ioType.push_back(aubo_robot_namespace::RobotBoardControllerDO);
    ioType.push_back(aubo_robot_namespace::RobotBoardControllerAI);
    ioType.push_back(aubo_robot_namespace::RobotBoardControllerAO);

```



```

ioType.push_back(aubo_robot_namespace::RobotBoardUserDI);
ioType.push_back(aubo_robot_namespace::RobotBoardUserDO);
ioType.push_back(aubo_robot_namespace::RobotBoardUserAI);
ioType.push_back(aubo_robot_namespace::RobotBoardUserAO);

ret = robotService.robotServiceGetBoardIOConfig(ioType, configVector);

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout<<"获取接口板 IO（包括控制柜 IO 和用户 IO）配置成功"<<std::
endl;
}
else
{
    std::cerr<<"ERROR:getBoardIOConfigAPI 失败."<<std::endl;
}

std::cout << "ioTypeVector lenth = " << ioType.size() << std::endl;
std::cout << "ioDescVector lenth = " << configVector.size() << std::endl;
for(unsigned int i = 0; i < configVector.size(); i++)
{
    std::cout << "U_DO_" << i << std::endl;
    std::cout << "ioID = " << configVector[i].ioId << " | ";
    std::cout << "ioType = " << configVector[i].ioType << " | ";
    std::cout << "ioName = " << configVector[i].ioName << " | ";
    std::cout << "ioAddr = " << configVector[i].ioAddr << " | ";
    std::cout << "ioValue = " << configVector[i].ioValue << std::endl;
}

//2.获取接口板 IO 状态
ret = aubo_robot_namespace::InterfaceCallSuccCode;
std::vector<aubo_robot_namespace::RobotIoType> ioType2;
std::vector<aubo_robot_namespace::RobotIoDesc> statusVector;
ioType2.push_back(aubo_robot_namespace::RobotBoardControllerDI);
ioType2.push_back(aubo_robot_namespace::RobotBoardControllerDO);
ioType2.push_back(aubo_robot_namespace::RobotBoardControllerAI);
ioType2.push_back(aubo_robot_namespace::RobotBoardControllerAO);
ioType2.push_back(aubo_robot_namespace::RobotBoardUserDI);
ioType2.push_back(aubo_robot_namespace::RobotBoardUserDO);
ioType2.push_back(aubo_robot_namespace::RobotBoardUserAI);
ioType2.push_back(aubo_robot_namespace::RobotBoardUserAO);

std::cout << std::endl;

```

```

ret = robotService.robotServiceGetBoardIOStatus(ioType2, statusVector);
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout<<"获取接口版 IO（包括控制柜 IO 和用户 IO）状态成功"<<std::endl;
}
else
{
    std::cerr<<"ERROR:getBoardIOStatusAPI 失败."<<std::endl;
}

std::cout << "ioTypeVector length = " << ioType2.size() << std::endl;
std::cout << "ioDescVector length = " << statusVector.size() << std::endl;
for(int i = 0; i < statusVector.size(); i++)
{
    std::cout << "U_DO_" << i << std::endl;
    std::cout << "ioID = " << statusVector[i].ioId << " | ";
    std::cout << "ioType = " << statusVector[i].ioType << " | ";
    std::cout << "ioName = " << statusVector[i].ioName << " | ";
    std::cout << "ioAddr = " << statusVector[i].ioAddr << " | ";
    std::cout << "ioValue = " << statusVector[i].ioValue << std::endl;
}

//3.根据接口板 IO 类型和名称设置 IO 状态
ret = robotService.robotServiceSetBoardIOStatus(aubo_robot_namespace::Robot
BoardUserDO, "U_DO_17", 1);
sleep(1);
if(ret != aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "根据接口板 IO 类型和名称设置 IO 状态失败!" << std::endl;
}
else
{
    std::cout << "根据接口版 IO 类型和名称设置 IO 状态成功!" << std::endl;
}

//5.根据接口板 IO 类型和地址设置 IO 状态
aubo_robot_namespace::RobotIoType iotype3 = aubo_robot_namespace::Robot
BoardUserDO;
int ioaddr2 = 40;
double iovalue2 = 1;
ret = robotService.robotServiceSetBoardIOStatus(iotype3,ioaddr2,iovalue2);

```

```

        if(ret != aubo_robot_namespace::InterfaceCallSuccCode)
        {
            std::cerr << "根据接口板 IO 类型和地址设置 IO 状态失败!" << std::endl;
        }
        else
        {
            std::cout << "根据接口板 IO 类型和地址设置 IO 状态成功!" << std::endl;
        }
        sleep(1);
        std::cout << std::endl;

        //4.根据接口板 IO 类型和名称获取 IO 状态
        double value;
        robotService.robotServiceGetBoardIOStatus(aubo_robot_namespace::RobotBoard
        UserDO, "U_DO_02", value);
        std::cerr<<"DO_02 状态:"<<value<<std::endl;

        //6.根据接口板 IO 类型和地址获取 IO 状态
        //获取 U_DI_00 的状态
        aubo_robot_namespace::RobotIoType iotype4 = aubo_robot_namespace::Robot
        BoardUserDI;
        int ioaddr3 = 36;
        double iovalue3;
        robotService.robotServiceGetBoardIOStatus(iotype4,ioaddr3,iovalue3);
        std::cout << "addr " << ioaddr3 << " = " << iovalue3 << std::endl;
        //获取 U_DO_00 的状态
        iotype4 = aubo_robot_namespace::RobotBoardUserDO;
        ioaddr3 = 40;
        robotService.robotServiceGetBoardIOStatus(iotype4,ioaddr3,iovalue3);
        std::cout << "addr " << ioaddr3 << " = " << iovalue3 << std::endl;
    }

```

4.6.3 安全 IO

本示例是关于安全 IO 之进入退出缩减模式。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置机械臂安全配置（4）进入缩减模式（5）关节运动到目标路点（6）退出缩减模式。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_5.cpp 中的 reduceMode()函数代码如下:

```
void Example_5::reduceMode()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位姿 默认为 true*/,
                                                true,      /*保留默认为 true */
                                                1000,     /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    aubo_robot_namespace::RobotSafetyConfig speed_config;
```

```

speed_config.robotReducedConfigJointSpeed[0] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigJointSpeed[1] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigJointSpeed[2] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigJointSpeed[3] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigJointSpeed[4] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigJointSpeed[5] = 15 / 180.0*M_PI;
speed_config.robotReducedConfigTcpSpeed = 10;
robotService.robotServiceSetRobotSafetyConfig(speed_config);

//进入缩减模式
robotService.robotServiceEnterRobotReduceMode();

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();
//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);
//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);
//关节角 jointAngle
double jointAngle[6] = {};
jointAngle[0] = 60.443151*M_PI/180;
jointAngle[1] = 42.275463*M_PI/180;
jointAngle[2] = -97.679737*M_PI/180;
jointAngle[3] = -49.990510*M_PI/180;
jointAngle[4] = -90.007372*M_PI/180;
jointAngle[5] = 62.567046*M_PI/180;
//关节运动
robotService.robotServiceJointMove(jointAngle, true);

//退出缩减模式

```

```

        robotService.robotServiceExitRobotReduceMode();
    }

```

4.7 TCP 转 CAN 透传模式

本示例是在 CAN 透传模式下获取关节状态。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）进入 TCP 转 CAN 透传模式（4）获取机械臂关节状态（5）退出 TCP 转 CAN 透传模式。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_6.cpp 中的 demo1()函数代码如下：

注意：代码中使用 Util 类的函数，请参考 4.15 章。

```

#include "example_6.h"
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"
#include "util.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>

#define SERVER_HOST "192.168.221.13"
#define SERVER_PORT 8899

Example_6::Example_6()
{

}

void Example_6::demo1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)

```

```

{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂，需要对机械臂进行初始化 */
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//进入 TCP 转 CAN 透传模式
ret = robotService.robotServiceEnterTcp2CanbusMode();
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "进入 TCP 转 CAN 透传模式成功。" << std::endl;
}
else
{
    std::cerr << "进入 TCP 转 CAN 透传模式失败。错误号为" << ret << st
d::endl;
}

```

```

aubo_robot_namespace::JointStatus jointStatus[6];
//获取机械臂关节状态
ret = robotService.robotServiceGetRobotJointStatus(jointStatus, 6);
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "获取关节状态成功。" << std::endl;
    //打印关节状态信息
    Util::printJointStatus(jointStatus, 6);
}
else
{
    std::cerr << "获取关节状态失败。错误号为" << ret << std::endl;
}

//退出 TCP 转 CAN 透传模式
robotService.robotServiceLeaveTcp2CanbusMode();
}

```

4.8 关节运动

4.8.1 robotServiceJointMove 函数

本示例是关节运动到指定关节角。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）初始化全局运动属性（4）设置关节运动的最大速度和加速度（5）关节运动到指定关节角。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movej()函数代码如下：

```

void Example_MoveJ::movej()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
}

```



```

    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 */
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;
    jointMaxAcc.jointPara[2] = 30*M_PI/180;
    jointMaxAcc.jointPara[3] = 30*M_PI/180;
    jointMaxAcc.jointPara[4] = 30*M_PI/180;
    jointMaxAcc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

```

```

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//关节角 jointAngle
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动
robotService.robotServiceJointMove(jointAngle, true);
}

```

4.8.2 robotMoveJointToTargetPositionByRelative 函数

示例 1：法兰盘中心在基坐标系下偏移

本示例是关节运动到目标位置，其中目标位置是法兰盘中心在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置基坐标系（5）设置偏移量（6）调用函数 robotMoveJointToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPositionByRelative1 ()函数代码如下：

```

void Example_MoveJ::movejToTargetPositionByRelative1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
}

```

```

    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;
    jointMaxAcc.jointPara[2] = 30*M_PI/180;

```

```

jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//关节运动到目标位置，其中目标位置是法兰盘中心在基坐标系下通过相对
//当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveJointToTargetPositionByRelative(baseCoord, moveRelative, true);
}

```

示例 2：法兰盘中心在用户坐标系下偏移

本示例是关节运动到目标位置，其中目标位置是法兰盘中心在用户坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始路点 (4) 设置用户坐标系及其工具参数 (5) 设置偏移量 (6) 调用函数 robotMoveJointToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPositionByRelative2 ()函数代码如下：

```
void Example_MoveJ::movejToTargetPositionByRelative2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6, /*碰撞等级*/,
                                                true /*是否允许读取位姿 默认为 true*/,
                                                true, /*保留默认为 true */
                                                1000, /*保留默认为 1000 */
                                                0 */
```

```

result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

```

```

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = -0.177341;
toolUserCoord.toolInEndPosition.y = 0.002327;
toolUserCoord.toolInEndPosition.z = 0.146822;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

```

```

//关节运动到目标位置，其中目标位置是法兰盘中心在用户坐标系下通过相
对当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveJointToTargetPositionByRelative(userCoord, moveRelati
ve, true);
}

```

示例 3：工具末端在工具坐标系下偏移

本示例是关节运动到目标位置，其中目标位置是工具末端在工具坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具的运动学参数（5）设置末端坐标系及其工具参数（6）设置偏移量（7）调用函数 robotMoveJointToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPositionByRelative3 ()函数代码如下：

```

void Example_MoveJ::movejToTargetPositionByRelative3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学

```



```

参数**/,
                                6          /*碰撞等级*/,
                                true        /*是否允许读取位
姿 默认为 true*/,
                                true,      /*保留默认为 true
*/
                                1000,      /*保留默认为 100
0 */
                                result); /*机械臂初始化*/

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;

```

```

initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置工具坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool endCoord;
endCoord.coordType = aubo_robot_namespace::EndCoordinate;
endCoord.toolDesc = toolEnd;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//关节运动到目标位置，其中目标位置是工具末端在工具坐标系下通过相对
//当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveJointToTargetPositionByRelative(endCoord, moveRelative, true);
}

```

示例 4：工具末端在基坐标系下偏移

本示例是关节运动到目标位置，其中目标位置是工具末端在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具的运动学参数（5）设置基坐标系（6）设置偏移量（7）调用函数

robotMoveJointToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPositionByRelative4 ()函数代码如下：

```
void Example_MoveJ::movejToTargetPositionByRelative4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {

```

```

        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;
    jointMaxAcc.jointPara[2] = 30*M_PI/180;
    jointMaxAcc.jointPara[3] = 30*M_PI/180;
    jointMaxAcc.jointPara[4] = 30*M_PI/180;
    jointMaxAcc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

    //设置关节运动最大速度
    aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
    jointMaxVelc.jointPara[0] = 30*M_PI/180;
    jointMaxVelc.jointPara[1] = 30*M_PI/180;
    jointMaxVelc.jointPara[2] = 30*M_PI/180;
    jointMaxVelc.jointPara[3] = 30*M_PI/180;
    jointMaxVelc.jointPara[4] = 30*M_PI/180;
    jointMaxVelc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

    //起始路点
    double initAngle[6] = {};
    initAngle[0] = 173.108713*M_PI/180;
    initAngle[1] = -12.075005*M_PI/180;
    initAngle[2] = -83.663342*M_PI/180;
    initAngle[3] = -15.641249*M_PI/180;
    initAngle[4] = -89.140000*M_PI/180;
    initAngle[5] = -28.328713*M_PI/180;

    //关节运动到起始路点
    robotService.robotServiceJointMove(initAngle, true);

    //设置工具
    aubo_robot_namespace::ToolInEndDesc toolEnd;
    toolEnd.toolInEndPosition.x = -0.177341;
    toolEnd.toolInEndPosition.y = 0.002327;
    toolEnd.toolInEndPosition.z = 0.146822;
    toolEnd.toolInEndOrientation.w = 1;

```

```

    toolEnd.toolInEndOrientation.x = 0;
    toolEnd.toolInEndOrientation.y = 0;
    toolEnd.toolInEndOrientation.z = 0;
    robotService.robotServiceSetToolKinematicsParam(toolEnd);

    //设置基坐标系
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
    baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    //设置偏移量
    aubo_robot_namespace::MoveRelative moveRelative;
    moveRelative.relativePosition[0] = 0.001;
    moveRelative.relativePosition[1] = 0;
    moveRelative.relativePosition[2] = 0;

    //关节运动到目标位置，其中目标位置是工具末端在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的
    robotService.robotMoveJointToTargetPositionByRelative(baseCoord, moveRelative, true);
}

```

示例 5：工具末端在用户坐标系下偏移

本示例是关节运动到目标位置，其中目标位置是工具末端在用户坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具的运动学参数（5）设置用户坐标系及其工具参数（6）设置偏移量（7）调用函数 robotMoveJointToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPositionByRelative5 ()函数代码如下：

```

void Example_MoveJ::movejToTargetPositionByRelative5()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
}

```

```

    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 */
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;
    jointMaxAcc.jointPara[2] = 30*M_PI/180;
    jointMaxAcc.jointPara[3] = 30*M_PI/180;
    jointMaxAcc.jointPara[4] = 30*M_PI/180;
    jointMaxAcc.jointPara[5] = 30*M_PI/180;
    robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

```

```

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;

```

```

userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

userCoord.toolDesc = toolEnd;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//关节运动到目标位置，其中目标位置是工具末端在用户坐标系下通过相对
当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveJointToTargetPositionByRelative(userCoord, moveRelative, true);
}

```

4.8.3 robotMoveJointToTargetPosition 函数

示例 1：法兰盘中心在基坐标系下的位置

本示例是关节运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数，为法兰盘中心（5）设置基坐标系（6）设置目标位置（7）调用函数 robotMoveJointToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPosition1()函数代码如下：


```

void Example_MoveJ::movejToTargetPosition1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位姿 默认为 true*/,
                                                true,      /*保留默认为 true */
                                                1000,     /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
}

```

```

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为法兰盘中心
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = 0;
toolEnd.toolInEndPosition.y = 0;
toolEnd.toolInEndPosition.z = 0;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置基坐标系

```

```

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置目标位置，法兰盘中心在基坐标系下的位置
aubo_robot_namespace::Pos posOnBase;
posOnBase.x = -0.2;
posOnBase.y = -0.6;
posOnBase.z = 0;

//轴动运动至目标位置
robotService.robotMoveJointToTargetPosition(baseCoord, posOnBase, toolEnd,
true);

}

```

示例 2：法兰盘中心在用户坐标系下的位置

本示例是关节运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数，为法兰盘中心（5）设置用户坐标系及其工具参数（6）设置目标位置（7）调用函数 robotMoveJointToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPosition2()函数代码如下：

```

void Example_MoveJ::movejToTargetPosition2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }
}

```

```

/** 如果是连接真实机械臂，需要对机械臂进行初始化 */
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true       /*是否允许读取位
姿 默认为 true*/,
                                true,     /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;

```

```

jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为法兰盘中心
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = 0;
toolEnd.toolInEndPosition.y = 0;
toolEnd.toolInEndPosition.z = 0;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;

```

```

userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

//坐标系的工具参数
aubo_robot_namespace::ToolInEndDesc toolDesc;
toolDesc.toolInEndPosition.x = -0.177341;
toolDesc.toolInEndPosition.y = 0.002327;
toolDesc.toolInEndPosition.z = 0.146822;
toolDesc.toolInEndOrientation.w = 1;
toolDesc.toolInEndOrientation.x = 0;
toolDesc.toolInEndOrientation.y = 0;
toolDesc.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolDesc;

//设置目标位置，法兰盘中心在用户坐标系下的位置
aubo_robot_namespace::Pos posOnUser;
posOnUser.x = -0.220305;
posOnUser.y = -1.152177;
posOnUser.z = 0.216184;

//轴动运动至目标位置
robotService.robotMoveJointToTargetPosition(userCoord, posOnUser, toolEnd, true);
}

```

示例 3：工具末端在基坐标系下的位置

本示例是关节运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数（5）设置基坐标系（6）设置目标位置（7）调用函数 `robotMoveJointToTargetPosition` 移动到目标位置。

注意：要将下面代码中的 IP 地址（`SERVER_HOST`）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 `movejToTargetPosition3()` 函数代码如下：

```
void Example_MoveJ::movejToTargetPosition3()
```

```

{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6, /*碰撞等级*/,
                                                true, /*是否允许读取位姿 默认为 true*/,
                                                true, /*保留默认为 true */
                                                1000, /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
}

```

```

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为工具末端
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;

```



```

baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置目标位置，工具末端在基坐标系下的位置
aubo_robot_namespace::Pos posOnBase;
posOnBase.x = 0.6;
posOnBase.y = 0.0;
posOnBase.z = 0.5;

//轴动运动至目标位置
robotService.robotMoveJointToTargetPosition(baseCoord, posOnBase, toolEnd,
true);
}

```

示例 4：工具末端在用户坐标系下的位置

本示例是关节运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数（5）设置用户坐标系及其工具参数（6）设置目标位置（7）调用函数 robotMoveJointToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movej.cpp 中的 movejToTargetPosition4()函数代码如下：

```

void Example_MoveJ::movejToTargetPosition4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

```

```

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true       /*是否允许读取位
姿 默认为 true*/,
                                true,     /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

```

```

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为工具末端
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;

```

```

userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

//坐标系的工具参数
aubo_robot_namespace::ToolInEndDesc toolDesc;
toolDesc.toolInEndPosition.x = -0.177341;
toolDesc.toolInEndPosition.y = 0.002327;
toolDesc.toolInEndPosition.z = 0.146822;
toolDesc.toolInEndOrientation.w = 1;
toolDesc.toolInEndOrientation.x = 0;
toolDesc.toolInEndOrientation.y = 0;
toolDesc.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolDesc;

//设置目标位置，工具末端在用户坐标系下的位置
aubo_robot_namespace::Pos posOnUser;
posOnUser.x = -0.368243;
posOnUser.y = -1.167162;
posOnUser.z = 0.195709;

//轴动运动至目标位置
robotService.robotMoveJointToTargetPosition(userCoord, posOnUser, toolEnd, true);
}

```

4.9 跟随模式

4.9.1 跟随模式的轴动 robotServiceFollowModeJointMove

本示例是跟随模式的轴动。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到初始位置（4）基于跟随模式的关节运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_followmode.cpp 中的 followModeJointMove()函数代码如下：

```

void Example_FollowMode::followModeJointMove()
{

```

```

ServiceInterface robotService;
int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度

```

```

aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//初始位置
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动到初始位置
robotService.robotServiceJointMove(jointAngle, true);

//基于跟随模式的关节运动
for(int i = 0; i < 1000; i++)
{
    jointAngle[0] = jointAngle[0] + 0.0001;
    std::cout << jointAngle[0] << std::endl;
    robotService.robotServiceFollowModeJointMove(jointAngle);
    usleep(1000*5);
}
}

```

4.9.2 跟随模式之提前到位

本示例是跟随模式之提前到位。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到零位姿态 (4) 循环 5 次：当 i=偶数时，是在提前到位的距离模式下，关节运动到路点 1 和路点 2；当 i=奇数时，是在无提前到位模式下，然后关节运动到路点 1 和路点 2。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_movefollowmode.cpp 中的 arrivalAhead() 函数代码如下：

```
void Example_FollowMode::arrivalAhead()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
}
```

```

else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//机械臂运动到零位姿态
double jointAngle[aubo_robot_namespace::ARM_DOF] = {0};
jointAngle[0] = 0.0/180.0*M_PI;
jointAngle[1] = 0.0/180.0*M_PI;
jointAngle[2] = 0.0/180.0*M_PI;
jointAngle[3] = 0.0/180.0*M_PI;
jointAngle[4] = 0.0/180.0*M_PI;
jointAngle[5] = 0.0/180.0*M_PI;

robotService.robotServiceJointMove(jointAngle, true);

for(int i=0; i<5; i++)
{
    if(i%2==0)
    {

```



```

        //跟随模式之提前到位 当前仅适用于关节运动
        //设置提前到位的距离模式
        robotService.robotServiceSetArrivalAheadDistanceMode(0.2);
    }
    else
    {
        robotService.robotServiceSetNoArrivalAhead();
    }

    jointAngle[0] = 20.0/180.0*M_PI;
    jointAngle[1] = 0.0/180.0*M_PI;
    jointAngle[2] = 90.0/180.0*M_PI;
    jointAngle[3] = 0.0/180.0*M_PI;
    jointAngle[4] = 90.0/180.0*M_PI;
    jointAngle[5] = 0.0/180.0*M_PI;
    robotService.robotServiceJointMove(jointAngle, true);
    if(ret != aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "运动 1 失败。错误号为: " << ret << std::endl;
        break;
    }
    else
    {
        std::cerr << "运动 1 成功。i = " << i << std::endl;
    }

    jointAngle[0] = 50.0/180.0*M_PI;
    jointAngle[1] = 40.0/180.0*M_PI;
    jointAngle[2] = 78.0/180.0*M_PI;
    jointAngle[3] = 20.0/180.0*M_PI;
    jointAngle[4] = 66.0/180.0*M_PI;
    jointAngle[5] = 0.0/180.0*M_PI;
    robotService.robotServiceJointMove(jointAngle, true);
    if(ret != aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "运动 2 失败。错误号为: " << ret << std::endl;
        break;
    }
    else
    {
        std::cerr << "运动 2 成功。 i = " << i << std::endl;
    }
}
}

```

4.10 直线运动

4.10.1 robotServiceLineMove 函数

本示例是直线运动到指定关节角。

本程序的主要流程是：本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到初始位置（4）直线运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moveL.cpp 中的 moveL()函数代码如下：

```
void Example_MoveL::moveL()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true
*/
                                                1000,      /*保留默认为 100
```

```

0 */
                                                                    result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//设置末端型运动最大加速度
double lineMaxAcc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineAcc(lineMaxAcc);

//设置末端型运动最大速度
double lineMaxVelc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineVelc(lineMaxVelc);

//初始位置
double jointAngle[6] = {};

```

```

    jointAngle[0] = 173.108713*M_PI/180;
    jointAngle[1] = -12.075005*M_PI/180;
    jointAngle[2] = -83.663342*M_PI/180;
    jointAngle[3] = -15.641249*M_PI/180;
    jointAngle[4] = -89.140000*M_PI/180;
    jointAngle[5] = -28.328713*M_PI/180;

    //关节运动到初始位置
    robotService.robotServiceJointMove(jointAngle, true);

    jointAngle[0] = 206.372431*M_PI/180;
    jointAngle[1] = -8.979195*M_PI/180;
    jointAngle[2] = -99.242567*M_PI/180;
    jointAngle[3] = -30.164649*M_PI/180;
    jointAngle[4] = -107.130486*M_PI/180;
    jointAngle[5] = 0.065458*M_PI/180;

    //直线运动
    robotService.robotServiceLineMove(jointAngle, true);

}

```

4.10.2 robotMoveLineToTargetPositionByRelative 函数

示例 1：法兰盘中心在基坐标系下偏移

本示例是直线运动到目标位置，其中目标位置是法兰盘中心在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置基坐标系（5）设置偏移量（6）调用函数 robotMoveLineToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 move1ToTargetPositionByRelative1 ()函数代码如下：

```

void Example_MoveL::move1ToTargetPositionByRelative1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
}

```

```

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;

```

```

jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//直线运动到目标位置，其中目标位置是法兰盘中心在基坐标系下通过相对
当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveLineToTargetPositionByRelative(baseCoord, moveRelative, true);
}

```

示例 2：法兰盘中心在用户坐标系下偏移

本示例是直线运动到目标位置，其中目标位置是法兰盘中心在用户坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始路点 (4) 设置用户坐标系及其工具参数 (5) 设置偏移量 (6) 调用函数 robotMoveLineToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_move1.cpp 中的 moveToTargetPositionByRelative2 ()函数代码如下：

```
void Example_MoveL::moveToTargetPositionByRelative2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true
*/
                                                1000,      /*保留默认为 100
0 */
```

```

result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

```



```

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = -0.177341;
toolUserCoord.toolInEndPosition.y = 0.002327;
toolUserCoord.toolInEndPosition.z = 0.146822;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

```

```

//直线运动到目标位置，其中目标位置是法兰盘中心在用户坐标系下通过相
对当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveLineToTargetPositionByRelative(userCoord, moveRelati
ve, true);
}

```

示例 3：工具末端在工具坐标系下偏移

本示例是直线运动到目标位置，其中目标位置是工具末端在工具坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具的运动学参数（5）设置末端坐标系及其工具参数（6）设置偏移量（7）调用函数 robotMoveLineToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 move1ToTargetPositionByRelative3 ()函数代码如下：

```

void Example_MoveL::move1ToTargetPositionByRelative3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学

```

```

参数**/,
                                6          /*碰撞等级*/,
                                true        /*是否允许读取位
姿 默认为 true*/,
                                true,      /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;

```

```

initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置工具坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool endCoord;
endCoord.coordType = aubo_robot_namespace::EndCoordinate;
endCoord.toolDesc = toolEnd;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//直线运动到目标位置，其中目标位置是工具末端在工具坐标系下通过相对
//当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveLineToTargetPositionByRelative(endCoord, moveRelative, true);
}

```

示例 4：工具末端在基坐标系下偏移

本示例是直线运动到目标位置，其中目标位置是工具末端在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始路点 (4) 设置工具的运动学参数 (5) 设置基坐标系 (6) 设置偏移量 (7) 调用函数 `robotMoveLineToTargetPositionByRelative` 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 move1ToTargetPositionByRelative4 ()函数代码如下

```
void Example_MoveL::move1ToTargetPositionByRelative4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,        /*保留默认为 true */
                                                1000,       /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }
}
```

```

}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;

```

```

    toolEnd.toolInEndOrientation.y = 0;
    toolEnd.toolInEndOrientation.z = 0;
    robotService.robotServiceSetToolKinematicsParam(toolEnd);

    //设置基坐标系
    aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
    baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

    //设置偏移量
    aubo_robot_namespace::MoveRelative moveRelative;
    moveRelative.relativePosition[0] = 0.001;
    moveRelative.relativePosition[1] = 0;
    moveRelative.relativePosition[2] = 0;

    //直线运动到目标位置，其中目标位置是工具末端在基坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的
    robotService.robotMoveLineToTargetPositionByRelative(baseCoord, moveRelative, true);
}

```

示例 5：工具末端在用户坐标系下偏移

本示例是直线运动到目标位置，其中目标位置是工具末端在用户坐标系下通过相对当前位置沿 X 轴偏移 0.001m 给出的。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具的运动学参数（5）设置用户坐标系及其工具参数（6）设置偏移量（7）调用函数 robotMoveLineToTargetPositionByRelative 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 moveToTargetPositionByRelative5 ()函数代码如下

```

void Example_MoveL::moveToTargetPositionByRelative5()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
}

```

```

else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度

```



```

aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

```

```

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

userCoord.toolDesc = toolEnd;

//设置偏移量
aubo_robot_namespace::MoveRelative moveRelative;
moveRelative.relativePosition[0] = 0.001;
moveRelative.relativePosition[1] = 0;
moveRelative.relativePosition[2] = 0;

//直线运动到目标位置，其中目标位置是工具末端在用户坐标系下通过相对
//当前位置沿 X 轴偏移 0.001m 给出的
robotService.robotMoveLineToTargetPositionByRelative(userCoord, moveRelative, true);
}

```

4.10.3 robotMoveLineToTargetPosition 函数

示例 1：法兰盘中心在基坐标系下的位置

本示例是直线运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数，为法兰盘中心（5）设置基坐标系（6）设置目标位置（7）调用函数 robotMoveLineToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 moveToTargetPosition1()函数代码如下：

```

void Example_MoveL::moveToTargetPosition1()
{

```

```

ServiceInterface robotService;
int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub
o", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度

```

```

aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为法兰盘中心
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = 0;
toolEnd.toolInEndPosition.y = 0;
toolEnd.toolInEndPosition.z = 0;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

```

```

//设置目标位置，法兰盘中心在基坐标系下的位置
aubo_robot_namespace::Pos posOnBase;
posOnBase.x = -0.2;
posOnBase.y = -0.6;
posOnBase.z = 0;

//直线运动至目标位置
robotService.robotMoveLineToTargetPosition(baseCoord, posOnBase, toolEnd,
true);
}

```

示例 2：法兰盘中心在用户坐标系下的位置

本示例是直线运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数，为法兰盘中心（5）设置用户坐标系及其工具参数（6）设置目标位置（7）调用函数 robotMoveLineToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 move1ToTargetPosition2()函数代码如下：

```

void Example_MoveL::move1ToTargetPosition2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;
}

```

```

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true       /*是否允许读取位
姿 默认为 true*/,
                                true,     /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

```

```

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为法兰盘中心
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = 0;
toolEnd.toolInEndPosition.y = 0;
toolEnd.toolInEndPosition.z = 0;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;

```

```

userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

//坐标系的工具参数
aubo_robot_namespace::ToolInEndDesc toolDesc;
toolDesc.toolInEndPosition.x = -0.177341;
toolDesc.toolInEndPosition.y = 0.002327;
toolDesc.toolInEndPosition.z = 0.146822;
toolDesc.toolInEndOrientation.w = 1;
toolDesc.toolInEndOrientation.x = 0;
toolDesc.toolInEndOrientation.y = 0;
toolDesc.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolDesc;

//设置目标位置，法兰盘中心在用户坐标系下的位置
aubo_robot_namespace::Pos posOnUser;
posOnUser.x = -0.220305;
posOnUser.y = -1.152177;
posOnUser.z = 0.216184;

//直线运动至目标位置
robotService.robotMoveLineToTargetPosition(userCoord, posOnUser, toolEnd, true);
}

```

示例 3：工具末端在基坐标系下的位置

本示例是直线运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数（5）设置基坐标系（6）设置目标位置（7）调用函数 `robotMoveLineToTargetPosition` 移动到目标位置。

注意：要将下面代码中的 IP 地址（`SERVER_HOST`）修改为对应服务器的 IP 地址。

`example_move1.cpp` 中的 `moveToTargetPosition3()` 函数代码如下：

```

void Example_MoveL::moveToTargetPosition3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;
}

```



```

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6, /*碰撞等级*/,
                                                true, /*是否允许读取位姿 默认为 true*/,
                                                true, /*保留默认为 true */
                                                1000, /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;

```

```

jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为工具末端
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//设置目标位置，工具末端在基坐标系下的位置
aubo_robot_namespace::Pos posOnBase;

```

```

    posOnBase.x = 0.56;
    posOnBase.y = -0.10;
    posOnBase.z = 0.33;

    //直线运动至目标位置
    robotService.robotMoveLineToTargetPosition(baseCoord, posOnBase, toolEnd,
true);
}

```

示例 4：工具末端在用户坐标系下的位置

本示例是直线运动到目标位置。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始路点（4）设置工具参数（5）设置用户坐标系及其工具参数（6）设置目标位置（7）调用函数 robotMoveLineToTargetPosition 移动到目标位置。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_move1.cpp 中的 move1ToTargetPosition2()函数代码如下：

```

void Example_MoveL::move1ToTargetPosition4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
}

```

```

    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};

```

```

initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具参数，为工具末端
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;

```

```

userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

//坐标系的工具参数
aubo_robot_namespace::ToolInEndDesc toolDesc;
toolDesc.toolInEndPosition.x = -0.177341;
toolDesc.toolInEndPosition.y = 0.002327;
toolDesc.toolInEndPosition.z = 0.146822;
toolDesc.toolInEndOrientation.w = 1;
toolDesc.toolInEndOrientation.x = 0;
toolDesc.toolInEndOrientation.y = 0;
toolDesc.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolDesc;

//设置目标位置，工具末端在用户坐标系下的位置
aubo_robot_namespace::Pos posOnUser;
posOnUser.x = 0.701194;
posOnUser.y = -0.730656;
posOnUser.z = 0.386717;

//直线运动至目标位置
robotService.robotMoveLineToTargetPosition(userCoord, posOnUser, toolEnd, true);
}

```

4.11 偏移运动

4.11.1 robotServiceSetMoveRelativeParam 函数

示例 1：法兰盘中心在基坐标系下

本示例是机械臂做姿态偏移运动，法兰盘中心在基坐标系下沿 X 轴姿态偏移 10 度。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置基坐标系（4）设置偏移量（5）关节运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverelative.cpp 中的 demo1()函数代码如下：

```

void Example_MoveRotate::demo1()
{
    ServiceInterface robotService;

```

```

int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位姿 默认为 true*/,
                                           true,       /*保留默认为 true */
                                           1000,      /*保留默认为 1000 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;

```

```

jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//关节角
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动到起始位置
robotService.robotServiceJointMove(jointAngle, true);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：法兰盘中心在基坐标系下沿 X+方向旋转 10 度，当前位置保持
不变
robotService.robotServiceRotateMove(baseCoord, rotateAxis, rotateAngle, true);
}

```


示例 2：法兰盘中心在用户坐标系下

本示例是机械臂做姿态偏移运动，法兰盘中心在用户坐标系下沿 X 轴姿态偏移 10 度。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置用户坐标系及其工具参数（4）设置偏移量（5）关节运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverelative.cpp 中的 demo2()函数代码如下：

```
void Example_MoveRotate::demo2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
```

```

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//关节角
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动到起始位置
robotService.robotServiceJointMove(jointAngle, true);

```

```

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = -0.177341;
toolUserCoord.toolInEndPosition.y = 0.002327;
toolUserCoord.toolInEndPosition.z = 0.146822;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：法兰盘中心在用户坐标系下沿 X+方向旋转 10 度，当前位置保
持不变
robotService.robotServiceRotateMove(userCoord, rotateAxis, rotateAngle, true);

```

```
}

```

示例 3：工具末端在工具坐标系下

本示例是机械臂做姿态偏移运动，工具在工具坐标系下沿 X 轴姿态偏移 10 度。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 设置工具的运动学参数 (4) 设置末端坐标系及其工具参数 (5) 设置偏移量 (6) 关节运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_moverelative.cpp 中的 demo3()函数代码如下：

```
void Example_MoveRotate::demo3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6, /*碰撞等级*/,
                                                true, /*是否允许读取位姿 默认为 true*/,
                                                true, /*保留默认为 true */
                                                1000, /*保留默认为 1000 */
                                                0 */

```

```

result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

```

```

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置工具坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool endCoord;
endCoord.coordType = aubo_robot_namespace::EndCoordinate;
endCoord.toolDesc = toolEnd;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：工具末端在工具坐标系下沿 X+方向旋转 10 度，当前位置保持
不变
robotService.robotServiceRotateMove(endCoord, rotateAxis, rotateAngle, true);
}

```

示例 4：工具末端在基坐标系下

本示例是机械臂做姿态偏移运动，工具在基坐标系下沿 X 轴姿态偏移 10 度。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置工具的运动学参数（4）设置基坐标系（5）设置偏移量（6）关节运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverelative.cpp 中的 demo4()函数代码如下：

```

void Example_MoveRotate::demo4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aub

```

```

o", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位
姿 默认为 true*/,
                                                true,      /*保留默认为 true
*/
                                                1000,     /*保留默认为 100
0 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();

    //设置关节运动最大加速度
    aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
    jointMaxAcc.jointPara[0] = 30*M_PI/180;
    jointMaxAcc.jointPara[1] = 30*M_PI/180;
    jointMaxAcc.jointPara[2] = 30*M_PI/180;
    jointMaxAcc.jointPara[3] = 30*M_PI/180;

```

```

jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

```



```

//旋转运动：工具末端在基坐标系下沿 X+方向旋转 10 度，当前位置保持不
变
robotService.robotServiceRotateMove(baseCoord, rotateAxis, rotateAngle, true);
}

```

示例 5：工具末端在用户坐标系下

本示例是机械臂做偏移运动，工具在用户坐标系下沿 X 轴姿态偏移 10 度。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）设置工具的运动学参数（4）设置用户坐标系及其工具参数（5）设置偏移量（6）关节运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverelative.cpp 中的 demo5()函数代码如下：

```

void Example_MoveRotate::demo5()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位

```

```

姿 默认为 true*/,
                                true,    /*保留默认为 true
*/
                                1000,    /*保留默认为 100
0 */
                                result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;

```

```

initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

```

```

        userCoord.toolDesc = toolEnd;

        //旋转轴和旋转角度
        double rotateAxis[3] = {1, 0, 0};
        double rotateAngle = 10.0*M_PI/180;

        //旋转运动：工具末端在用户坐标系下沿 X+方向旋转 10 度，当前位置保持
        不变
        robotService.robotServiceRotateMove(userCoord, rotateAxis, rotateAngle, true);
    }

```

4.12 旋转运动

4.12.1 robotServiceRotateMove 函数

示例 1：法兰盘中心在基坐标系下

本示例是法兰盘中心在用户坐标系下沿 X+方向旋转 10 度，当前位置保持不变。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始位置 (5) 设置基坐标系 (6) 设置旋转轴和旋转角度 (7) 旋转运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_moverotate.cpp 中的 demo1()函数代码如下：

```

void Example_MoveRotate::demo1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }
}

```

```

/** 如果是连接真实机械臂，需要对机械臂进行初始化 */
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,       /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;

```

```

jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//关节角
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动到起始位置
robotService.robotServiceJointMove(jointAngle, true);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：法兰盘中心在基坐标系下沿 X+方向旋转 10 度，当前位置保持
不变
robotService.robotServiceRotateMove(baseCoord, rotateAxis, rotateAngle, true);
}

```

示例 2：法兰盘中心在用户坐标系下

本示例是法兰盘中心在用户坐标系下沿 X+方向旋转 10 度，当前位置保持不变。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始位置 (5) 设置用户坐标系及其工具参数 (6) 设置旋转轴和旋转角度 (7) 旋转运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_moverotate.cpp 中的 demo2()函数代码如下：

```

void Example_MoveRotate::demo2()
{
    ServiceInterface robotService;

```

```

int ret = aubo_robot_namespace::InterfaceCallSuccCode;

/** 接口调用: 登录 ***/
ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位姿 默认为 true*/,
                                           true,       /*保留默认为 true */
                                           1000,      /*保留默认为 1000 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;

```

```

jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//关节角
double jointAngle[6] = {};
jointAngle[0] = 173.108713*M_PI/180;
jointAngle[1] = -12.075005*M_PI/180;
jointAngle[2] = -83.663342*M_PI/180;
jointAngle[3] = -15.641249*M_PI/180;
jointAngle[4] = -89.140000*M_PI/180;
jointAngle[5] = -28.328713*M_PI/180;

//关节运动到起始位置
robotService.robotServiceJointMove(jointAngle, true);

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;

```



```

userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

aubo_robot_namespace::ToolInEndDesc toolUserCoord;
toolUserCoord.toolInEndPosition.x = -0.177341;
toolUserCoord.toolInEndPosition.y = 0.002327;
toolUserCoord.toolInEndPosition.z = 0.146822;
toolUserCoord.toolInEndOrientation.w = 1;
toolUserCoord.toolInEndOrientation.x = 0;
toolUserCoord.toolInEndOrientation.y = 0;
toolUserCoord.toolInEndOrientation.z = 0;
userCoord.toolDesc = toolUserCoord;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：法兰盘中心在用户坐标系下沿 X+方向旋转 10 度，当前位置保持不变
robotService.robotServiceRotateMove(userCoord, rotateAxis, rotateAngle, true);
}

```

示例 3：工具末端在工具坐标系下

本示例是工具末端在工具坐标系下沿 X+方向旋转 10 度，当前位置保持不变。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始位置（5）设置工具（6）设置末端坐标系及其工具参数（7）设置旋转轴和旋转角度（8）旋转运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverotate.cpp 中的 demo3()函数代码如下：

```
void Example_MoveRotate::demo3()
```

```

{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂, 需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6, /*碰撞等级*/,
                                                true, /*是否允许读取位姿 默认为 true*/,
                                                true, /*保留默认为 true */
                                                1000, /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }

    //初始化运动属性
    robotService.robotServiceInitGlobalMoveProfile();
}

```

```

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置工具坐标系

```

```

aubo_robot_namespace::CoordCalibrateByJointAngleAndTool endCoord;
endCoord.coordType = aubo_robot_namespace::EndCoordinate;
endCoord.toolDesc = toolEnd;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：工具末端在工具坐标系下沿 X+方向旋转 10 度，当前位置保持不变
robotService.robotServiceRotateMove(endCoord, rotateAxis, rotateAngle, true);
}

```

示例 4：工具末端在基坐标系下

本示例是工具末端在基坐标系下沿 X+方向旋转 10 度，当前位置保持不变。本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到起始位置 (5) 设置工具 (6) 设置基坐标系 (7) 设置旋转轴和旋转角度 (8) 旋转运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_moverotate.cpp 中的 demo4()函数代码如下：

```

void Example_MoveRotate::demo4()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数

```

```

aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                6          /*碰撞等级*/,
                                true       /*是否允许读取位
姿 默认为 true*/,
                                true,     /*保留默认为 true
*/
                                1000,     /*保留默认为 100
0 */
                                result); /*机械臂初始化*/

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

```

```

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;
toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
baseCoord.coordType = aubo_robot_namespace::BaseCoordinate;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：工具末端在基坐标系下沿 X+方向旋转 10 度，当前位置保持不
变
robotService.robotServiceRotateMove(baseCoord, rotateAxis, rotateAngle, true);
}

```

示例 5：工具末端在用户坐标系下

本示例是工具末端在用户坐标系下沿 X+方向旋转 10 度，当前位置保持不变。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到起始位置（5）设置工具（6）设置用户坐标系及其工具参数（7）设置旋转轴和旋转角度（8）旋转运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_moverotate.cpp 中的 demo5()函数代码如下：

```
void Example_MoveRotate::demo5()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cerr << "机械臂初始化成功." << std::endl;
    }
    else
    {
        std::cerr << "机械臂初始化失败." << std::endl;
    }
}
```

```

}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 30*M_PI/180;
jointMaxVelc.jointPara[1] = 30*M_PI/180;
jointMaxVelc.jointPara[2] = 30*M_PI/180;
jointMaxVelc.jointPara[3] = 30*M_PI/180;
jointMaxVelc.jointPara[4] = 30*M_PI/180;
jointMaxVelc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//起始路点
double initAngle[6] = {};
initAngle[0] = 173.108713*M_PI/180;
initAngle[1] = -12.075005*M_PI/180;
initAngle[2] = -83.663342*M_PI/180;
initAngle[3] = -15.641249*M_PI/180;
initAngle[4] = -89.140000*M_PI/180;
initAngle[5] = -28.328713*M_PI/180;

//关节运动到起始路点
robotService.robotServiceJointMove(initAngle, true);

//设置工具
aubo_robot_namespace::ToolInEndDesc toolEnd;
toolEnd.toolInEndPosition.x = -0.177341;
toolEnd.toolInEndPosition.y = 0.002327;
toolEnd.toolInEndPosition.z = 0.146822;
toolEnd.toolInEndOrientation.w = 1;
toolEnd.toolInEndOrientation.x = 0;

```



```

toolEnd.toolInEndOrientation.y = 0;
toolEnd.toolInEndOrientation.z = 0;
robotService.robotServiceSetToolKinematicsParam(toolEnd);

//设置用户坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool userCoord;
userCoord.coordType = aubo_robot_namespace::WorldCoordinate;
userCoord.methods = aubo_robot_namespace::Origin_AnyPointOnPositiveXAxis
_AnyPointOnFirstQuadrantOfXOYPlane;

userCoord.wayPointArray[0].jointPos[0] = -75.093279*M_PI/180;
userCoord.wayPointArray[0].jointPos[1] = 28.544643*M_PI/180;
userCoord.wayPointArray[0].jointPos[2] = -114.313905*M_PI/180;
userCoord.wayPointArray[0].jointPos[3] = -62.769247*M_PI/180;
userCoord.wayPointArray[0].jointPos[4] = -87.343517*M_PI/180;
userCoord.wayPointArray[0].jointPos[5] = -27.888262*M_PI/180;

userCoord.wayPointArray[1].jointPos[0] = -89.239837*M_PI/180;
userCoord.wayPointArray[1].jointPos[1] = 23.936171*M_PI/180;
userCoord.wayPointArray[1].jointPos[2] = -122.299277*M_PI/180;
userCoord.wayPointArray[1].jointPos[3] = -65.208902*M_PI/180;
userCoord.wayPointArray[1].jointPos[4] = -85.011123*M_PI/180;
userCoord.wayPointArray[1].jointPos[5] = -41.87417*M_PI/180;

userCoord.wayPointArray[2].jointPos[0] = -77.059212*M_PI/180;
userCoord.wayPointArray[2].jointPos[1] = 35.509518*M_PI/180;
userCoord.wayPointArray[2].jointPos[2] = -101.108547*M_PI/180;
userCoord.wayPointArray[2].jointPos[3] = -56.433133*M_PI/180;
userCoord.wayPointArray[2].jointPos[4] = -87.006734*M_PI/180;
userCoord.wayPointArray[2].jointPos[5] = -29.827440*M_PI/180;

userCoord.toolDesc = toolEnd;

//旋转轴和旋转角度
double rotateAxis[3] = {1, 0, 0};
double rotateAngle = 10.0*M_PI/180;

//旋转运动：工具末端在用户坐标系下沿 X+方向旋转 10 度，当前位置保持
不变
robotService.robotServiceRotateMove(userCoord, rotateAxis, rotateAngle, true);
}

```

4.13 轨迹运动

4.13.1 robotServiceTrackMove 函数

示例 1：圆运动

本示例是机械臂做轨迹运动之圆运动。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到初始位置 (4) 清空全局路点容器 (5) 添加路点 (6) 设置圆运动的圈数 (7) 做轨迹运动之圆运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_movetrack.cpp 中的 demo1()函数代码如下：

```
void Example_MoveTrack::demo1()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6          /*碰撞等级*/,
                                                true       /*是否允许读取位姿 默认为 true*/,
```

```

true,    /*保留默认为 true
*/
1000,    /*保留默认为 100
0 */
result); /*机械臂初始化*/

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置末端型运动最大加速度
double lineMaxAcc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineAcc(lineMaxAcc);

//设置末端型运动最大速度
double lineMaxVelc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineVelc(lineMaxVelc);

double jointAngle1[6] = {};
jointAngle1[0] = 60.443151*M_PI/180;
jointAngle1[1] = 42.275463*M_PI/180;
jointAngle1[2] = -97.679737*M_PI/180;
jointAngle1[3] = -49.990510*M_PI/180;
jointAngle1[4] = -90.007372*M_PI/180;
jointAngle1[5] = 62.567046*M_PI/180;

double jointAngle2[6] = {};
jointAngle2[0] = 83.411541*M_PI/180;
jointAngle2[1] = 39.625360*M_PI/180;
jointAngle2[2] = -103.796807*M_PI/180;
jointAngle2[3] = -53.491856*M_PI/180;
jointAngle2[4] = -90.021641*M_PI/180;
jointAngle2[5] = 85.530279*M_PI/180;

double jointAngle3[6] = {};
jointAngle3[0] = 81.206455*M_PI/180;
jointAngle3[1] = 28.381980*M_PI/180;

```

```

jointAngle3[2] = -129.233955*M_PI/180;
jointAngle3[3] = -67.700289*M_PI/180;
jointAngle3[4] = -90.019516*M_PI/180;
jointAngle3[5] = 83.325883*M_PI/180;

//关节运动
robotService.robotServiceJointMove(jointAngle1, true);

//清空全局路点容器
robotService.robotServiceClearGlobalWayPointVector();

//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle1);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle2);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle3);

//设置圆弧运动圈数
robotService.robotServiceSetGlobalCircularLoopTimes(0);

//圆弧运动
robotService.robotServiceTrackMove(aubo_robot_namespace::ARC_CIR, true);
}

```

示例 2：圆弧运动

本示例是机械臂做轨迹运动之圆弧运动。

本程序的主要流程是：（1）机械臂登录（2）机械臂初始化（3）关节运动到初始位置（4）清空全局路点容器（5）添加路点（6）设置圆运动的圈数为 0（7）做轨迹运动之圆弧运动。

注意：要将下面代码中的 IP 地址（SERVER_HOST）修改为对应服务器的 IP 地址。

example_movetrack.cpp 中的 demo2()函数代码如下：

```

void Example_MoveTrack::demo2()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
}

```

```

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cout << "登录成功" << std::endl;
}
else
{
    std::cerr << "登录失败" << std::endl;
}

/** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
aubo_robot_namespace::ROBOT_SERVICE_STATE result;

//工具动力学参数
aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学
参数**/,
                                           6           /*碰撞等级*/,
                                           true        /*是否允许读取位
姿 默认为 true*/,
                                           true,       /*保留默认为 true
*/
                                           1000,      /*保留默认为 100
0 */
                                           result); /*机械臂初始化*/
if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置末端型运动最大加速度
double lineMaxAcc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineAcc(lineMaxAcc);

//设置末端型运动最大速度
double lineMaxVelc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineVelc(lineMaxVelc);

```

```

double jointAngle1[6] = {};
jointAngle1[0] = 60.443151*M_PI/180;
jointAngle1[1] = 42.275463*M_PI/180;
jointAngle1[2] = -97.679737*M_PI/180;
jointAngle1[3] = -49.990510*M_PI/180;
jointAngle1[4] = -90.007372*M_PI/180;
jointAngle1[5] = 62.567046*M_PI/180;

double jointAngle2[6] = {};
jointAngle2[0] = 83.411541*M_PI/180;
jointAngle2[1] = 39.625360*M_PI/180;
jointAngle2[2] = -103.796807*M_PI/180;
jointAngle2[3] = -53.491856*M_PI/180;
jointAngle2[4] = -90.021641*M_PI/180;
jointAngle2[5] = 85.530279*M_PI/180;

double jointAngle3[6] = {};
jointAngle3[0] = 81.206455*M_PI/180;
jointAngle3[1] = 28.381980*M_PI/180;
jointAngle3[2] = -129.233955*M_PI/180;
jointAngle3[3] = -67.700289*M_PI/180;
jointAngle3[4] = -90.019516*M_PI/180;
jointAngle3[5] = 83.325883*M_PI/180;

//关节运动
robotService.robotServiceJointMove(jointAngle1, true);

//清空全局路点容器
robotService.robotServiceClearGlobalWayPointVector();

//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle1);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle2);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle3);

//设置圆运动圈数
robotService.robotServiceSetGlobalCircularLoopTimes(3);

//圆运动
robotService.robotServiceTrackMove(aubo_robot_namespace::ARC_CIR, true);
}

```

示例 3：MOVEP

本示例是机械臂做轨迹运动之 MOVEP 运动。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 关节运动到初始位置 (4) 清空全局路点容器 (5) 添加路点 (6) 设置交融半径 (7) 做轨迹运动之 MOVEP 运动。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_movetrack.cpp 中的 demo3()函数代码如下：

```
void Example_MoveTrack::demo3()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用：登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
```

```

if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置末端型运动最大加速度
double lineMaxAcc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineAcc(lineMaxAcc);

//设置末端型运动最大速度
double lineMaxVelc = 0.2;
robotService.robotServiceSetGlobalMoveEndMaxLineVelc(lineMaxVelc);

double jointAngle1[6] = {};
jointAngle1[0] = 60.443151*M_PI/180;
jointAngle1[1] = 42.275463*M_PI/180;
jointAngle1[2] = -97.679737*M_PI/180;
jointAngle1[3] = -49.990510*M_PI/180;
jointAngle1[4] = -90.007372*M_PI/180;
jointAngle1[5] = 62.567046*M_PI/180;

double jointAngle2[6] = {};
jointAngle2[0] = 83.411541*M_PI/180;
jointAngle2[1] = 39.625360*M_PI/180;
jointAngle2[2] = -103.796807*M_PI/180;
jointAngle2[3] = -53.491856*M_PI/180;
jointAngle2[4] = -90.021641*M_PI/180;
jointAngle2[5] = 85.530279*M_PI/180;

double jointAngle3[6] = {};
jointAngle3[0] = 81.206455*M_PI/180;
jointAngle3[1] = 28.381980*M_PI/180;
jointAngle3[2] = -129.233955*M_PI/180;
jointAngle3[3] = -67.700289*M_PI/180;
jointAngle3[4] = -90.019516*M_PI/180;
jointAngle3[5] = 83.325883*M_PI/180;

```



```

//关节运动
robotService.robotServiceJointMove(jointAngle1, true);

//清空全局路点容器
robotService.robotServiceClearGlobalWayPointVector();

//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle1);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle2);
//添加路点
robotService.robotServiceAddGlobalWayPoint(jointAngle3);

//设置交融半径
float blendradius = 0.05;
std::cout << "set blendradius ret is " << robotService.robotServiceSetGlobalBlendRadius(blendradius) <<
std::endl;

//MoveP 运动
robotService.robotServiceTrackMove(aubo_robot_namespace::CARTESIAN_MOVEP, true);
}

```

4.14 示教运动

本示例是法兰盘中心在基坐标系下做关节示教、位置示教和姿态示教。

本程序的主要流程是：(1) 机械臂登录 (2) 机械臂初始化 (3) 初始化运动属性 (4) 设置最大速度和最大加速度 (5) 设置基坐标系 (6) JOINT1 关节示教 (7) MOV_X 位置示教 (8) ROT_Z 姿态示教。

注意：要将下面代码中的 IP 地址 (SERVER_HOST) 修改为对应服务器的 IP 地址。

example_teachmove.cpp 中的 demo ()函数代码如下：

```

#include "example_teachmove.h"
#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>

```

```

#include <fstream>

#define SERVER_HOST "192.168.221.13"
#define SERVER_PORT 8899

Example_TeachMove::Example_TeachMove()
{

}

void Example_TeachMove::demo()
{
    ServiceInterface robotService;
    int ret = aubo_robot_namespace::InterfaceCallSuccCode;

    /** 接口调用: 登录 ***/
    ret = robotService.robotServiceLogin(SERVER_HOST, SERVER_PORT, "aubo", "123456");
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)
    {
        std::cout << "登录成功" << std::endl;
    }
    else
    {
        std::cerr << "登录失败" << std::endl;
    }

    /** 如果是连接真实机械臂，需要对机械臂进行初始化 **/
    aubo_robot_namespace::ROBOT_SERVICE_STATE result;

    //工具动力学参数
    aubo_robot_namespace::ToolDynamicsParam toolDynamicsParam;
    memset(&toolDynamicsParam, 0, sizeof(toolDynamicsParam));
    ret = robotService.rootServiceRobotStartup(toolDynamicsParam/**工具动力学参数**/,
                                                6           /*碰撞等级*/,
                                                true        /*是否允许读取位姿 默认为 true*/,
                                                true,       /*保留默认为 true */
                                                1000,      /*保留默认为 1000 */
                                                result); /*机械臂初始化*/
    if(ret == aubo_robot_namespace::InterfaceCallSuccCode)

```

```

{
    std::cerr << "机械臂初始化成功." << std::endl;
}
else
{
    std::cerr << "机械臂初始化失败." << std::endl;
}

//初始化运动属性
robotService.robotServiceInitGlobalMoveProfile();

//设置关节运动最大加速度
aubo_robot_namespace::JointVelcAccParam jointMaxAcc;
jointMaxAcc.jointPara[0] = 30*M_PI/180;
jointMaxAcc.jointPara[1] = 30*M_PI/180;
jointMaxAcc.jointPara[2] = 30*M_PI/180;
jointMaxAcc.jointPara[3] = 30*M_PI/180;
jointMaxAcc.jointPara[4] = 30*M_PI/180;
jointMaxAcc.jointPara[5] = 30*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxAcc(jointMaxAcc);

//设置关节运动最大速度
aubo_robot_namespace::JointVelcAccParam jointMaxVelc;
jointMaxVelc.jointPara[0] = 15*M_PI/180;
jointMaxVelc.jointPara[1] = 15*M_PI/180;
jointMaxVelc.jointPara[2] = 15*M_PI/180;
jointMaxVelc.jointPara[3] = 15*M_PI/180;
jointMaxVelc.jointPara[4] = 15*M_PI/180;
jointMaxVelc.jointPara[5] = 15*M_PI/180;
robotService.robotServiceSetGlobalMoveJointMaxVelc(jointMaxVelc);

//设置示教坐标系为基坐标系
aubo_robot_namespace::CoordCalibrateByJointAngleAndTool baseCoord;
robotService.robotServiceSetTeachCoordinateSystem(baseCoord);

//设置示教模式——关节示教
aubo_robot_namespace::teach_mode teachmode1;
teachmode1 = aubo_robot_namespace::JOINT1;

//开始示教
robotService.robotServiceTeachStart(teachmode1, true);
sleep(2);

//结束示教

```

```

robotService.robotServiceTeachStop();

//设置示教模式——位置示教
aubo_robot_namespace::teach_mode teachmode2;
teachmode2 = aubo_robot_namespace::MOV_X;

//开始示教
robotService.robotServiceTeachStart(teachmode2, true);
sleep(5);
//结束示教
robotService.robotServiceTeachStop();

//设置示教模式——姿态示教
aubo_robot_namespace::teach_mode teachmode3;
teachmode3 = aubo_robot_namespace::ROT_Z;

//开始示教
robotService.robotServiceTeachStart(teachmode3, true);
sleep(5);
//结束示教
robotService.robotServiceTeachStop();
}

```

4.15 打印路点信息、关节状态信息、事件信息、诊断信息

本程序包括：打印路点信息的函数 `printWaypoint()`、打印关节状态信息 `printJointStatus()`、打印事件信息 `printEventInfo()`、打印诊断信息 `printRobotDiagnosis()`、初始化关节角数组 `initJointAngleArray()`。

注意：手册中的某些使用案例可能会用到 `util.h`，以便打印路点信息或者打印关节状态信息或者打印事件信息等等。

`util.h` 的代码如下：

```

#ifndef UTIL_H
#define UTIL_H

#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

class Util
{
public:
    Util();

```

```

public:
    /** 打印路点信息 */
    static void printWaypoint(aubo_robot_namespace::wayPoint_S &wayPoint);

    /** 打印关节状态信息 */
    static void printJointStatus(const aubo_robot_namespace::JointStatus *jointStatus, int len);

    /** 打印事件信息 */
    static void printEventInfo(const aubo_robot_namespace::RobotEventInfo &eventInfo);

    /** 打印诊断信息 */
    static void printRobotDiagnosis(const aubo_robot_namespace::RobotDiagnosis &robotDiagnosis);

    static void initJointAngleArray(double *array, double joint0, double joint1, double joint2, double joint3, double joint4, double joint5);

};

#endif // UTIL_H

```

util.cpp 的代码如下：

```

#include "util.h"

#include "AuboRobotMetaType.h"
#include "serviceinterface.h"

#include <unistd.h>
#include <math.h>
#include <string.h>
#include <stdio.h>
#include <sstream>
#include <fstream>

Util::Util()
{

}

//打印路点信息
void Util::printWaypoint(aubo_robot_namespace::wayPoint_S &wayPoint)
{

```

```

std::cout<<std::endl<<"start-----路点信息-----"<<std::endl;
//位置信息
std::cout<<"位置信息: ";
std::cout<<"x:"<<wayPoint.cartPos.position.x<<" ";
std::cout<<"y:"<<wayPoint.cartPos.position.y<<" ";
std::cout<<"z:"<<wayPoint.cartPos.position.z<<std::endl;

//姿态信息
std::cout<<"姿态信息: ";
std::cout<<"w:"<<wayPoint.orientation.w<<" ";
std::cout<<"x:"<<wayPoint.orientation.x<<" ";
std::cout<<"y:"<<wayPoint.orientation.y<<" ";
std::cout<<"z:"<<wayPoint.orientation.z<<std::endl;

ServiceInterface robotService;
aubo_robot_namespace::Rpy tempRpy;
robotService.quaternionToRPY(wayPoint.orientation,tempRpy);
std::cout<<"RX:"<<tempRpy.rx*180/M_PI<<" RY:"<<tempRpy.ry*180/M_PI<
<" RZ:"<<tempRpy.rz*180/M_PI<<std::endl;

//关节信息
std::cout<<"关节信息: "<<std::endl;
for(int i=0;i<aubo_robot_namespace::ARM_DOF;i++)
{
    std::cout<<"joint"<<i+1<<": "<<wayPoint.jointpos[i]<<" ~ "<<wayPoint.joi
ntpos[i]*180.0/M_PI<<std::endl;
}
}

//打印关节状态信息
void Util::printJointStatus(const aubo_robot_namespace::JointStatus *jointStatus, int
len)
{
    std::cout<<std::endl<<"start-----关节状态信息-----" << std::endl;

    for(int i=0; i<len; i++)
    {
        std::cout<<"关节 ID:" <<i<<" ";
        std::cout<<"电流:" <<jointStatus[i].jointCurrentI<<" ";
        std::cout<<"速度:" <<jointStatus[i].jointSpeedMoto<<" ";
        std::cout<<"关节角:" <<jointStatus[i].jointPosJ<<" "<<" ~ "<<jointStatu
s[i].jointPosJ*180.0/M_PI;
        std::cout<<"电压 :<<jointStatus[i].jointCurVol<<" ";
    }
}

```

```

        std::cout<<"温度    :" <<jointStatus[i].jointCurTemp<<" ";
        std::cout<<"目标电流:" <<jointStatus[i].jointTagCurrentI<<" ";
        std::cout<<"目标电机速度:" <<jointStatus[i].jointTagSpeedMoto<<" ";
        std::cout<<"目标关节角 :" <<jointStatus[i].jointTagPosJ<<" ";
        std::cout<<"关节错误    :" <<jointStatus[i].jointErrorNum <<std::endl;
    }
    std::cout<<std::endl;
}

//打印事件信息
void Util::printEventInfo(const aubo_robot_namespace::RobotEventInfo &eventInfo)
{
    std::cout<<"事件类型:"<<eventInfo.eventType <<" code:"<<eventInfo.eventCo
de<<" 内容:"<<eventInfo.eventContent<<std::endl;
}

//打印诊断信息
void Util::printRobotDiagnosis(const aubo_robot_namespace::RobotDiagnosis &robo
tDiagnosis)
{
    std::cout<<std::endl<<"start-----机械臂诊断信息-----" << std::endl;

    std::cout<<std::endl<<"    "<<"CAN 通信状态:"<<(int)robotDiagnosis.armCanbu
sStatus;
    std::cout<<std::endl<<"    "<<"电源当前电流:"<<robotDiagnosis.armPowerCurr
ent;
    std::cout<<std::endl<<"    "<<"电源当前电压:"<<robotDiagnosis.armPowerVolt
age;

    (robotDiagnosis.armPowerStatus)? std::cout<<std::endl<<"    "<<"48V 电源状
态:开":std::cout<<std::endl<<"    "<<"48V 电源状态:关";

    std::cout<<std::endl<<"    "<<"控制箱温度:"<<(int)robotDiagnosis.contorllerTe
mp;
    std::cout<<std::endl<<"    "<<"控制箱湿度:"<<(int)robotDiagnosis.contorllerHu
midity;
    std::cout<<std::endl<<"    "<<"远程关机信号:"<<robotDiagnosis.remoteHalt;
    std::cout<<std::endl<<"    "<<"机械臂软急停:"<<robotDiagnosis.softEmergenc
y;
    std::cout<<std::endl<<"    "<<"远程急停信号:"<<robotDiagnosis.remoteEmerge
ncy;
    std::cout<<std::endl<<"    "<<"碰撞检测位:"<<robotDiagnosis.robotCollision;
    std::cout<<std::endl<<"    "<<"进入力控模式标志位:"<<robotDiagnosis.forceCo

```

```

ntrolMode;
    std::cout<<std::endl<<"    "<<"刹车状态:"<<robotDiagnosis.brakeStatus;
    std::cout<<std::endl<<"    "<<"末端速度:"<<robotDiagnosis.robotEndSpeed;
    std::cout<<std::endl<<"    "<<"最大加速度:"<<robotDiagnosis.robotMaxAcc;
    std::cout<<std::endl<<"    "<<"上位机软件状态位:"<<robotDiagnosis.orpeStatus;
s;
    std::cout<<std::endl<<"    "<<"位姿读取使能位:"<<robotDiagnosis.enableReadPose;
PoseChanged;
    std::cout<<std::endl<<"    "<<"安装位置状态:"<<robotDiagnosis.robotMountingPoseChanged;
    std::cout<<std::endl<<"    "<<"磁编码器错误状态:"<<robotDiagnosis.encoderErrorStatus;
    std::cout<<std::endl<<"    "<<"静止碰撞检测开关:"<<robotDiagnosis.staticCollisionDetect;
    std::cout<<std::endl<<"    "<<"关节碰撞检测:"<<robotDiagnosis.jointCollisionDetect;
    std::cout<<std::endl<<"    "<<"光电编码器不一致错误:"<<robotDiagnosis.encoderLinesError;
    std::cout<<std::endl<<"    "<<"关节错误状态:"<<robotDiagnosis.jointErrorStatus;
s;
    std::cout<<std::endl<<"    "<<"奇异点过速警告:"<<robotDiagnosis.singularityOverSpeedAlarm;
    std::cout<<std::endl<<"    "<<"电流错误警告:"<<robotDiagnosis.robotCurrentAlarm;
    std::cout<<std::endl<<"    "<<"tool error:"<<(int)robotDiagnosis.toolIoError;
    std::cout<<std::endl<<"    "<<"安装位置错位:"<<robotDiagnosis.robotMountingPoseWarning;
    std::cout<<std::endl<<"    "<<"mac 缓冲器长度:"<<robotDiagnosis.macTargetPosBufferSize;
    std::cout<<std::endl<<"    "<<"mac 缓冲器有效数据长度:"<<robotDiagnosis.macTargetPosDataSize;
    std::cout<<std::endl<<"    "<<"mac 数据中断:"<<robotDiagnosis.macDataInterruptWarning;

    std::cout<<std::endl<<"-----end."<<std::endl;
}

void Util::initJointAngleArray(double *array, double joint0, double joint1, double joint2, double joint3, double joint4, double joint5)
{
    array[0] = joint0;
    array[1] = joint1;
    array[2] = joint2;
    array[3] = joint3;

```



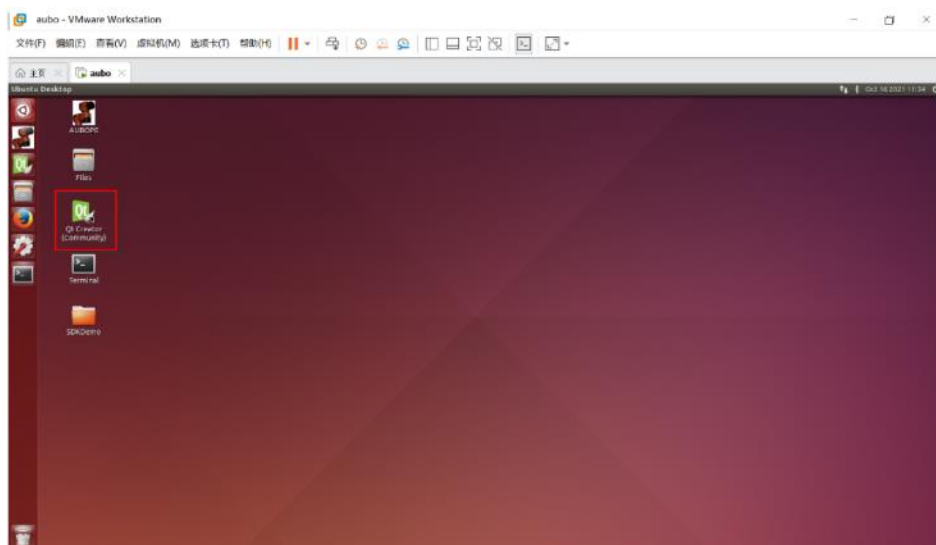
```
    array[4] = joint4;  
    array[5] = joint5;  
}
```

5 环境配置说明

注意：本手册中描述的环境配置说明是在 32 位的 AUBO 虚拟机中进行操作的。用户也可以参考此说明在 32 位或 64 位的 Linux 系统下进行 SDK 二次开发。

5.1 新建自己的程序

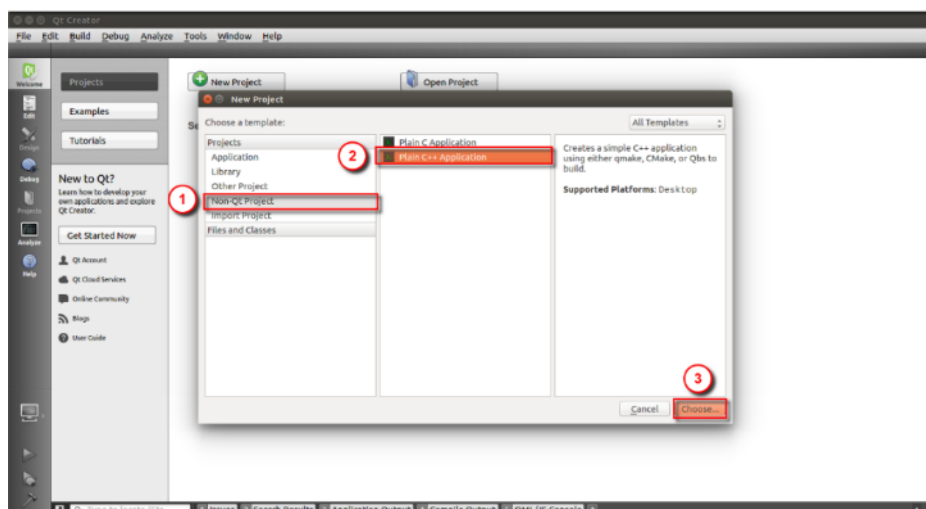
1. 打开 AUBO 虚拟机。在桌面上，打开 Qt Creator。



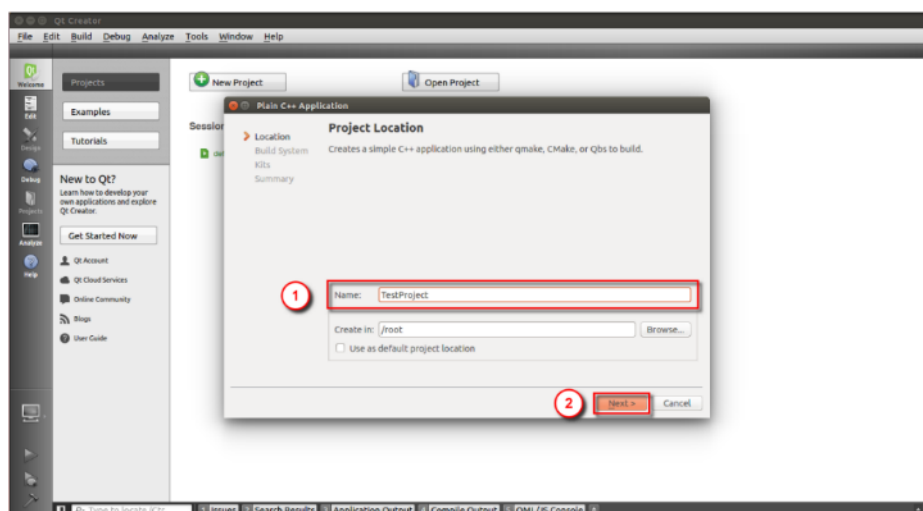
2. 在“Welcome”——“Projects”栏中，点击“New Project”。新建一个项目。



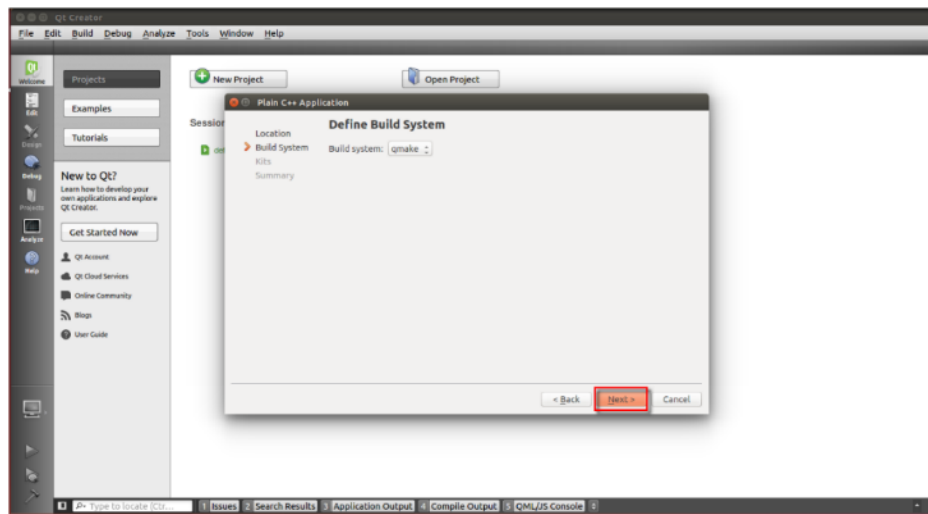
3. 选中“Non-Qt Project” —— “Plain C++ Application”，点击“Choose...”。



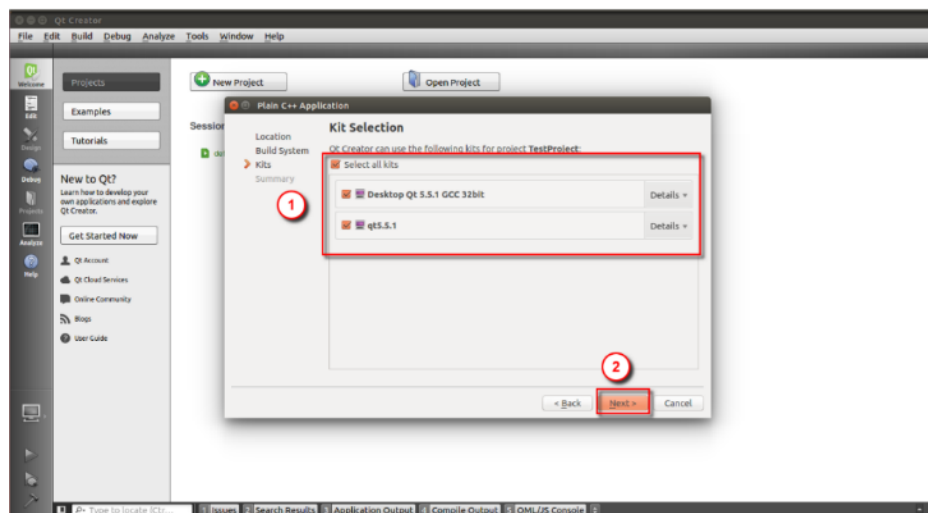
4. 项目命名为“TestProject”，点击“Next”。



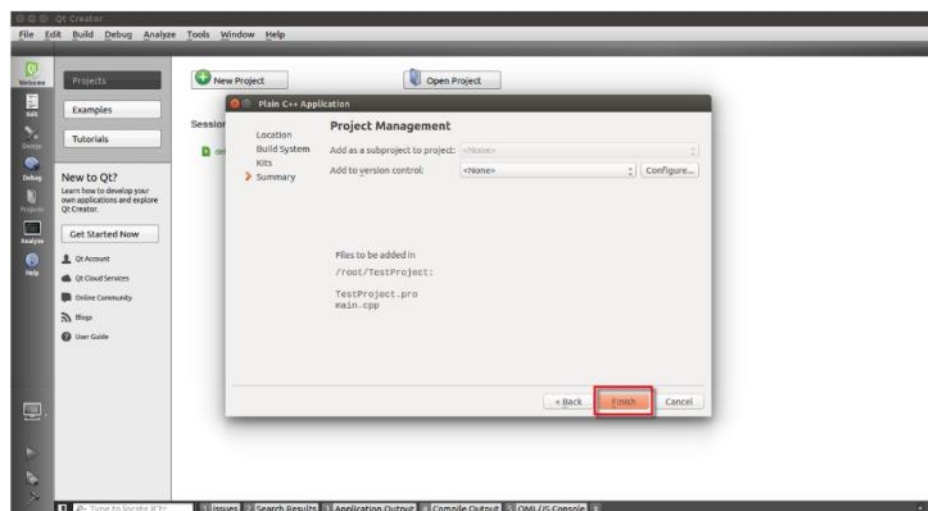
5. 点击“Next”。



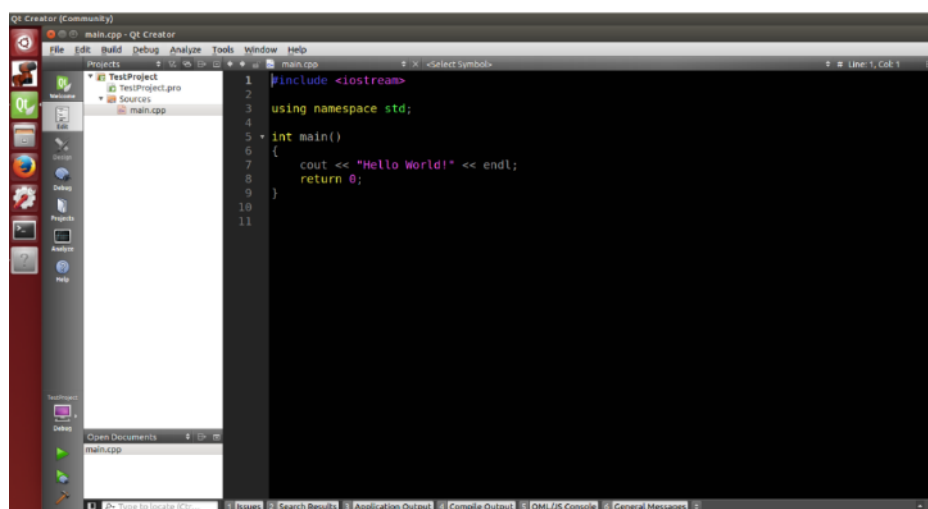
6. 点击“Select all kits”，点击“Next”。



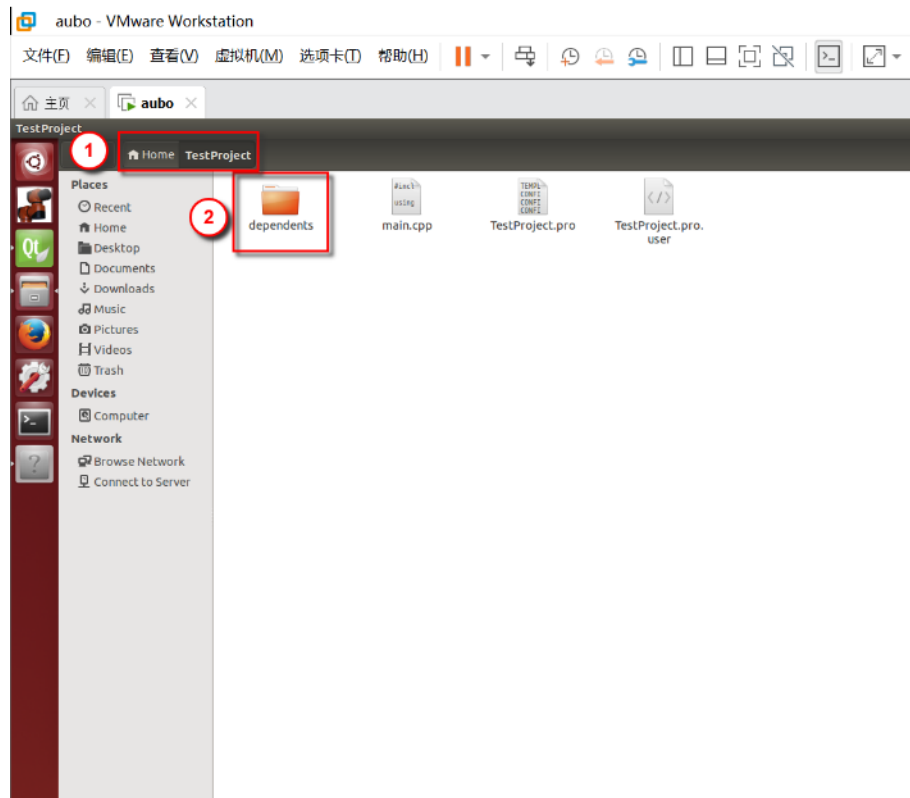
7. 点击“Finish”。



8. 界面如下。



9. 打开项目所在的文件夹（此处是《TestProject》），将 SDK 包里的 depends 文件拷贝到文件夹中。



10. 打开 TestProject.pro 文件，输入下面的代码并保存。

注意：在引入头文件和 lib 库文件时，用户需要根据实际收到的 SDK 包以及文件实际被存放的路径来进行配置。

```

1. unix {
2.     #32bit OS
3.     contains(QT_ARCH, i386) {
4.
5.         CONFIG += c++11
6.
7.         DEFINES += _GLIBCXX_USE_CXX11_ABI=0
8.
9.         INCLUDEPATH += $$PWD/depends/robotSDK/inc
10.
11.        LIBS += -L$$PWD/depends/robotSDK/lib/linux_x32/ -
            laubo_sdk
12.
13.        LIBS += -lpthread
14.    }
15.    #64bit OS
16.    contains(QT_ARCH, x86_64) {

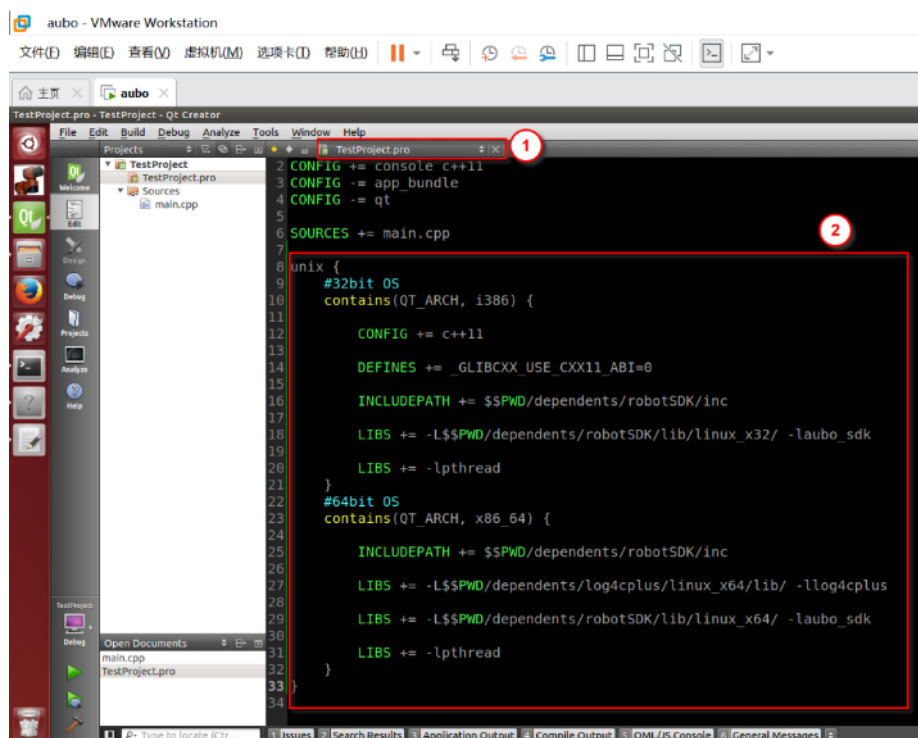
```

```

17.
18.     INCLUDEPATH += $$PWD/dependents/robotSDK/inc
19.
20.     LIBS += -L$$PWD/dependents/log4cplus/linux_x64/lib/ -
        llog4cplus
21.
22.     LIBS += -L$$PWD/dependents/robotSDK/lib/linux_x64/ -
        laubo_sdk
23.
24.     LIBS += -lpthread
25. }
26. }

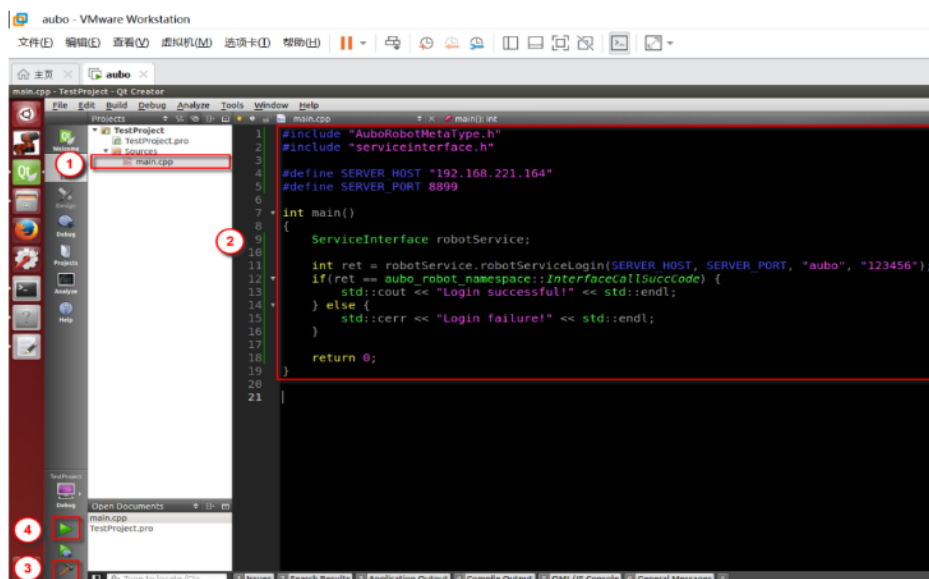
```

输入代码后，如下图所示。

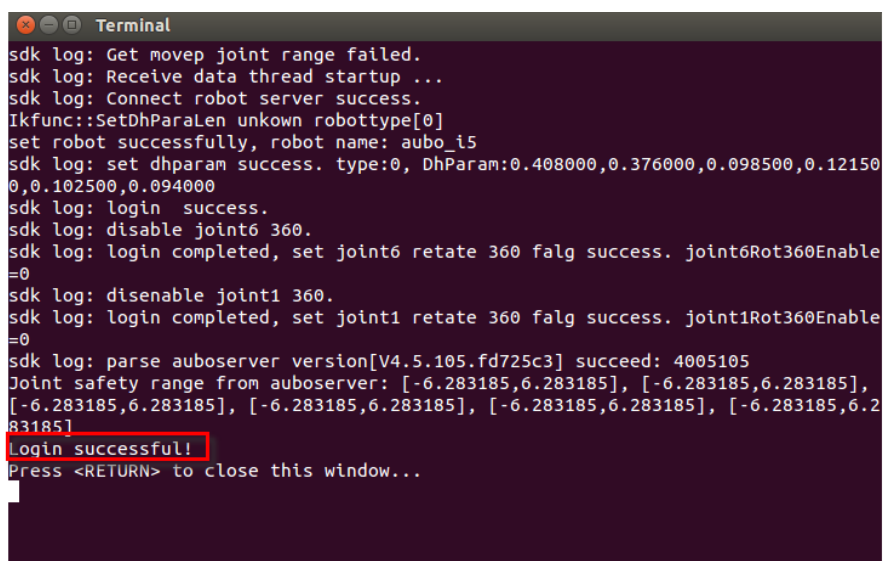


11. 在 main.cpp 中输入代码，注意 SERVER_HOST 是对应的 IP 地址。运行程序。

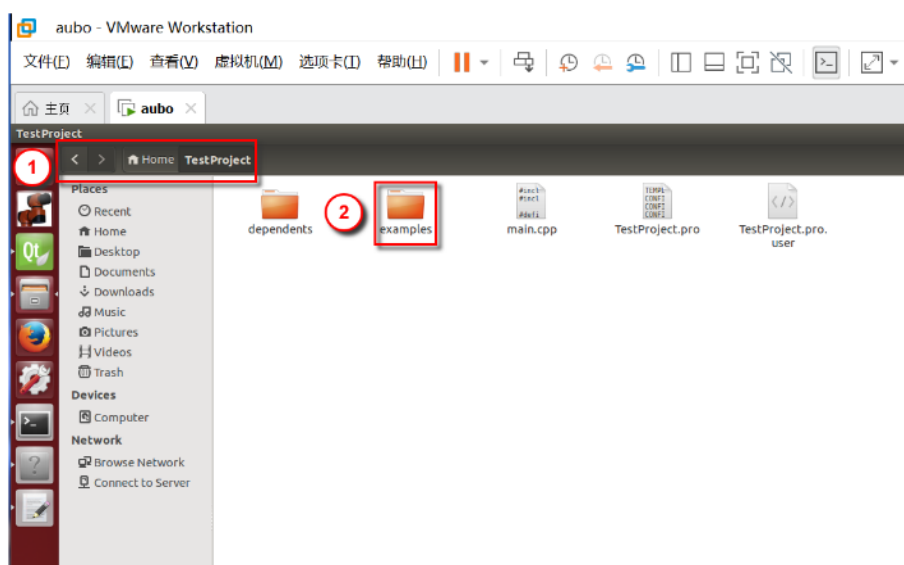
注意：运行程序前，要打开 AUBO 虚拟机中的 AUBOPE 或者连接真实机械臂。并且要正确地设置程序中的 IP 地址和端口号，以便确保机械臂登录成功。



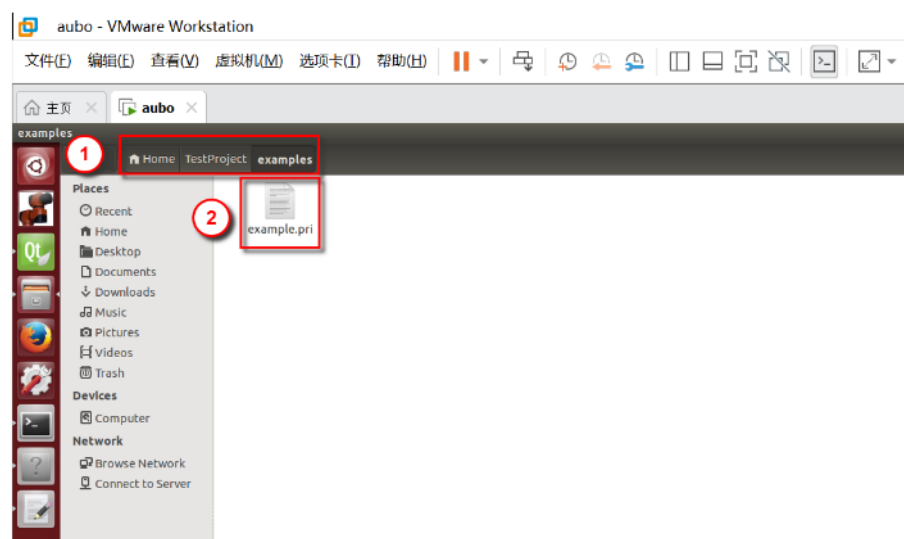
12. 运行结果显示登录成功。说明环境配置成功。



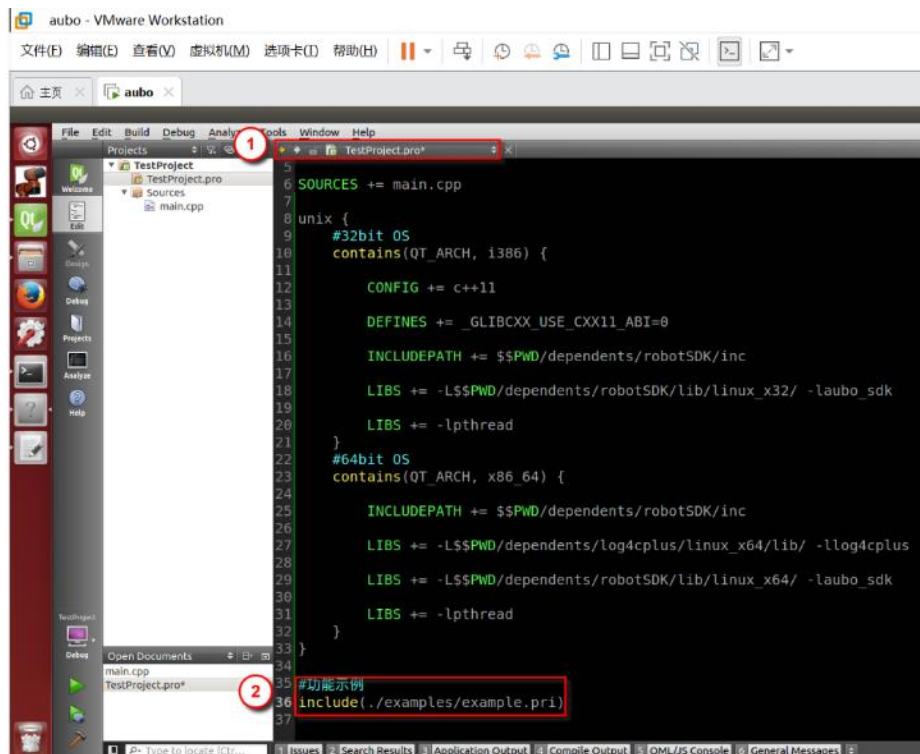
13. 在项目所在的文件夹（《TestProject》）中新建文件夹《examples》。



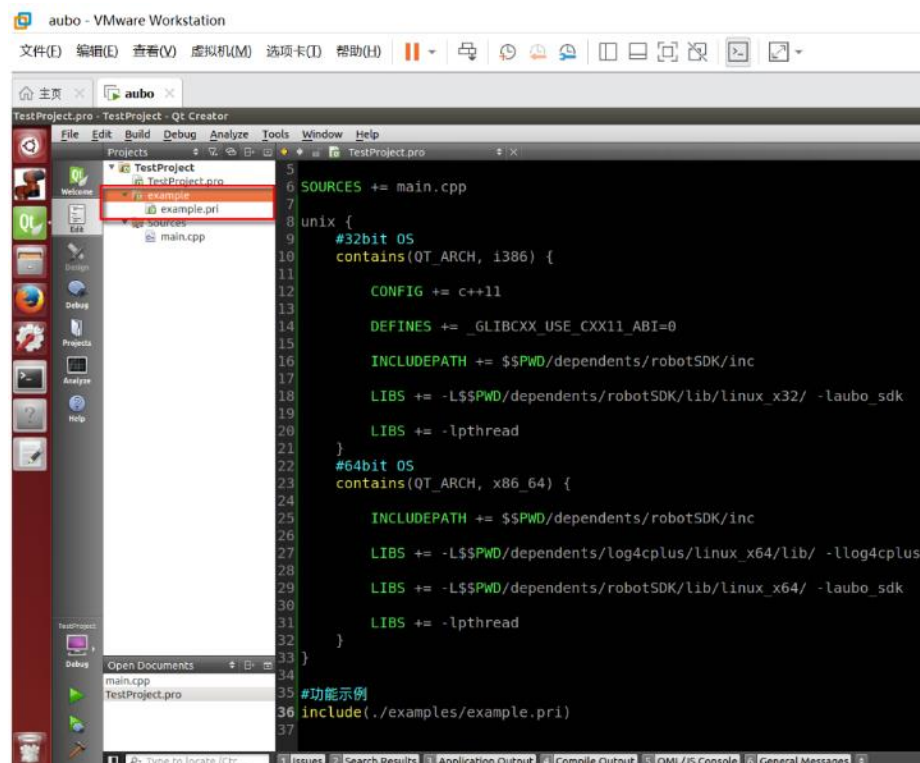
14. 在《examples》文件夹中新建一个文本，命名为《example.pri》。



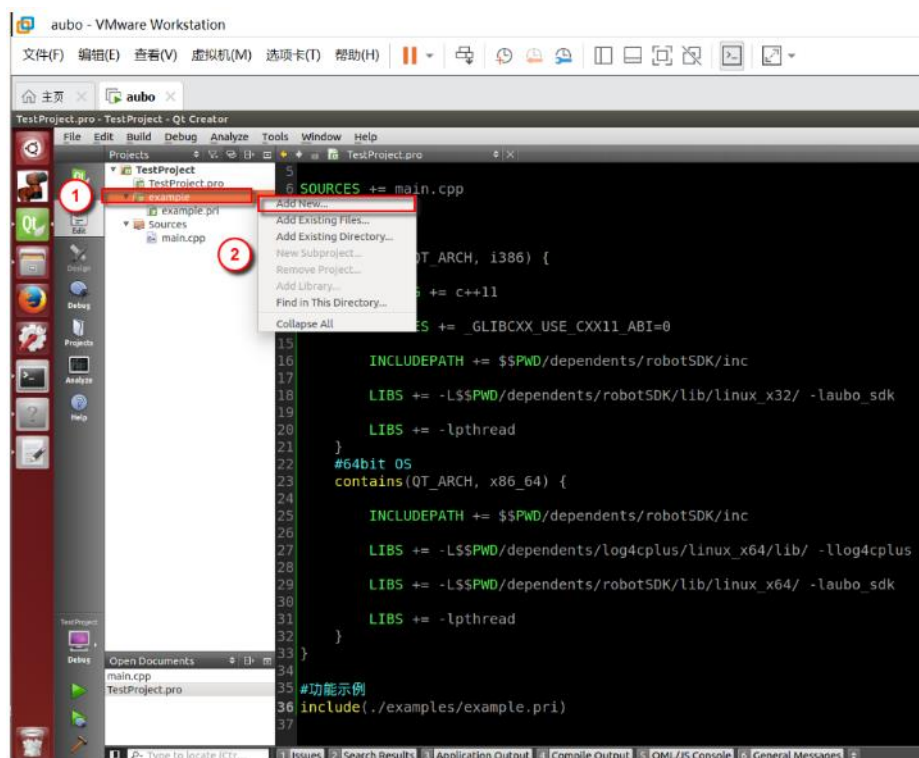
15. 打开 TestProejct.pro 文件，在《TestProject.pro》文件中添加代码，如下图所示。保存。



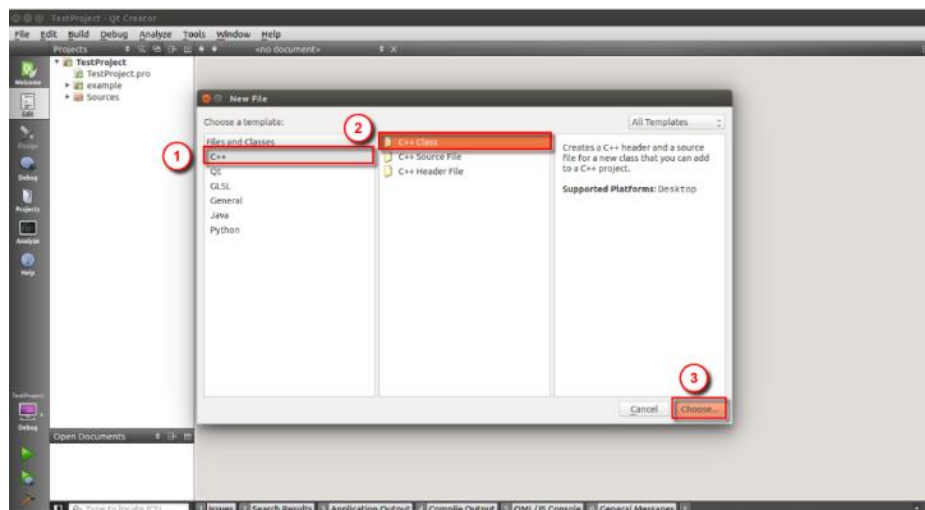
16. 然后出现如下界面。



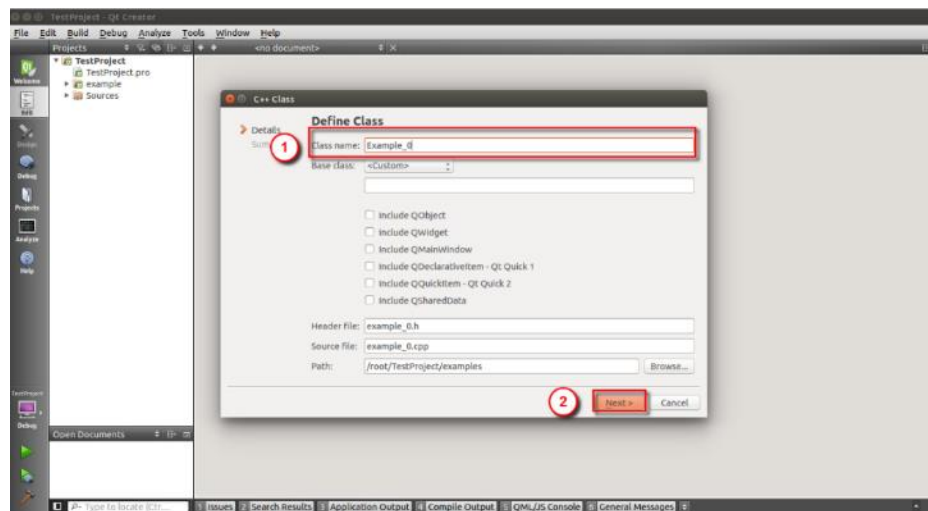
17. 选中“example”，右键，点击“Add New...”。



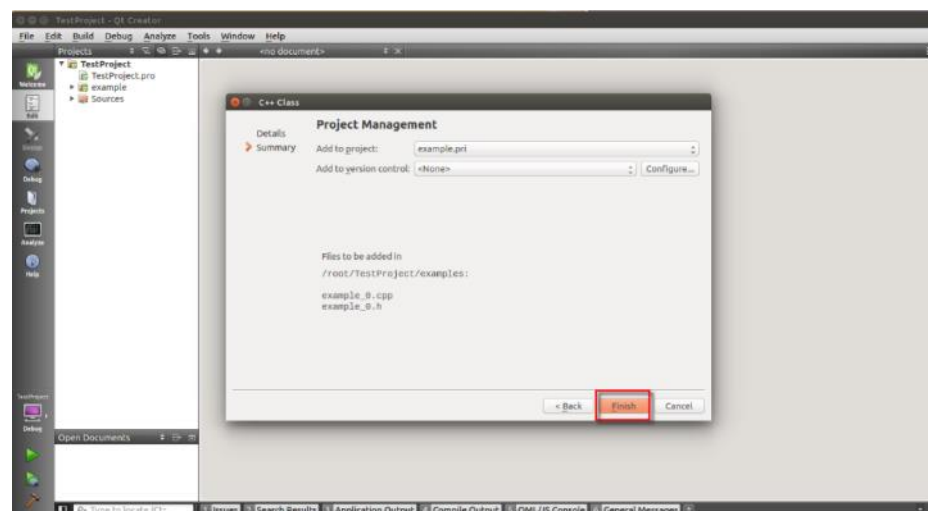
选中“C++”——“C++Class”，点击“Choose...”。



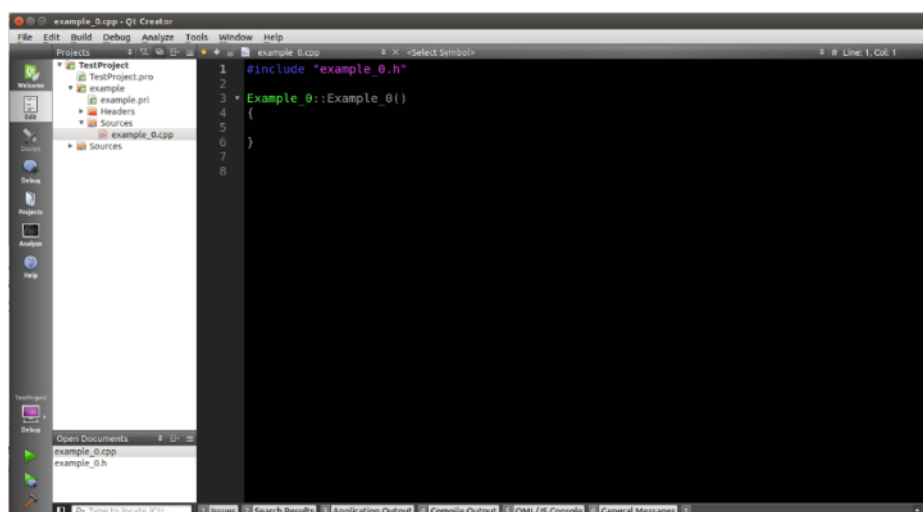
输入类名为“Example_0”，点击“Next”。



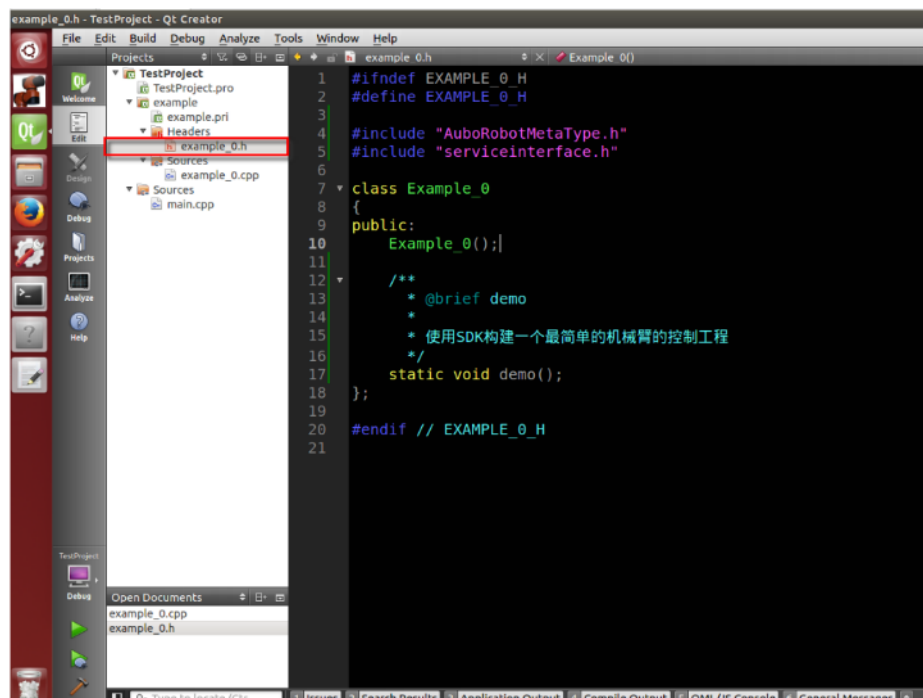
点击“Finish”。



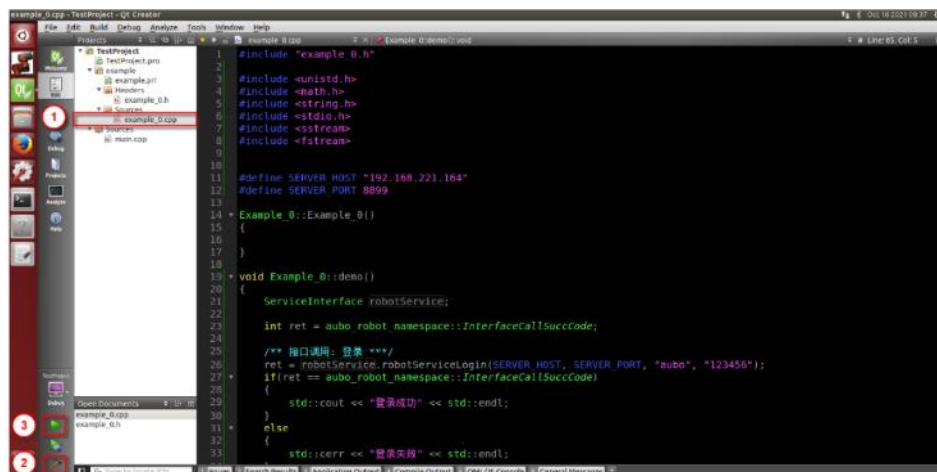
出现如下图所示。



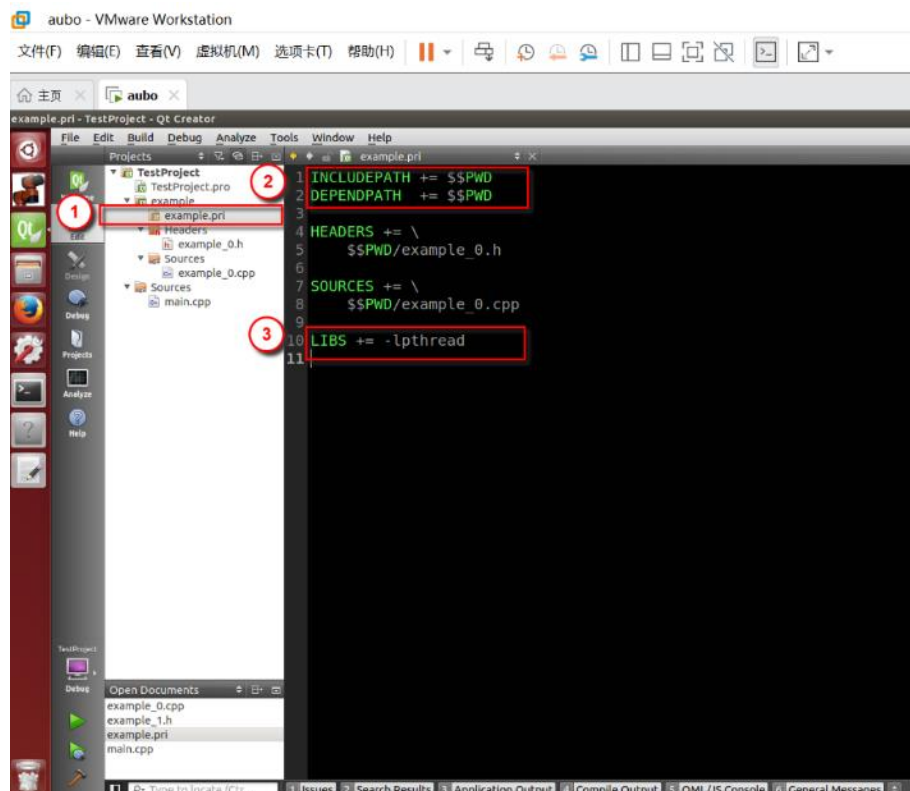
18. 选中“example_0.h”，输入如下图所示的代码。



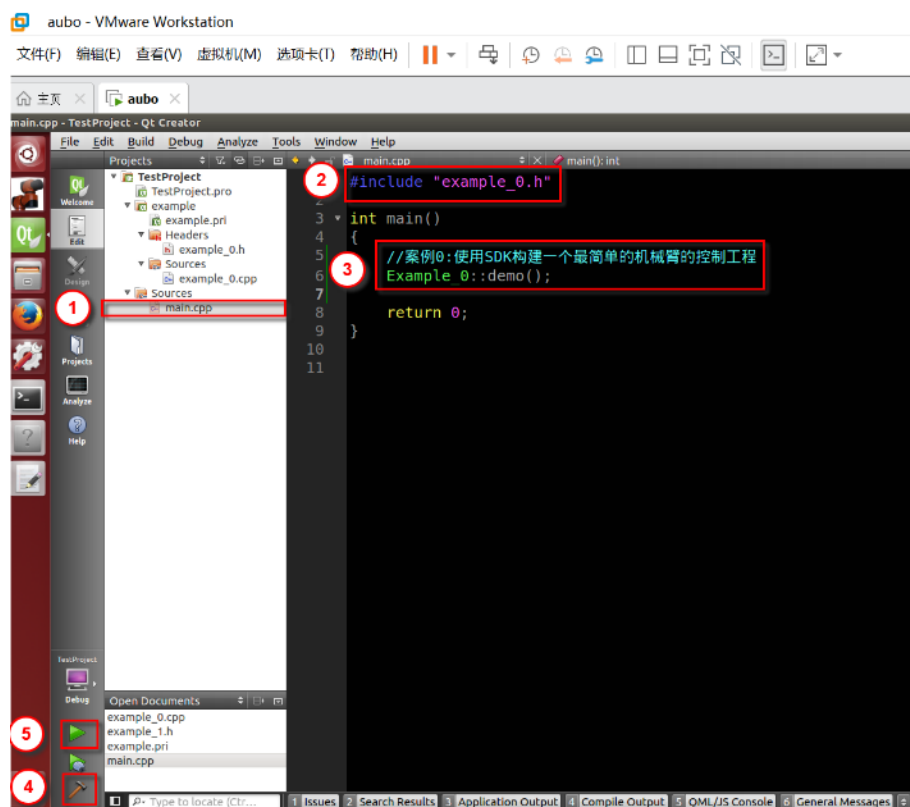
19. 选中“example_0.cpp”，输入代码。编译并运行程序。



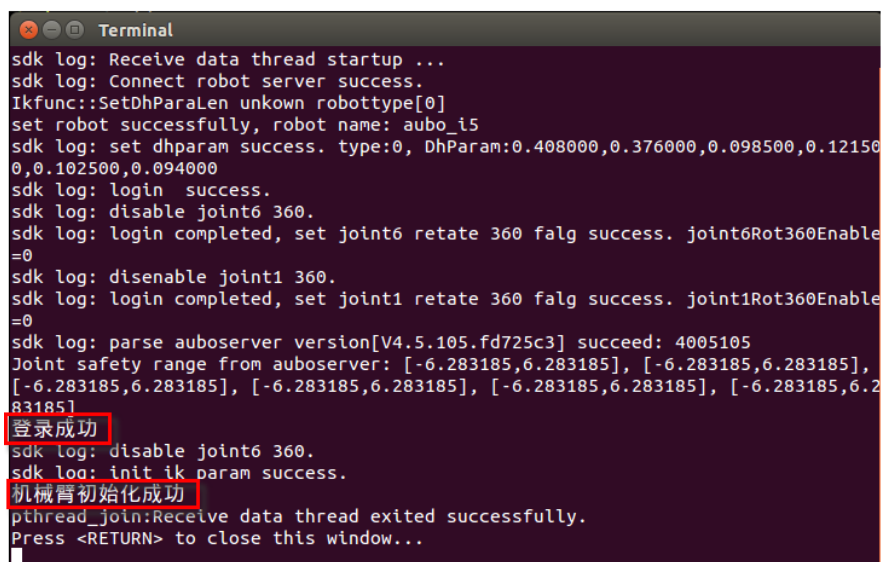
20. 在 example.pri 中添加代码。



21. 在 main.cpp 中添加代码。编译并运行程序。

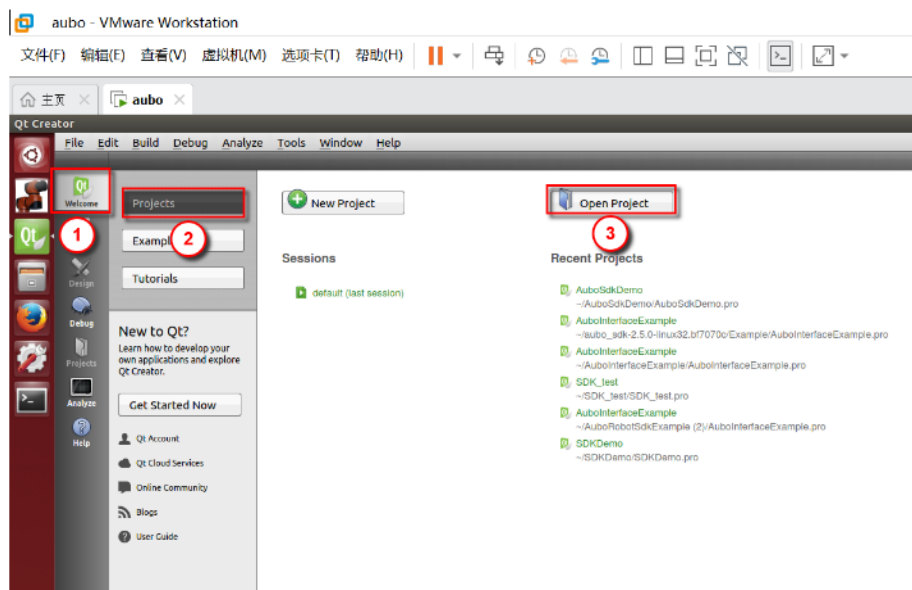


22. 运行结果如下。表明机械臂登录及初始化成功。

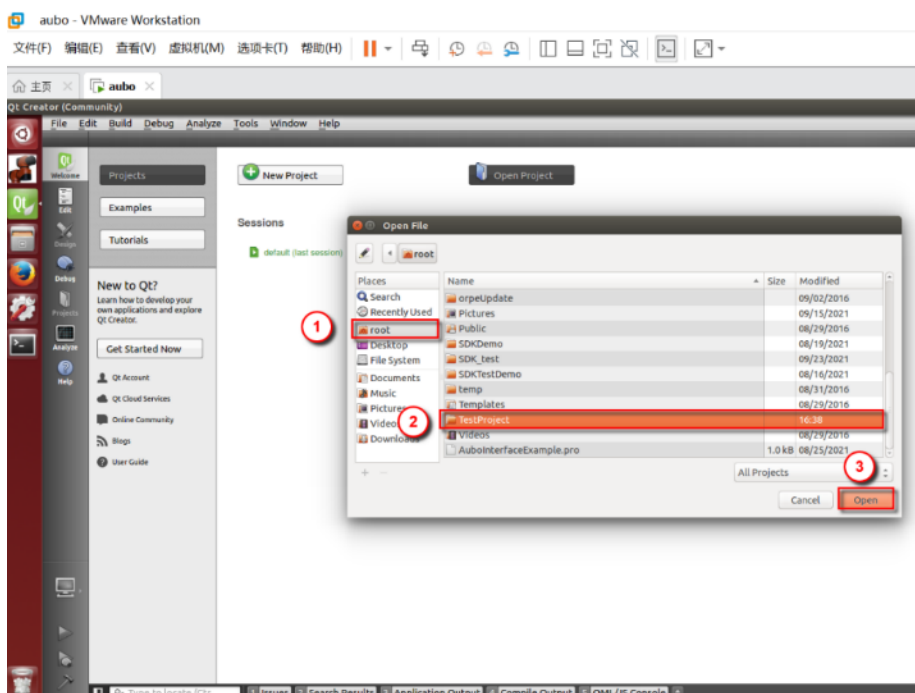


5.2 打开项目

1. 打开 Qt Creator，在“Welcome”——“Projects”栏中，点击“Open Project”。



2. 选中项目所在的文件夹，然后点击“OPEN”（文档中的项目文件夹《TestProject》保存在 root 路径下）。



3. 选中“TestProject.pro”，点击“Open”，然后项目就被打开了。

